



媒体与认知

Media and Cognition

清华大学电子工程系

王生进 李亚利

Email: liyali13@tsinghua.edu.cn

第5讲 卷积神经网络

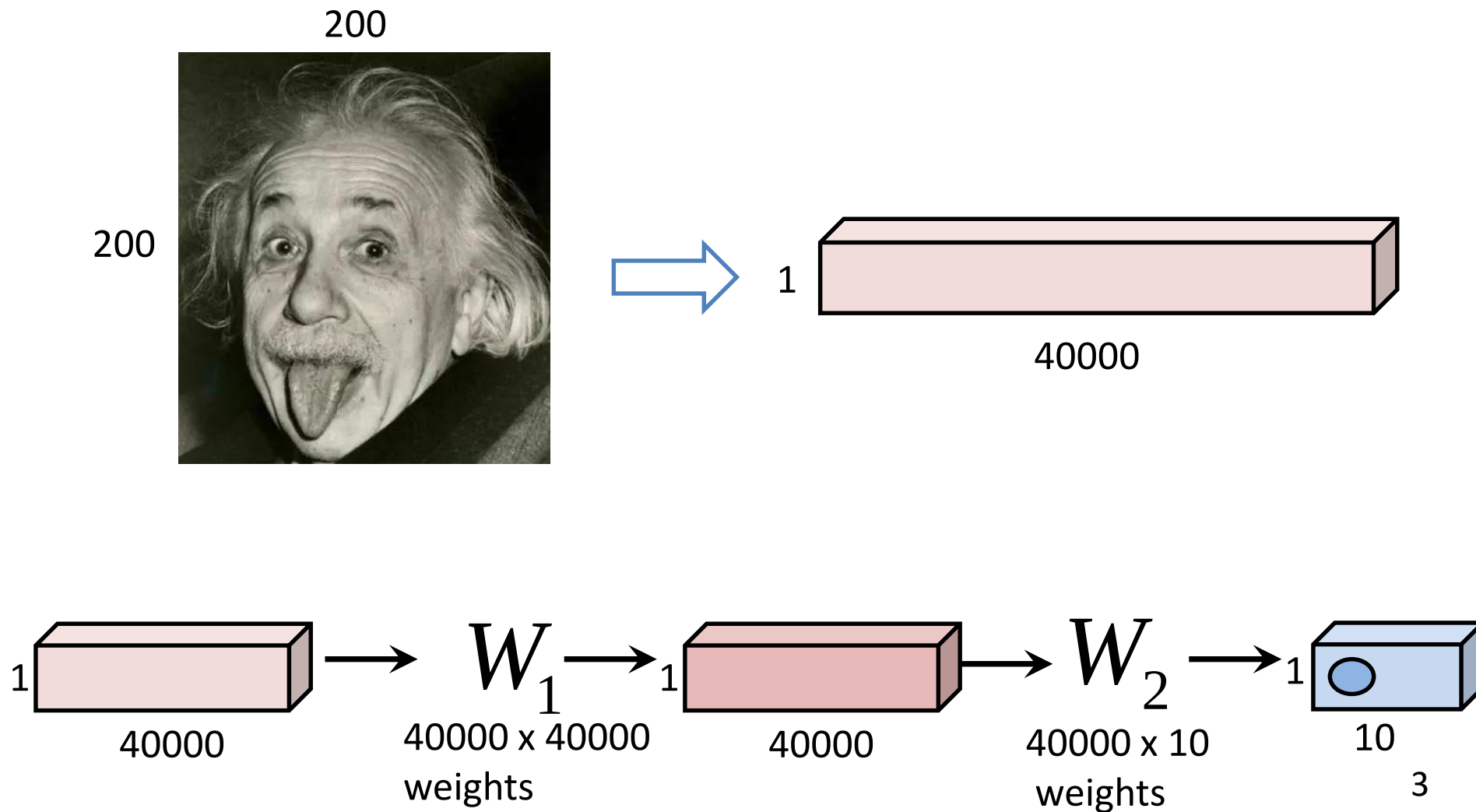
5.1 基本单元

5.2 典型卷积神经网络架构

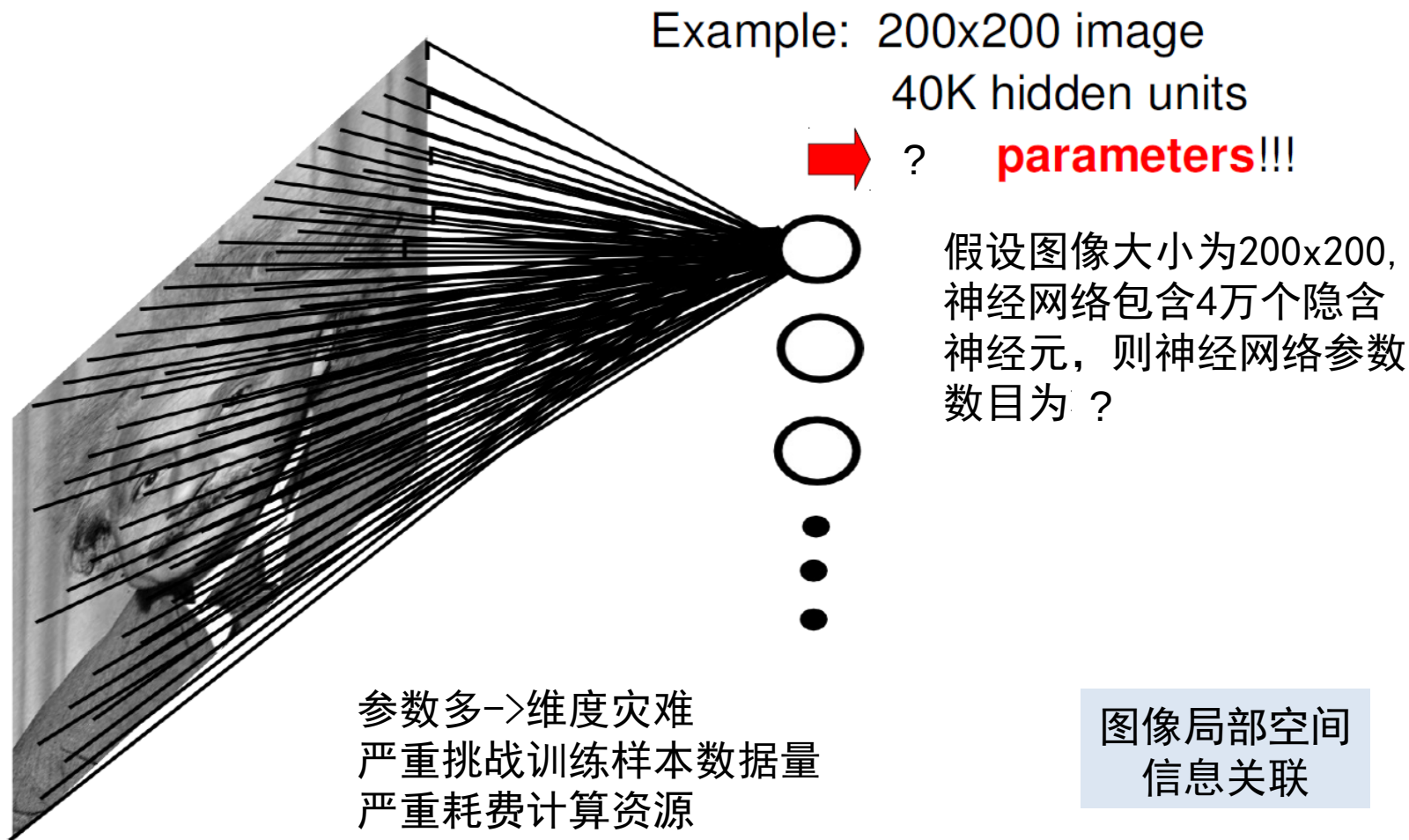
5.3 卷积神经网络应用

设计思想：为什么需要卷积神经网络？

➤ 利用全连接前馈神经网络处理图像：

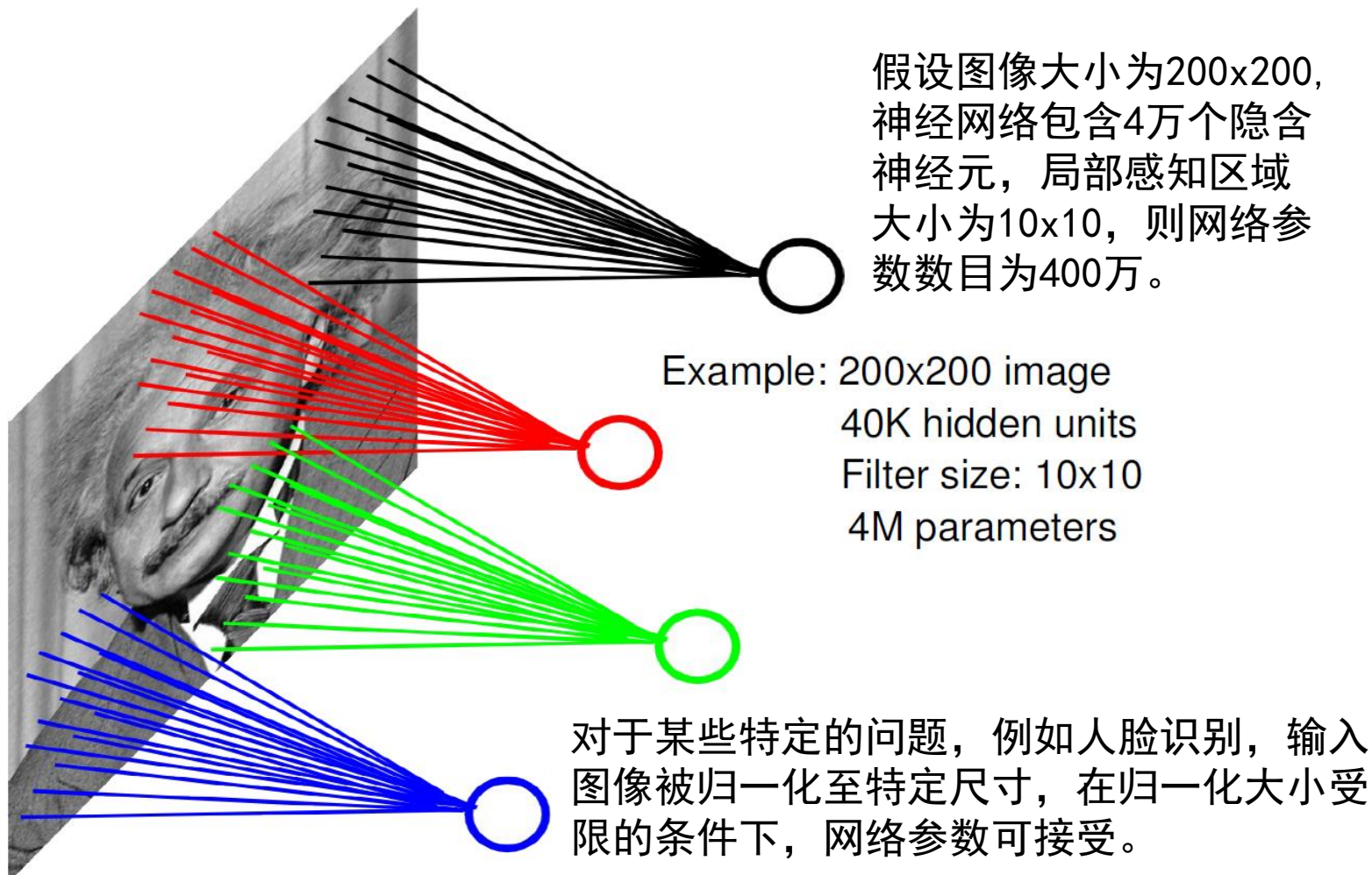


设计思想：为什么需要卷积神经网络？



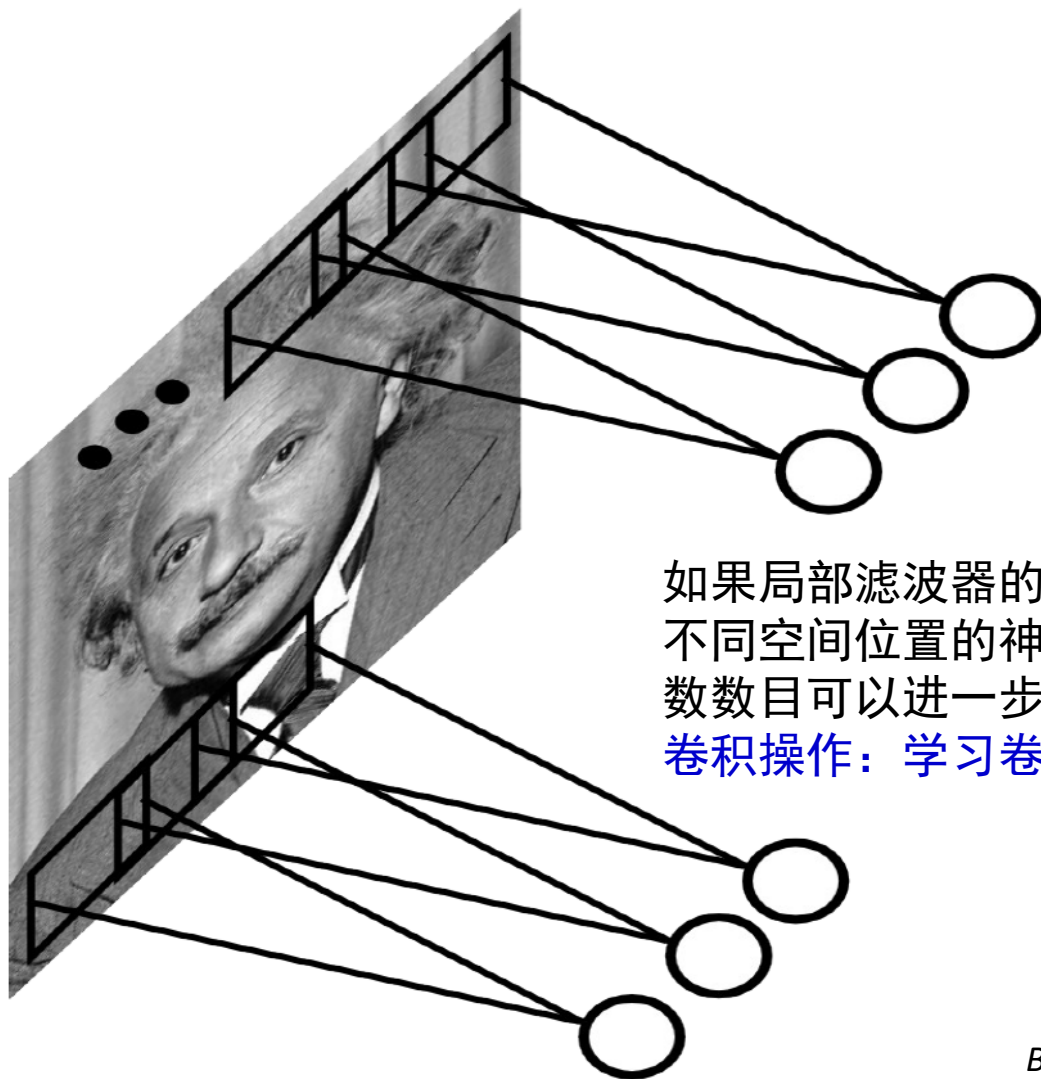
设计思想：为什么需要卷积神经网络？

➤ 局部感知



设计思想：为什么需要卷积神经网络？

➤ 权值共享



如果局部滤波器的权重是相同的，即不同空间位置的神经元共享权值，参数数目可以进一步减少；

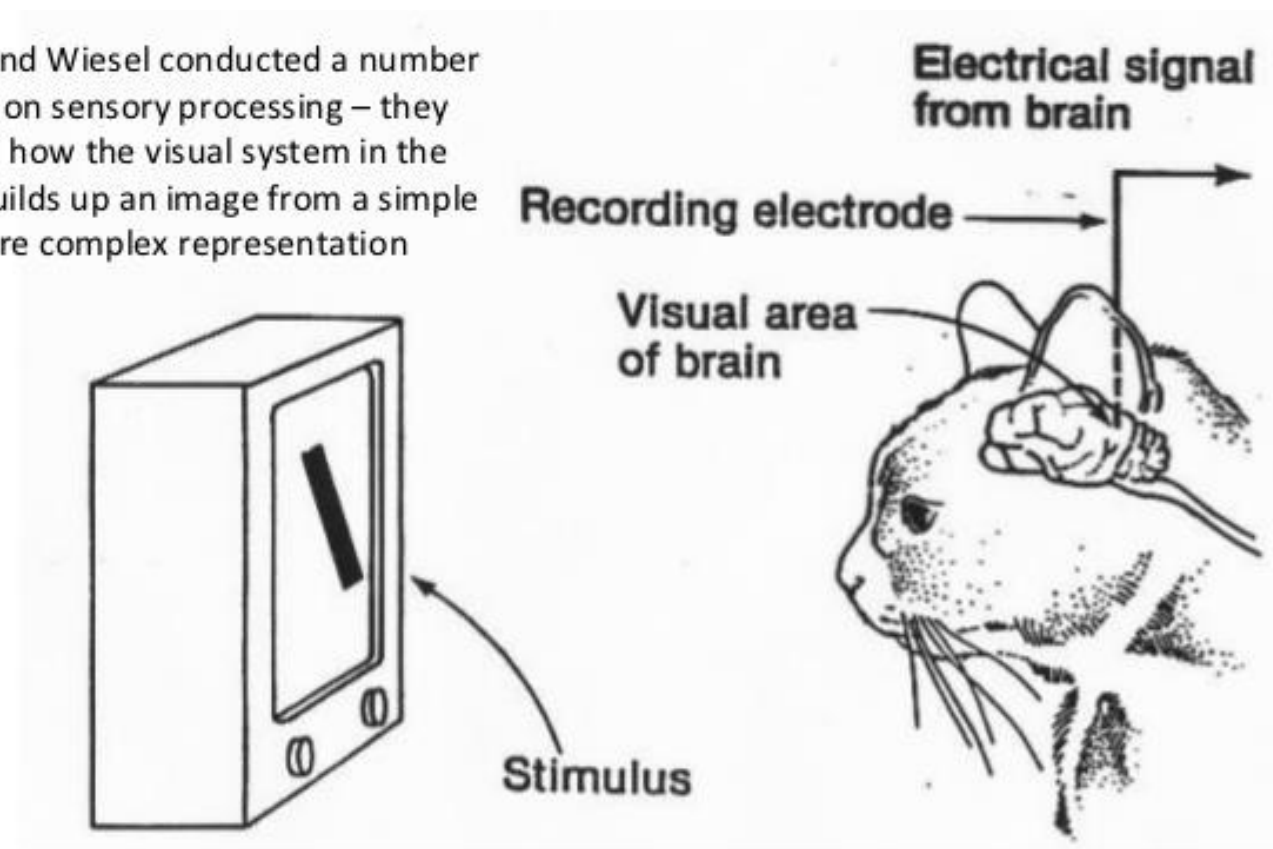
卷积操作：学习卷积核参数

神经学基础

➤ 卷积神经网络：认知神经心理学基础

◆ Hubel & Wiesel 小猫视觉神经元实验

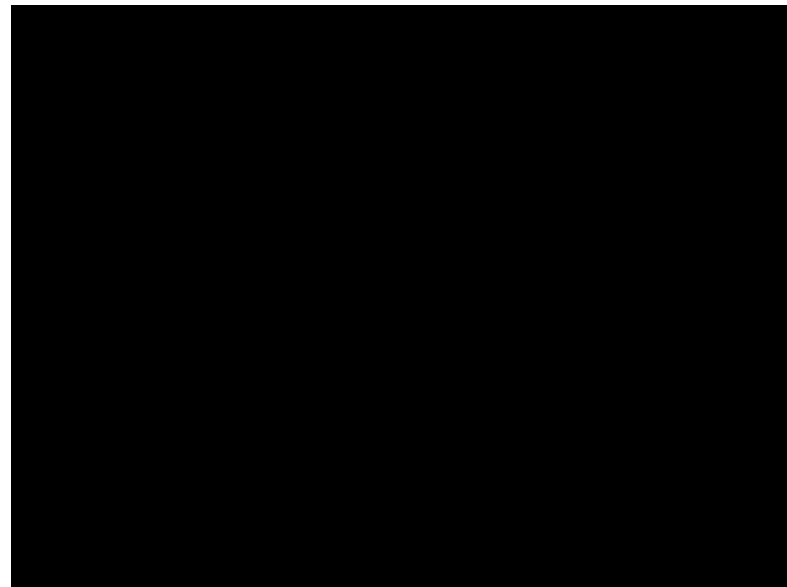
Hubel and Wiesel conducted a number of tests on sensory processing – they showed how the visual system in the brain builds up an image from a simple to a more complex representation



神经学基础

► 视觉认知中的感受野

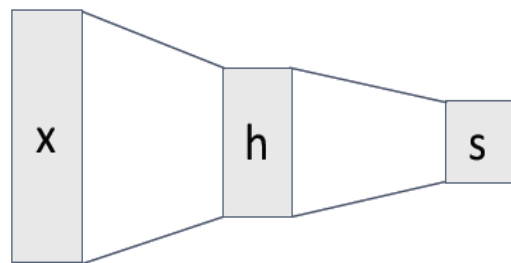
- ◆ 感受野是感觉系统中视网膜或皮肤等组织的感受器阵列所在的特定区域
- ◆ 该区域中的感受器直接或简接地联结到后续的某个神经细胞，当这个区域受到适宜刺激时，会使该神经细胞的活动模式发生改变



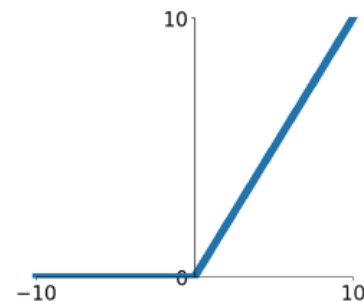
认知心理学-特征分析理论生理学基础

基本结构

➤ 全连接神经网络：基本结构

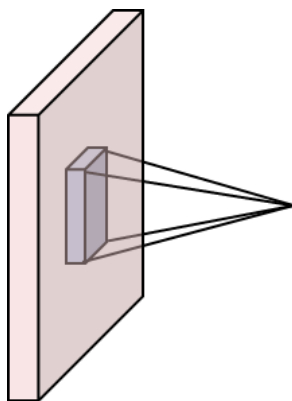


全连接层



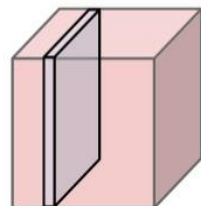
激活函数

➤ 卷积神经网络：基本结构



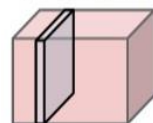
卷积层

224x224x64

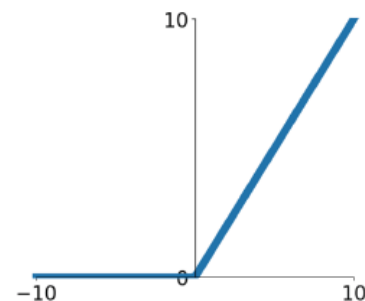


pool

112x112x64



池化层

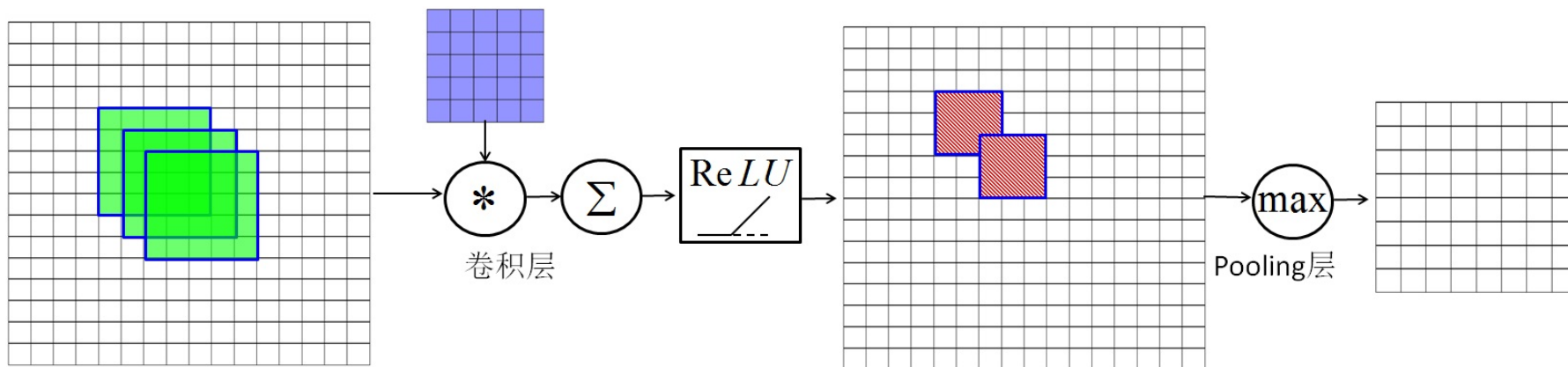


激活函数

5.1 基本单元

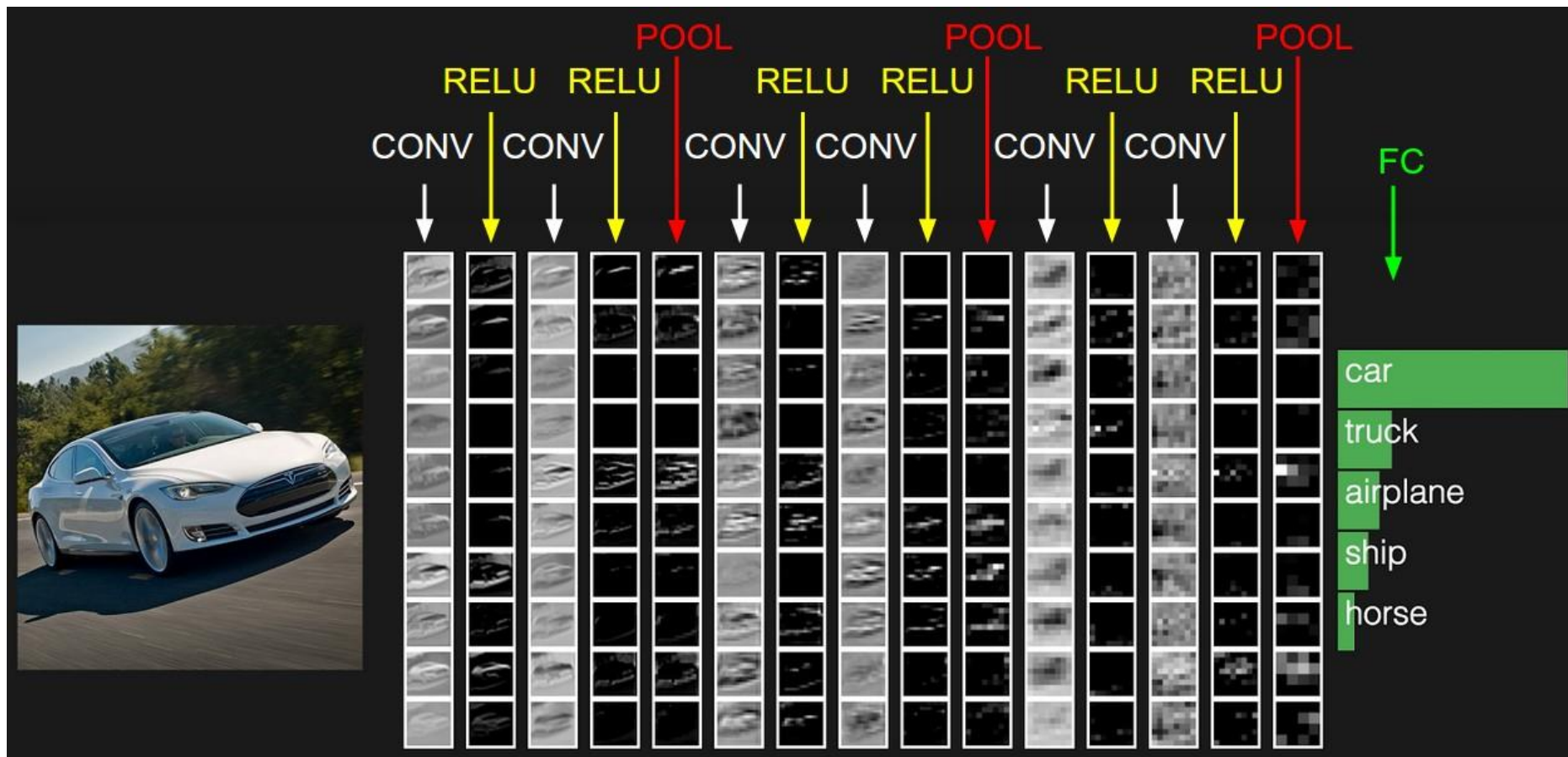
➤ 卷积神经网络的基本结构单元

- ◆ 包括卷积层、池化层、激活函数、Batch Normalization;
- ◆ 在卷积层，每个神经元的输入为前层输出的局部感受区域，通过卷积计算提取该局部区域的特征。通过卷积层的运算，提取到的局部特征之间的相互位置关系同时被保留；
- ◆ 在池化层，利用图像局部相关性原理，对卷积层输出的局部特征图进行下抽样，在保留有用信息的基础上减少数据处理量。



5.1 基本单元

➤ 示例：VGGNet for imageNet classification



5.1.1 卷积层

二维卷积的数学形式

□ 连续卷积

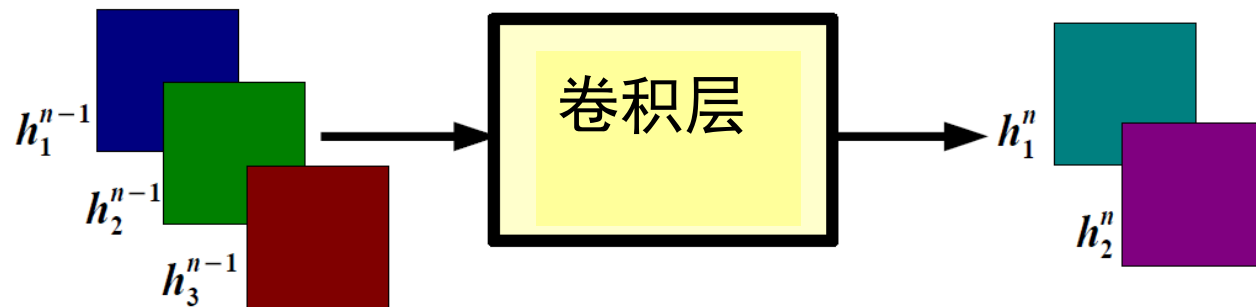
$$h(x, y) = i * k(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} i(u, v) k(x - u, y - v) du dv$$

□ 离散卷积

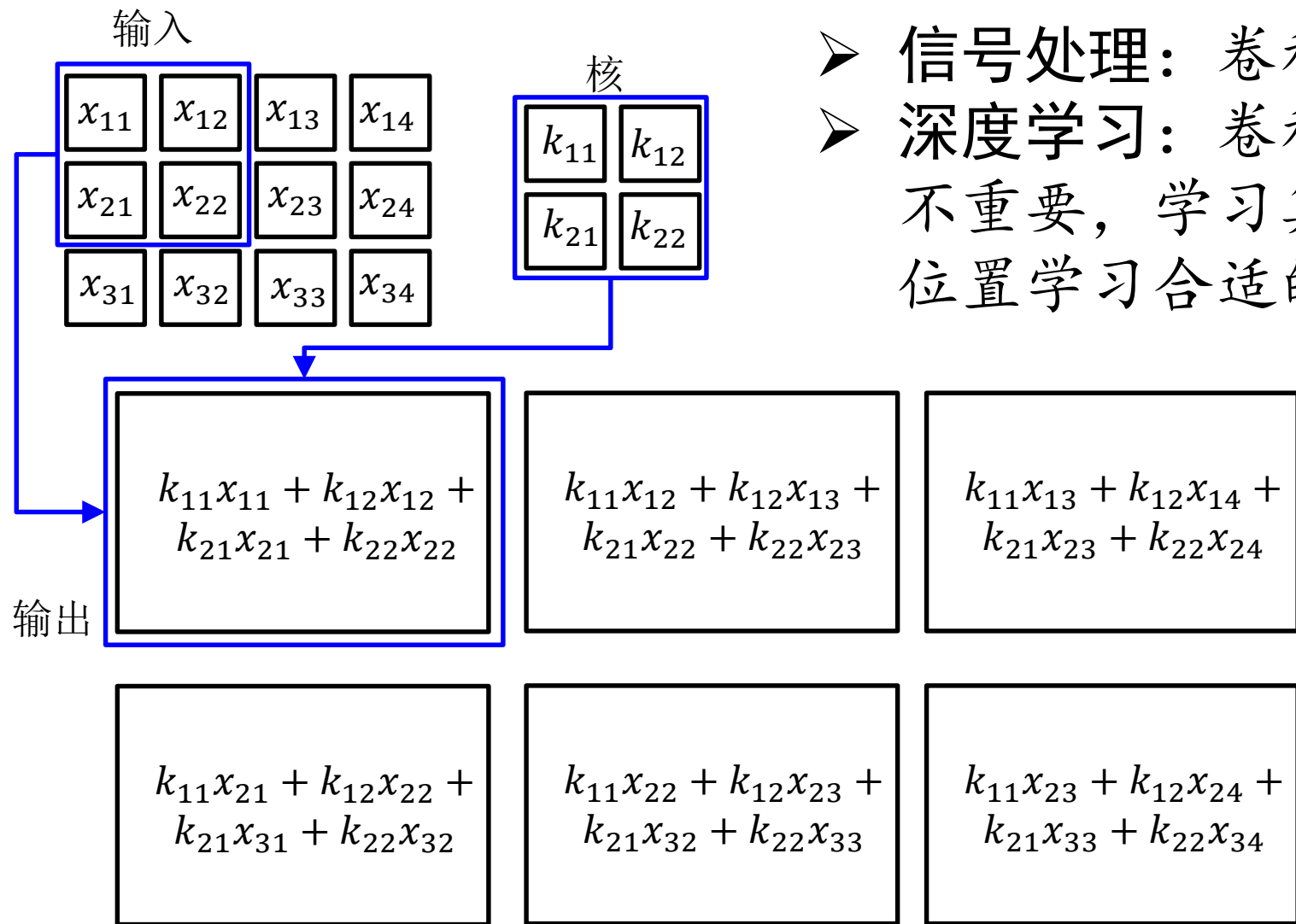
$$H(x, y) = I * K(x, y) = \sum_m \sum_n I(m, n) K(x - m, y - n)$$

□ 编程实现(Caffe)

$$H(x, y) = I * K(x, y) = \sum_m \sum_n I(x + m, y + n) K(m, n)$$

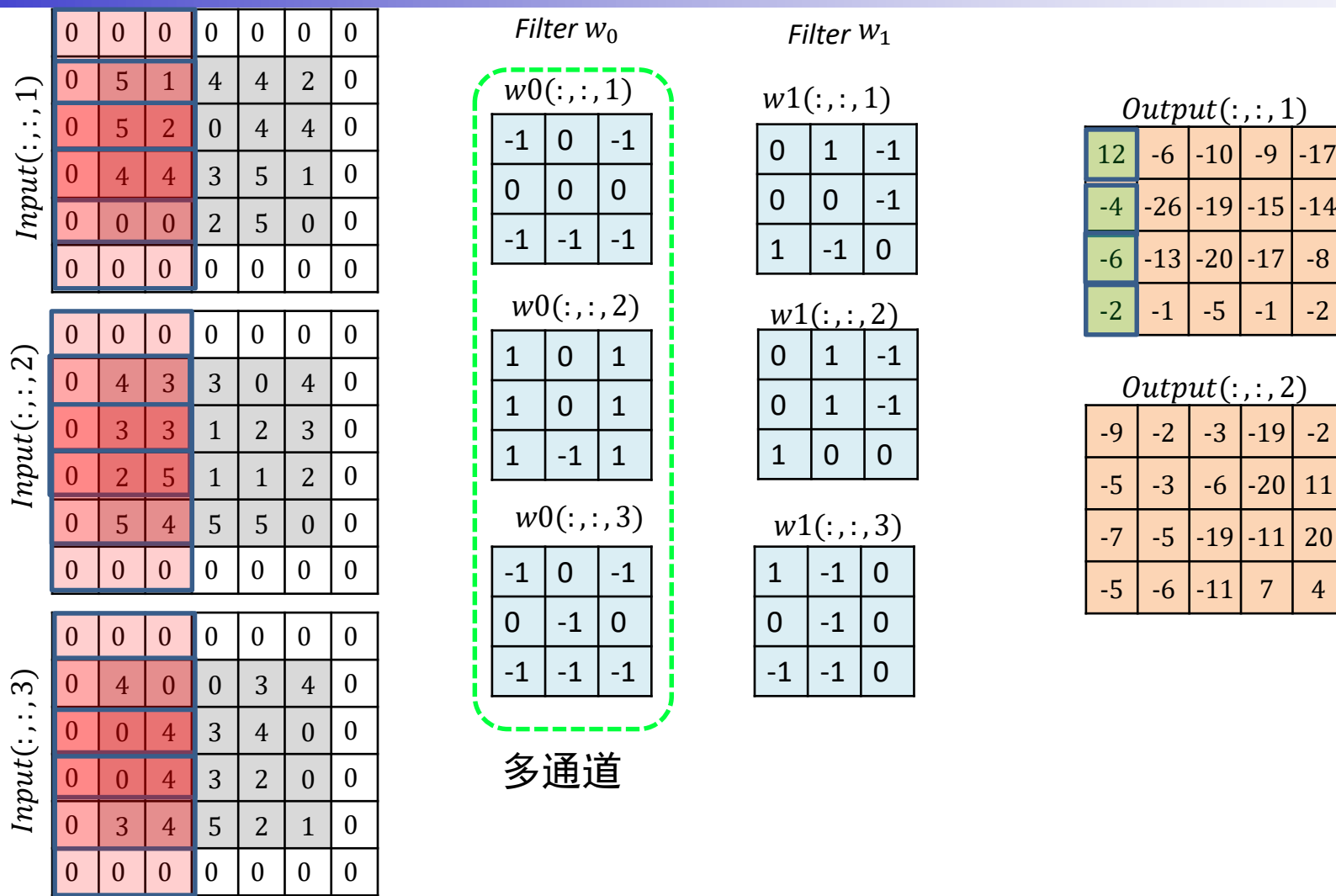


5.1.1 卷积层



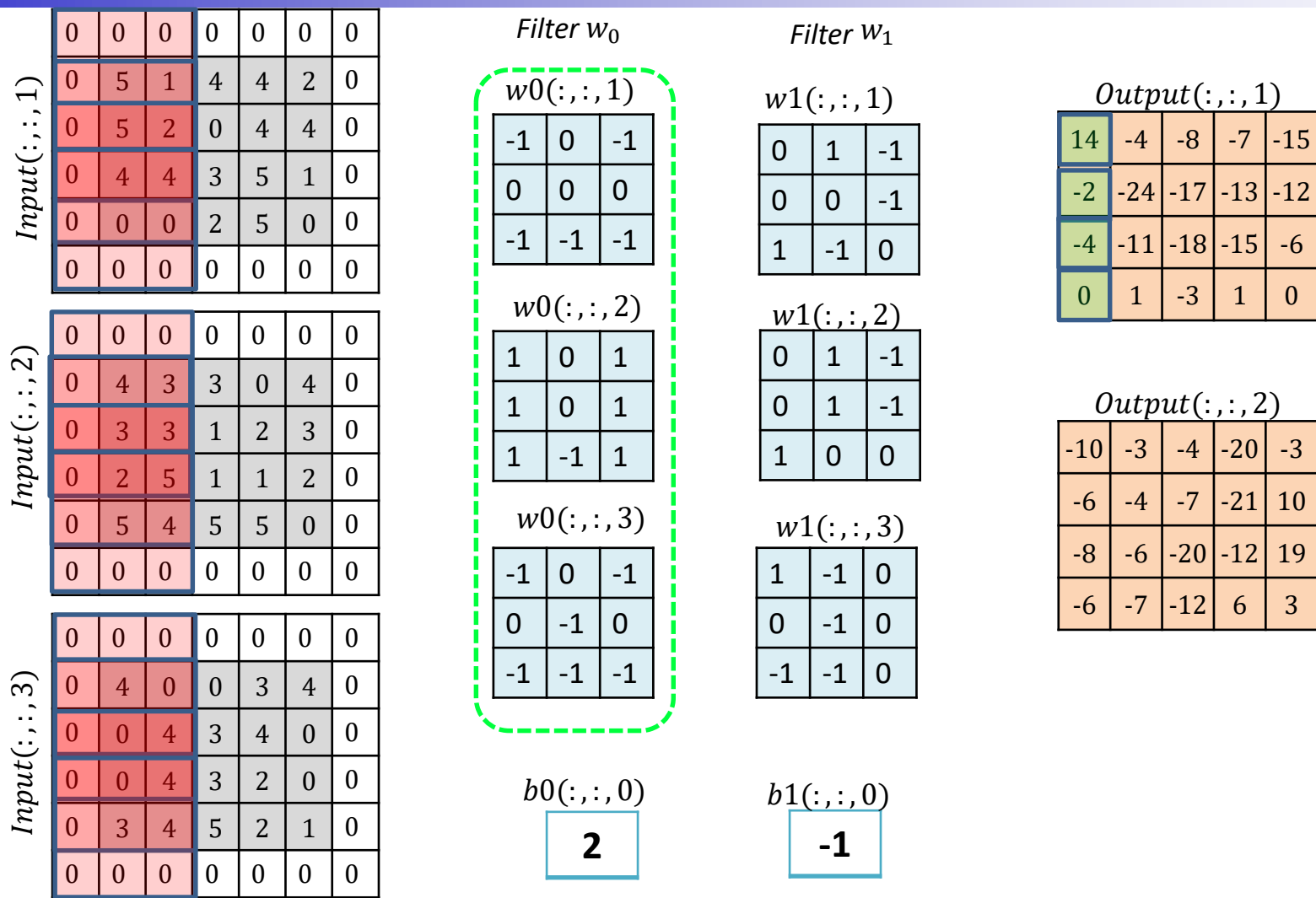
- 信号处理：卷积与互相关
- 深度学习：卷积的可交换性质不重要，学习算法会在合适的位置学习合适的数值；

5.1.1 卷积层



基于信号互相关(cross-correlation)定义的运算示例 (无偏置)

5.1.1 卷积层



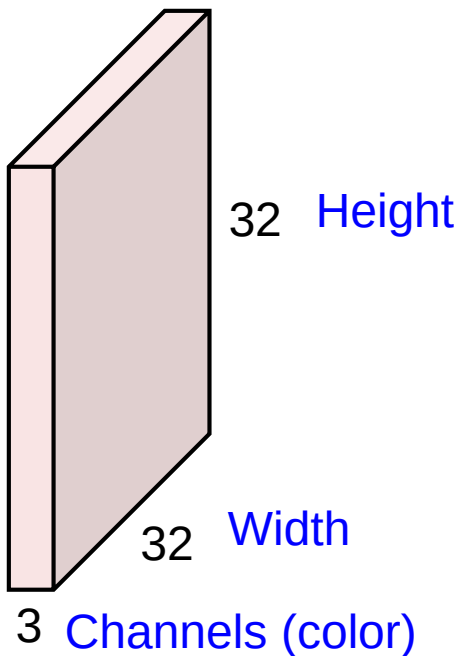
基于信号互相关(cross-correlation)定义的运算示例(有偏置)

5.1.1 卷积层

➤ 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积滤波器 (convolutional filter): $C_{in} \times K_H \times K_W$

32x32x3 image



全通道卷积：卷积核通道数
同输入数据通道数一致

5x5x3 filter

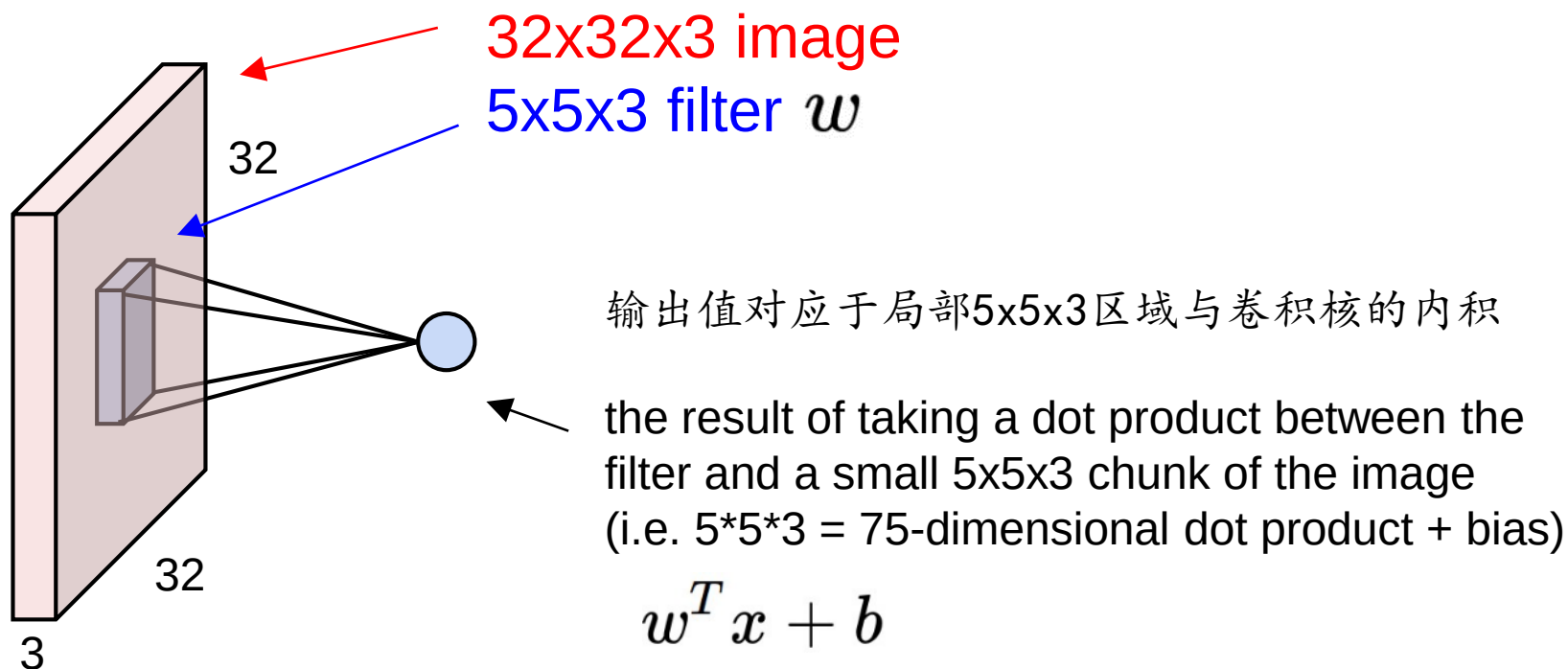


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

5.1.1 卷积层

► 卷积层:

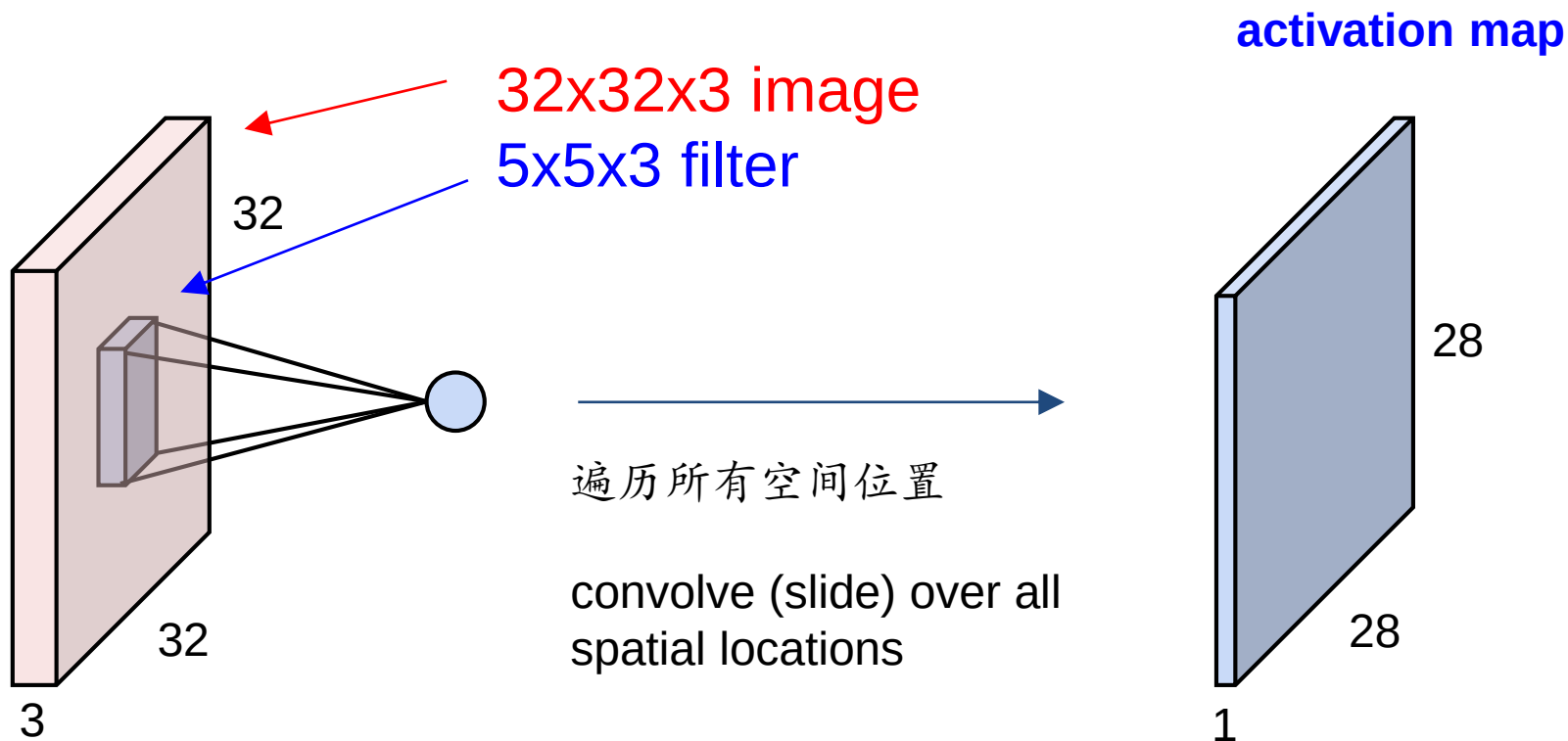
- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积滤波器 (convolutional filter): $C_{in} \times K_H \times K_W$



5.1.1 卷积层

➤ 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积滤波器 (convolutional filter): $C_{in} \times K_H \times K_W$



5.1.1 卷积层

➤ 卷积层:

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

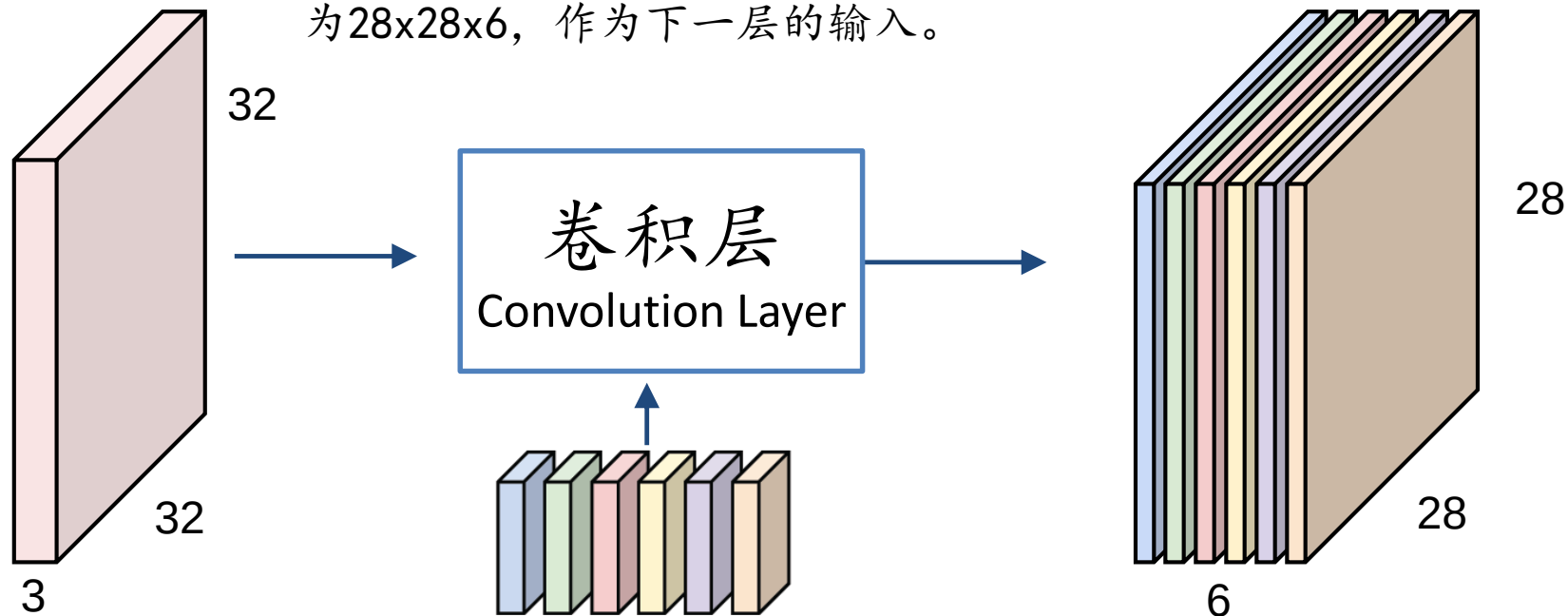


5.1.1 卷积层

► 卷积层：

- ◆ 输入数据 (input volume) : $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer) : $C_{out} \times C_{in} \times K_H \times K_W$

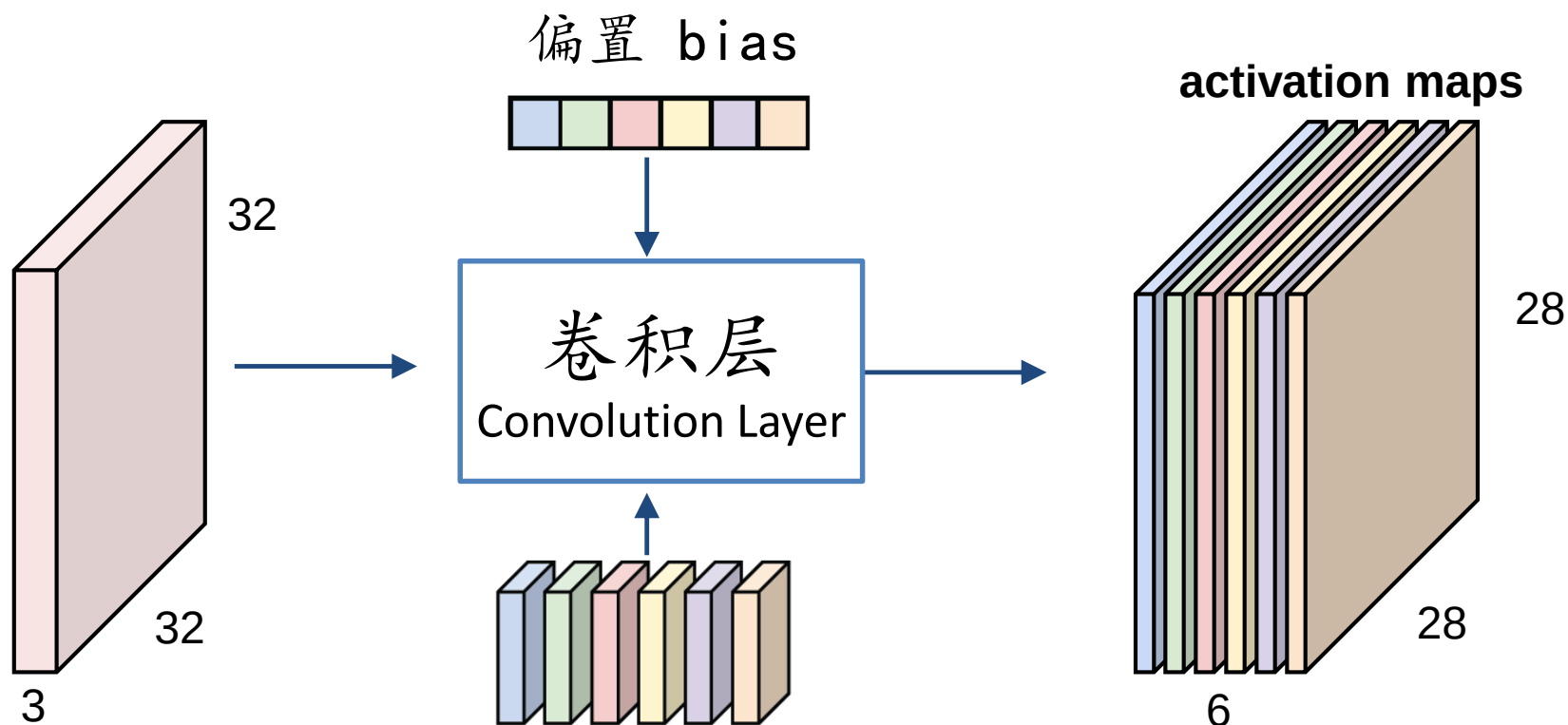
假设有6个5x5x3的卷积滤波器，则可获取6通道的输出特征图，因此输出数据维度为28x28x6，作为下一层的输入。



5.1.1 卷积层

► 卷积层：

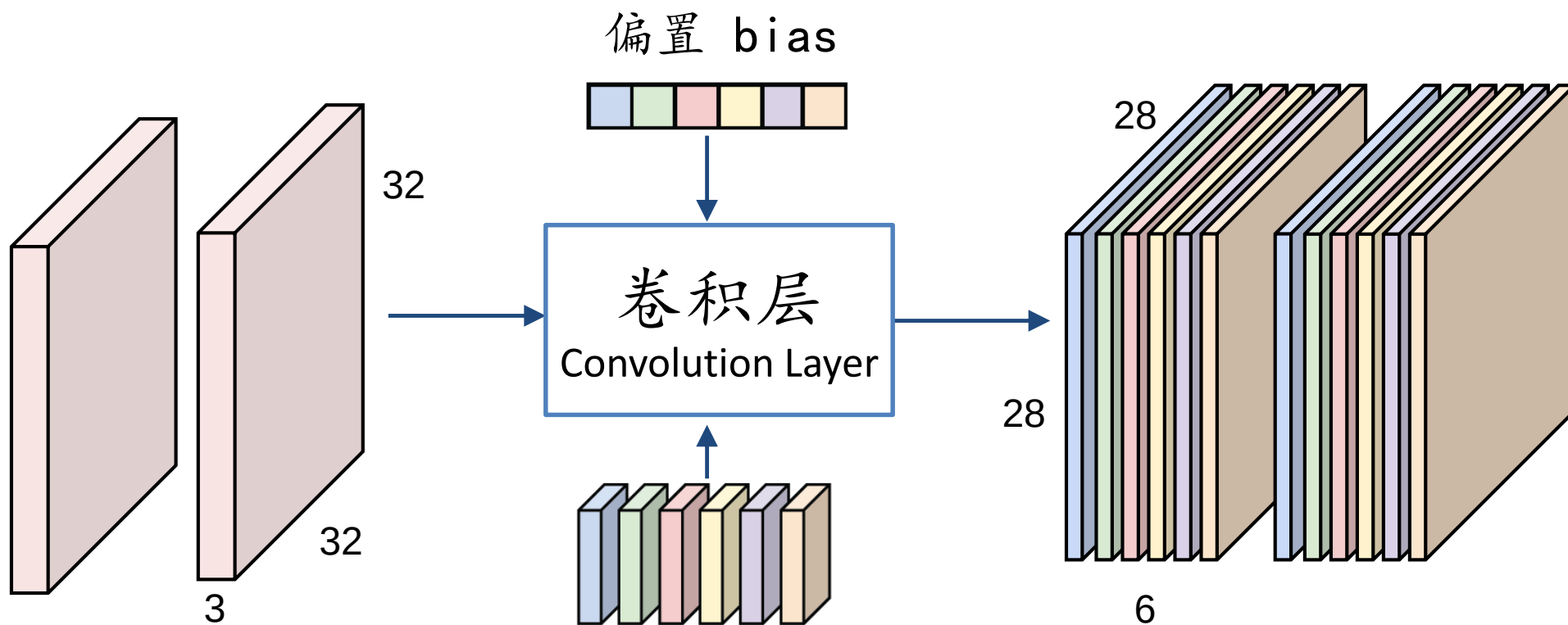
- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer) : $C_{out} \times C_{in} \times K_H \times K_W$



5.1.1 卷积层

► 卷积层：

- ◆ 输入批处理数据： $N \times C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer)： $C_{out} \times C_{in} \times K_H \times K_W$



5.1.1 卷积层—张量表示

➤ 卷积层张量(Tensor)表示

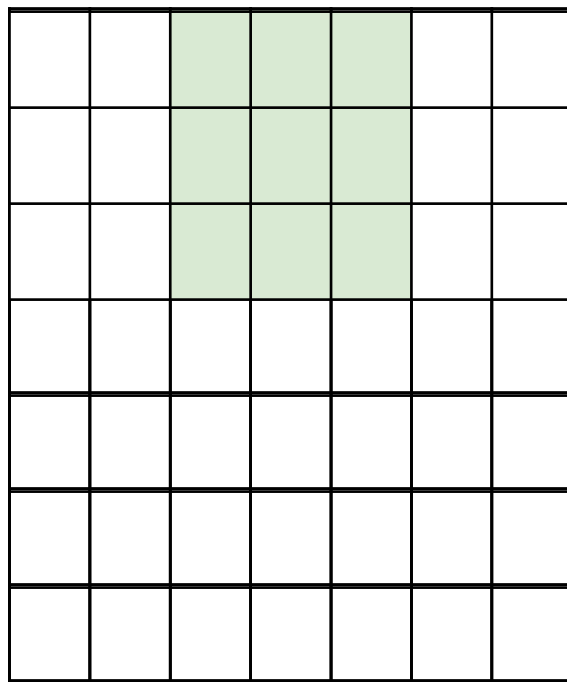
- ◆ 通常每个卷积层由卷积滤波器组构成，其(卷积权重)参数可表示成4D张量 (4D Tensors):
 - 张量维度为: $C_{out} \times C_{in} \times K_H \times K_W$
 - C_{out} = 输出通道数 number of output channels
 - C_{in} = 输入通道数 number of input channels
 - K_H = 卷积滤波器高度 filter height
 - K_W = 卷积滤波器宽度 filter width
- ◆ 偏置参数为 C_{out} 维向量。

5.1.1 卷积层—超参数之卷积核尺寸

► 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

7



输入空域尺寸7x7

卷积滤波器尺寸/卷积核大小3x3

则输出特征图空域尺寸是 5x5

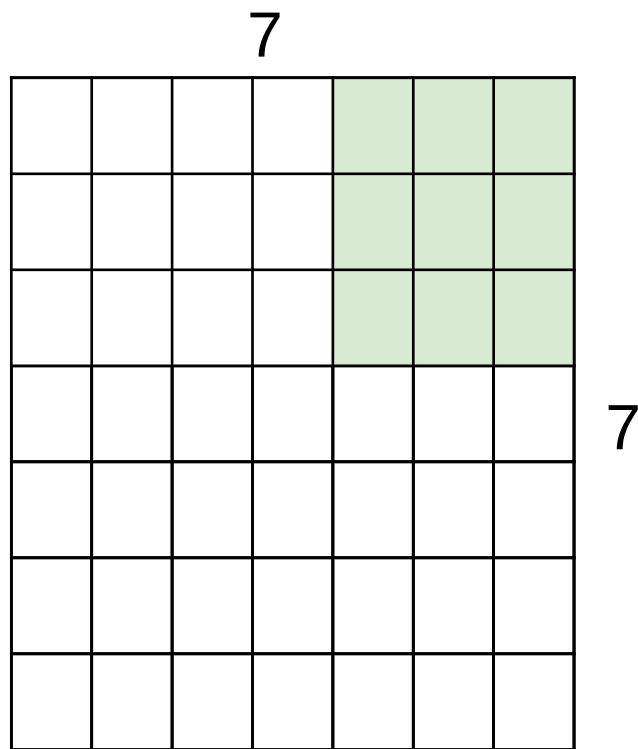
7

俯视图

5.1.1 卷积层—超参数之步长Stride

► 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

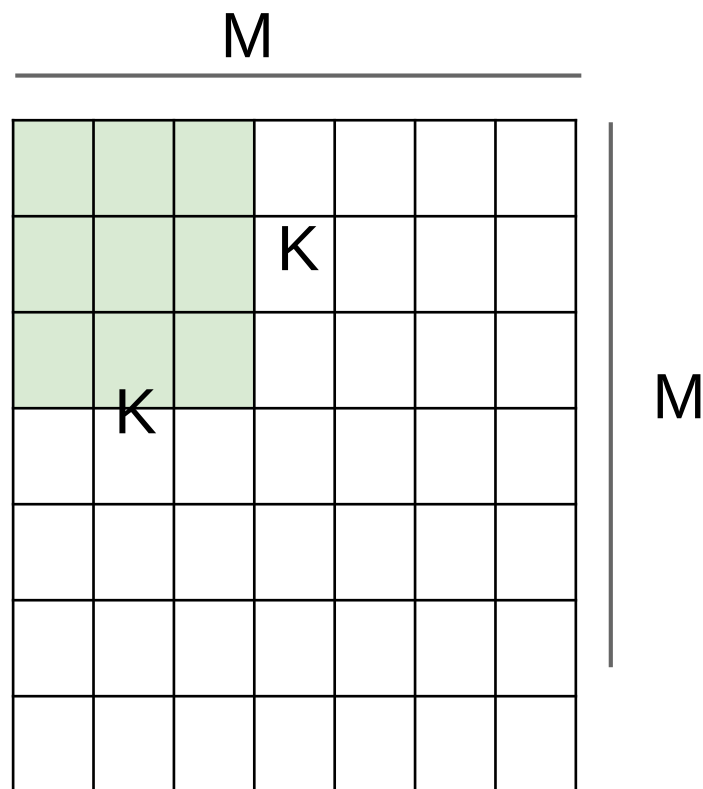


输入空域尺寸7x7,
卷积核大小3x3,
步长stride 2,
则输出特征图空域尺寸是 3x3

5.1.1 卷积层—超参数之步长Stride

► 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$



输入空域尺寸MxM,
卷积核大小KxK,
步长stride S,
则输出特征图空域尺寸是 ?

$$(M - K) / S + 1$$



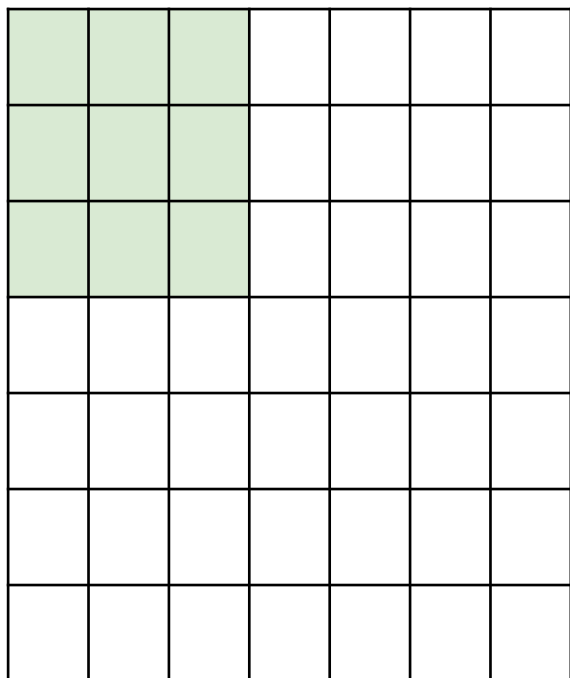
$$\lfloor (M - K) / S \rfloor + 1$$

5.1.1 卷积层—超参数之边界延拓Padding

► 卷积层:

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

7



输入空域尺寸7x7,
卷积核大小3x3,
步长stride 3,
则输出特征图空域尺寸是 ?

7

输出空域尺寸: $(M - K)/S + 1$

例如: $M = 7, K = 3$:

stride 1 $\Rightarrow (7 - 3)/1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3)/2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3)/3 + 1 = 2.33$

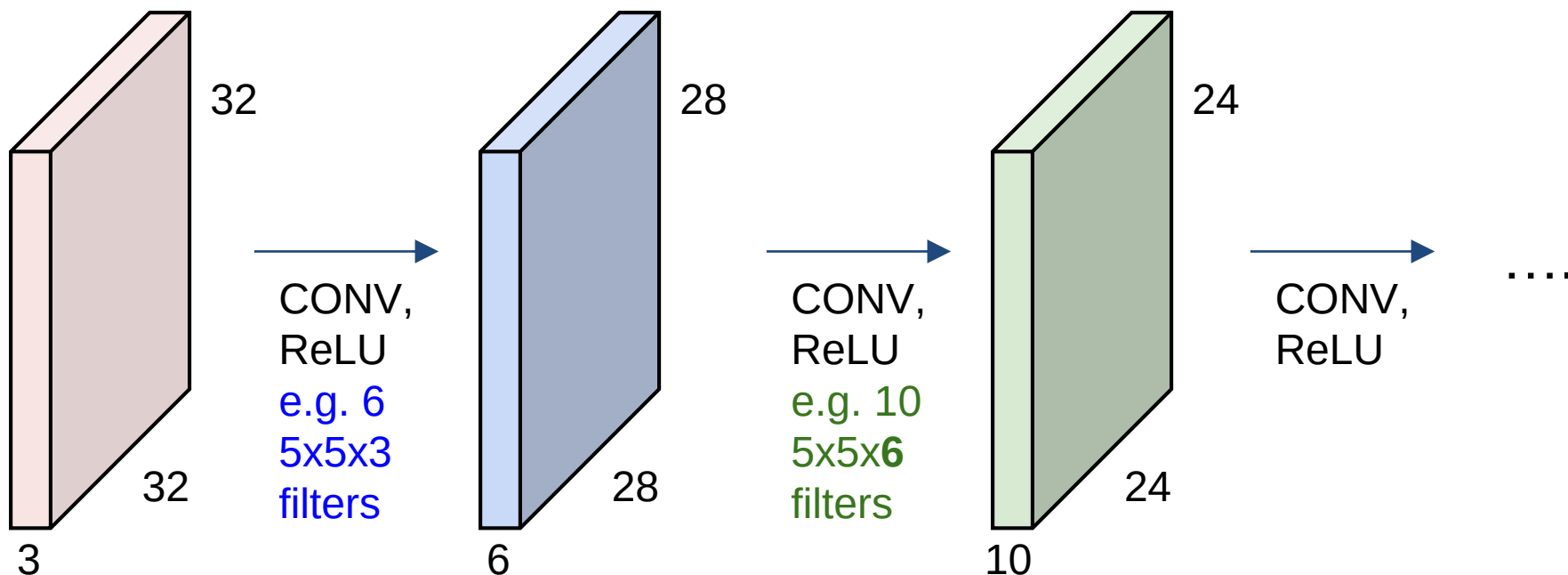
有边界信息损失

5.1.1 卷积层—超参数之边界延拓Padding

► 卷积层：

- ◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$
- ◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

Preview: ConvNet is a sequence of Convolution Layers, interspersed with non-linear activation functions



5.1.1 卷积层—超参数之边界延拓Padding

► 卷积层：

◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$

◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

0	0	0	0	0	0			
0								
0								
0								
0								

输入空域尺寸7x7,

卷积核大小3x3,

步长stride 1,

边界延拓(零值)为1,

则输出特征图空域尺寸是 7x7

5.1.1 卷积层—超参数之边界延拓Padding

➤ 卷积层:

◆ 输入数据 (input volume): $C_{in} \times H_{in} \times W_{in}$

◆ 卷积层 (convolution layer): $C_{out} \times C_{in} \times K_H \times K_W$

0	0	0	0	0	0			
0								
0								
0								
0								

输入空域尺寸7x7,

卷积核大小3x3,

步长stride 1,

边界延拓(零值)为1,

则输出特征图空域尺寸是 **7x7**

通常卷积层滤波器尺寸为K时, 边界延拓
(padding with zero) P设置为 $(K - 1)/2$,
该设置可保持输出特征图尺寸。

e.g. $K = 3 \Rightarrow$ zero pad with 1

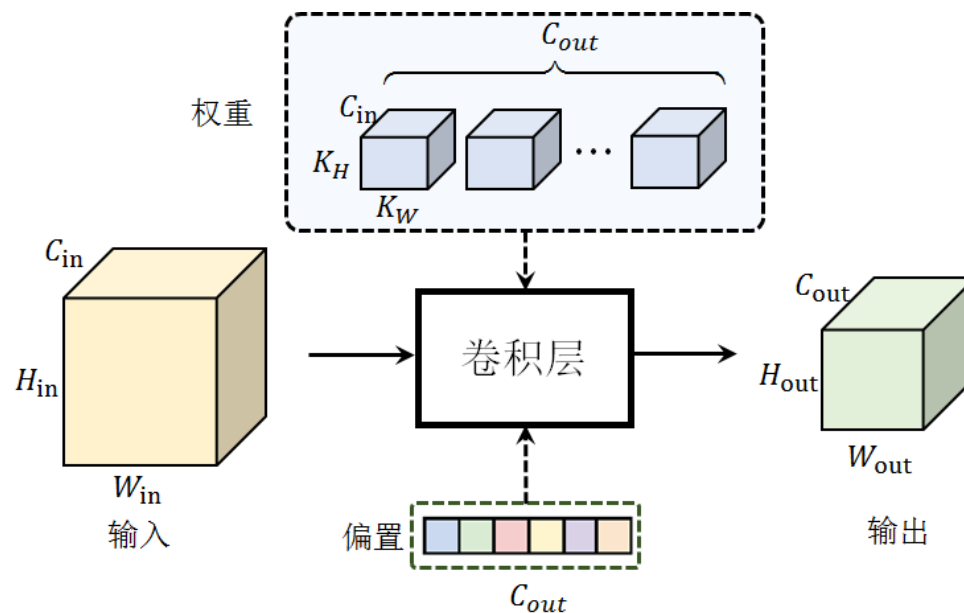
$K = 5 \Rightarrow$ zero pad with 2

$K = 7 \Rightarrow$ zero pad with 3

5.1.1 卷积层—输出特征图尺寸

► 卷积层:

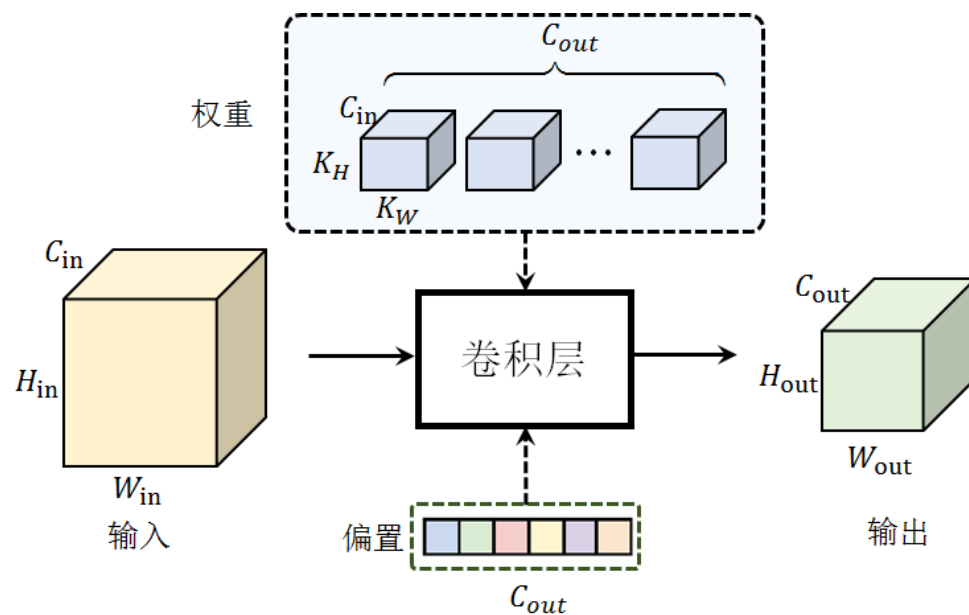
- ◆ 输入数据 (input volume): $N \times C_{in} \times H_{in} \times W_{in}$
 - 输入数据 batch size: N
- ◆ 卷积层超参数(conv filter):
 - 卷积核大小kernel size: $K_H \times K_W$
 - 滤波器数目 number of filters: C_{out}
 - 步长stride: S
 - 延拓padding: P
- ◆ 输出数据 (output volume): $N \times C_{out} \times H_{out} \times W_{out}$
 - $W_{out} = (W_{in} - K_W + 2P)/S + 1$
 - $H_{out} = (H_{in} - K_H + 2P)/S + 1$



5.1.1 卷积层—参数量估计

➤ 卷积层:

- ◆ 输入数据 (input volume): $N \times C_{in} \times H_{in} \times W_{in}$
 - 输入数据 batch size: N
- ◆ 卷积层超参数(conv filter):
 - 卷积核大小kernel size: $K_H \times K_W$
 - 滤波器数目 number of filters: C_{out}
 - 步长stride: S
 - 延拓padding: P
- ◆ 参数量(不考虑偏置): $K_W \cdot K_H \cdot C_{in} \cdot C_{out}$
- ◆ 参数量(考虑偏置): $(K_W \cdot K_H \cdot C_{in} + 1) \cdot C_{out}$



5.1.1 卷积层—浮点运算次数FLOPs

► 卷积层:

◆ 输入数据 (input volume): $N \times C_{in} \times H_{in} \times W_{in}$

- 输入数据 batch size: N

◆ 卷积层超参数(conv filter):

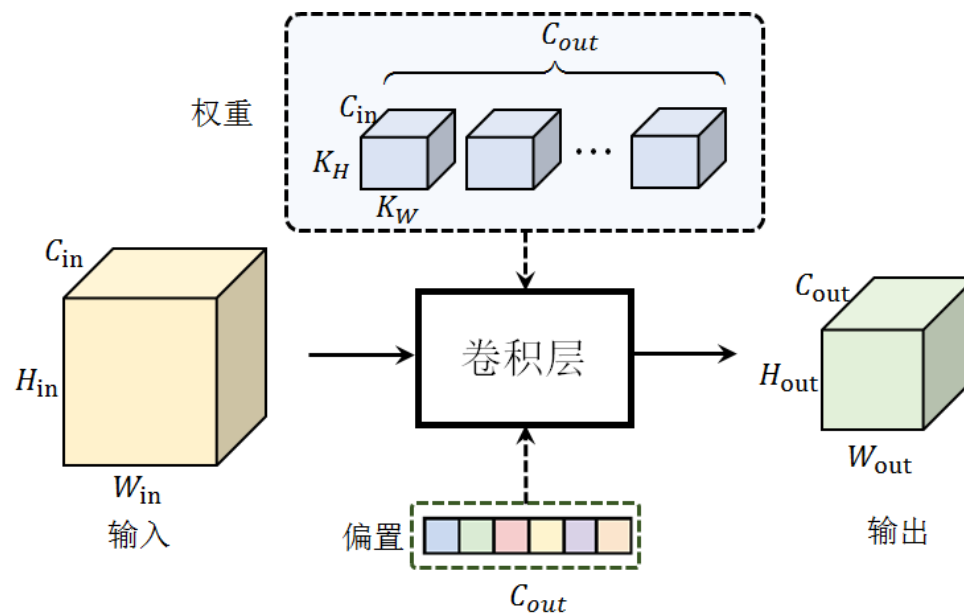
- 卷积核大小kernel size: $K_H \times K_W$
- 滤波器数目 number of filters: C_{out}
- 步长stride: S
- 延拓padding: P

◆ 乘法FLOPs(不考虑偏置): $K_W \cdot K_H \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out}$

◆ 乘法FLOPs(考虑偏置): $(K_W K_H C_{in} + 1) \cdot C_{out} \cdot H_{out} \cdot W_{out}$

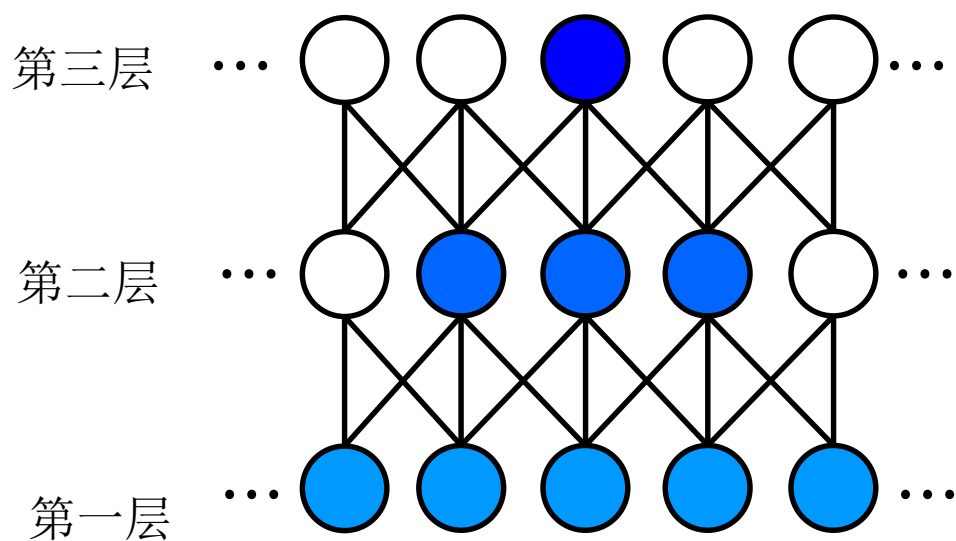
◆ 加法FLOPs(不考虑偏置): $(K_W K_H C_{in} - 1) \cdot C_{out} \cdot H_{out} \cdot W_{out}$

◆ 加法FLOPs(考虑偏置): $K_W \cdot K_H \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out}$

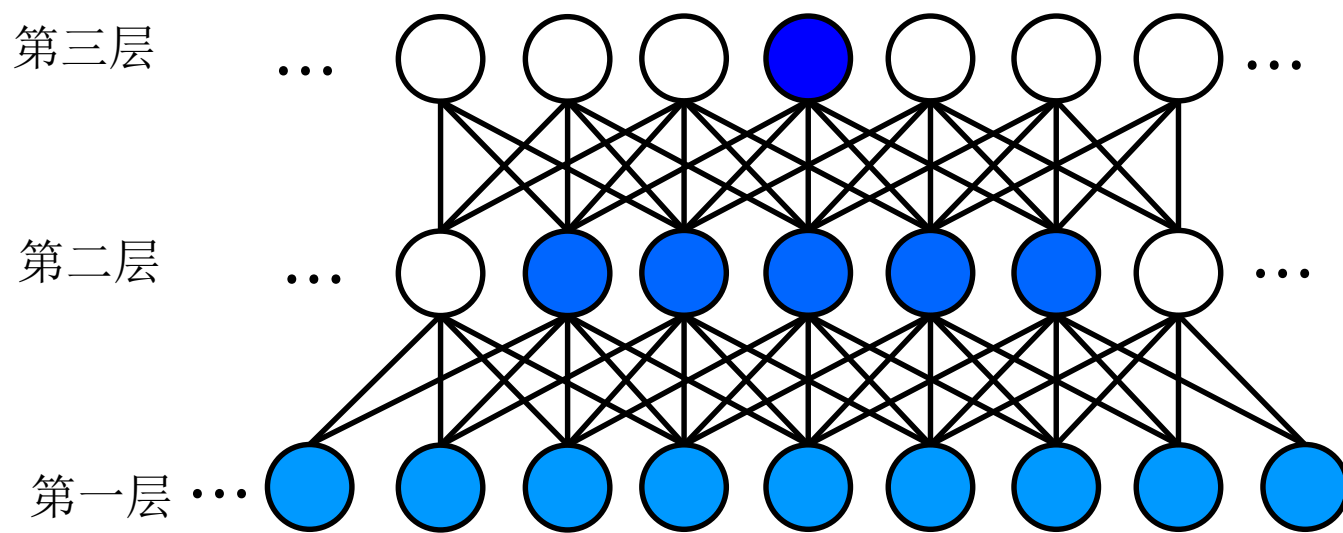


5.1.1 卷积层—感受野估计

- 感受野：神经网络每一层输出特征图上的像素点对应于原始输入图的映射区域大小；
 - ◆ 单层卷积层的感受野与卷积核大小、卷积步长有关；
 - ◆ 多层卷积层的感受野采用回溯方法估计；



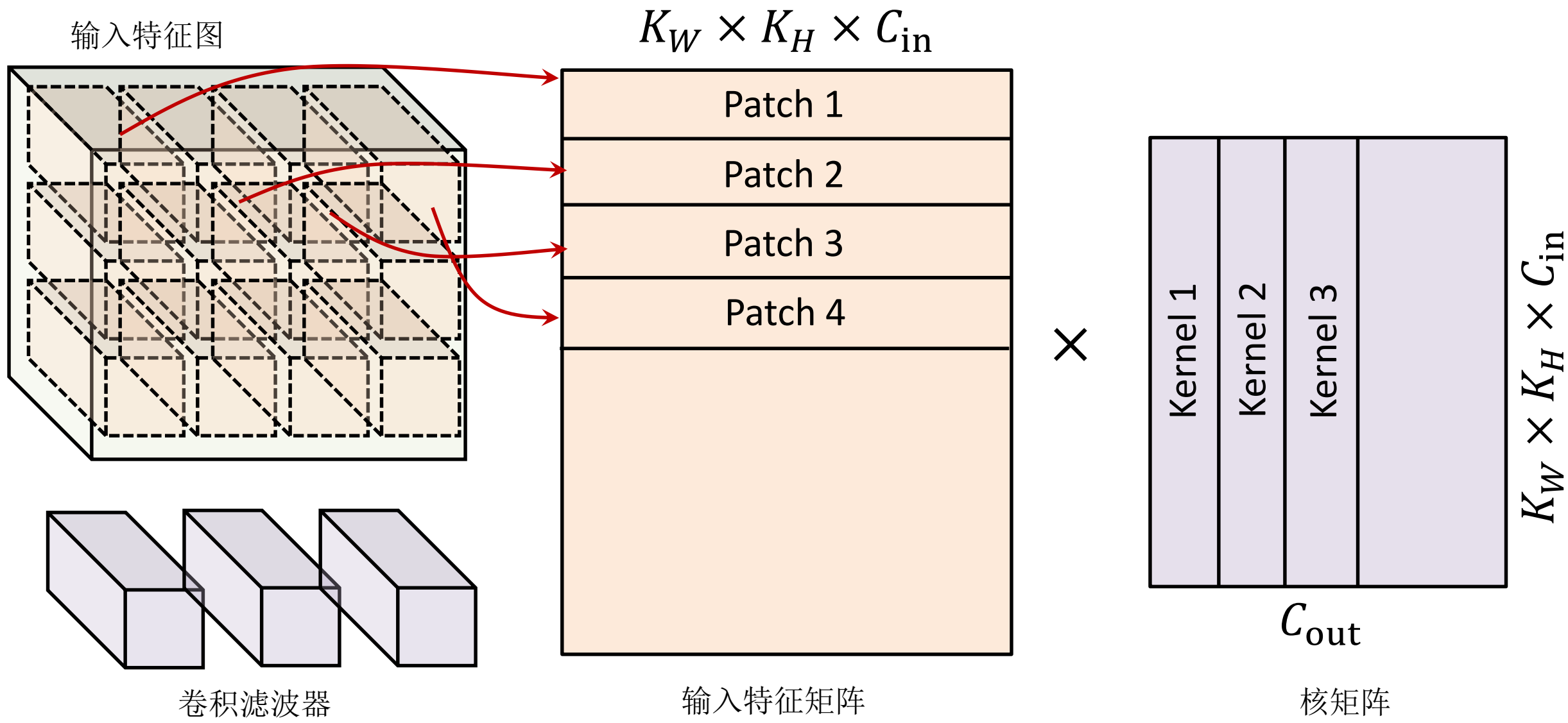
(a)



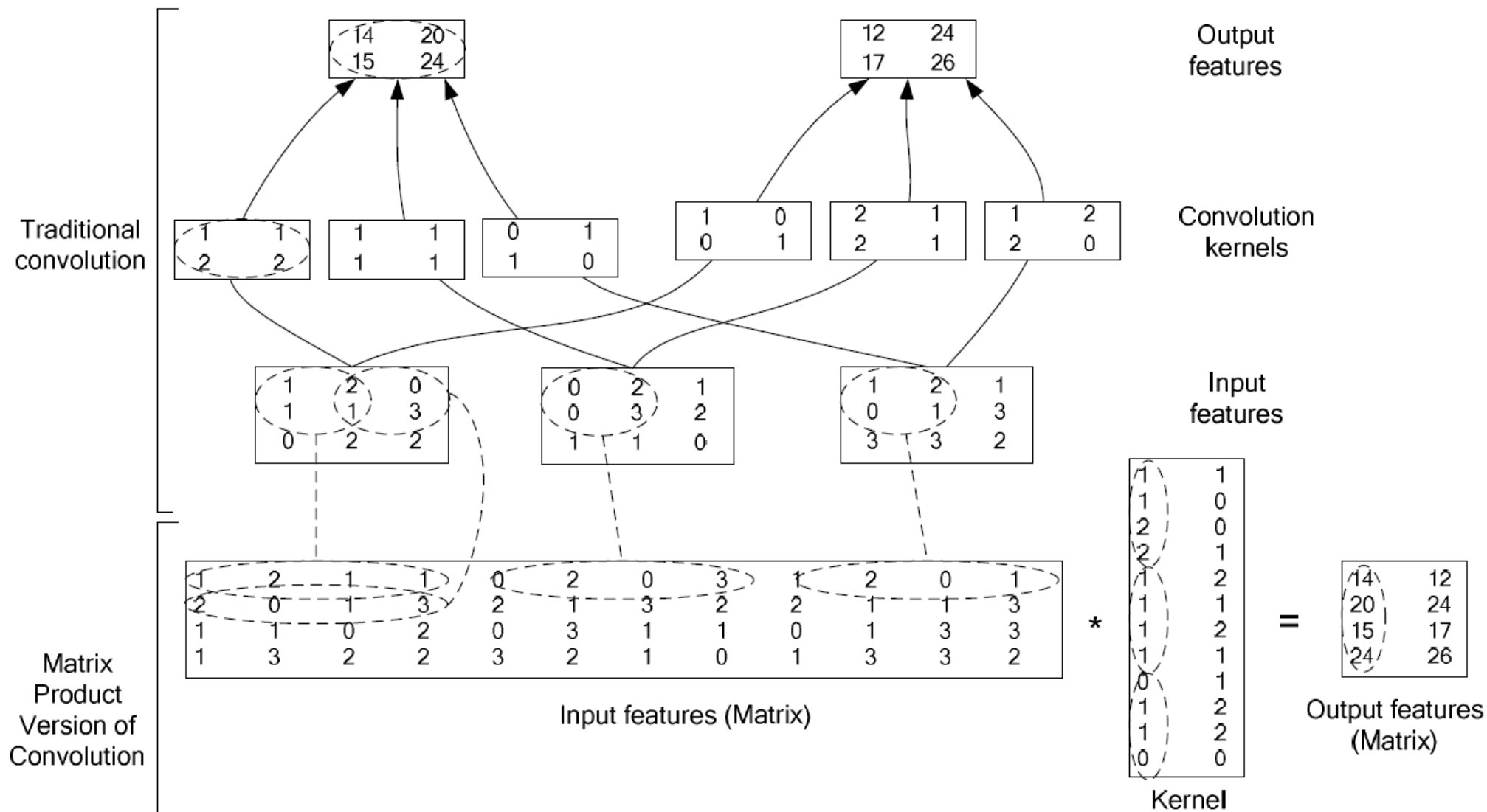
(b)

5.1.1 卷积层—高效计算

卷积运算转化为矩阵相乘



5.1.1 卷积层—高效计算



5.1.1 卷积层—pytorch实现

CONV2D

超参数

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,  
dilation=1, groups=1, bias=True, padding_mode='zeros')
```

[SOURCE]

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{out}, H_{out}, W_{out})$ can be precisely described as:

$$\text{out}(N_i, C_{out_j}) = \text{bias}(C_{out_j}) + \sum_{k=0}^{C_{in}-1} \text{weight}(C_{out_j}, k) \star \text{input}(N_i, k)$$

参数(可学习)

where $\star$ is the valid 2D **cross-correlation** operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

This module supports **TensorFloat32**.

- `stride` controls the stride for the cross-correlation, a single number or a tuple.
- `padding` controls the amount of implicit padding on both sides for `padding` number of points for each dimension.
- `dilation` controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.
- `groups` controls the connections between inputs and outputs. `in_channels` and `out_channels` must both be divisible by `groups`. For example,

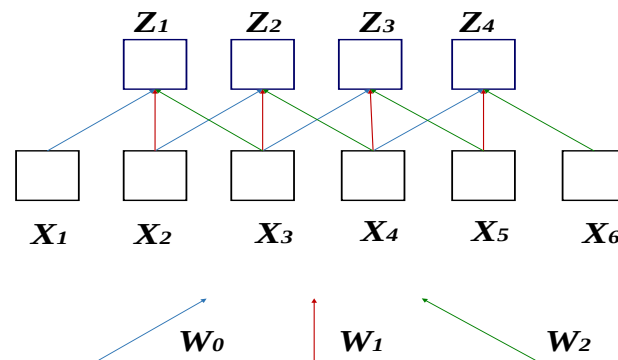
5.1.1 卷积层—误差反向传播

➤ 示例：一维卷积神经网络误差反传

假设一维卷积层具有 $Z_m = \sum_{j=0}^{K-1} W_j X_{m+j}$ 的形式，其中 $m \in [1, M]$ 。 K 为卷积核 W 的大小。假定我们已知 $\frac{\partial L}{\partial Z_m}$ ， $m \in [1, M]$ ，其中 L 为目标函数，即由误差信号构造的能量函数 $L = \frac{1}{2} \sum_{m=1}^M e_i^2 = \frac{1}{2} \sum_{m=1}^M (Z_m - y_m)^2$ ， Z_m 为实际输出， y_m 为应有输出。

利用微积分中求复合函数导数的链式法则，给出计算 $\frac{\partial L}{\partial W_j}$ 、 $\frac{\partial L}{\partial X_i}$ 的公式。

$$Z_m = \sum_{j=0}^{K-1} W_j X_{m+j}$$



一维卷积示意图

(输出数值个数 $M=4$ ，
卷积核大小 $K=3$)

5.1.1 卷积层—误差反向传播

注意：作用范围

➤ 示例：一维卷积神经网络误差反传

➤ 解答：

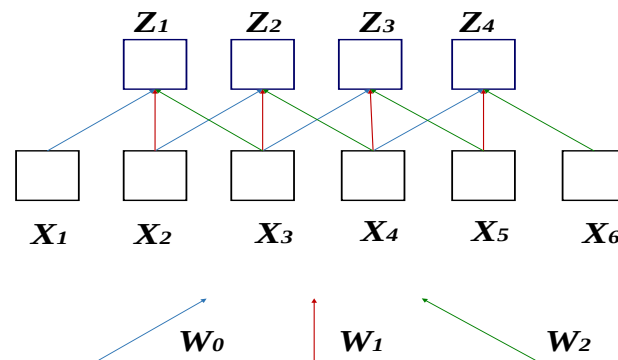
$$(1) \frac{\partial L}{\partial W_j} = \sum_{m=1}^M \frac{\partial L}{\partial Z_m} \frac{\partial Z_m}{\partial W_j} = \sum_{m=1}^M \frac{\partial L}{\partial Z_m} X_{m+j}$$

(2) 只有当 $m \leq i \leq m+K-1$, $m \in [1, M]$ 时,

$\frac{\partial Z_m}{\partial X_i}$ 的值非零, 为 W_{i-m} ,

$$\frac{\partial L}{\partial X_i} = \sum_{m=1}^M \frac{\partial L}{\partial Z_m} \frac{\partial Z_m}{\partial X_i} = \sum_{m=\max\{1, i-K+1\}}^i \frac{\partial L}{\partial Z_m} W_{i-m}$$

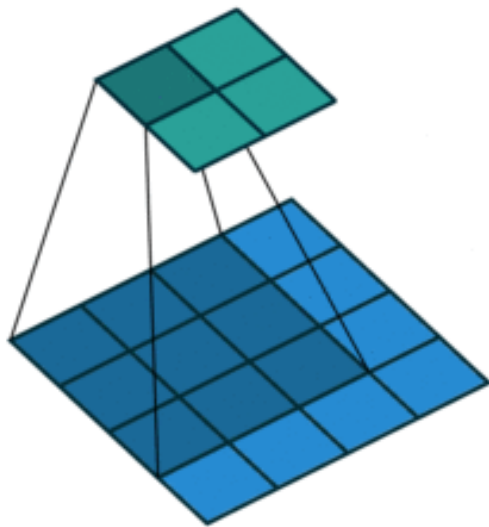
$$Z_m = \sum_{j=0}^{K-1} W_j X_{m+j}$$



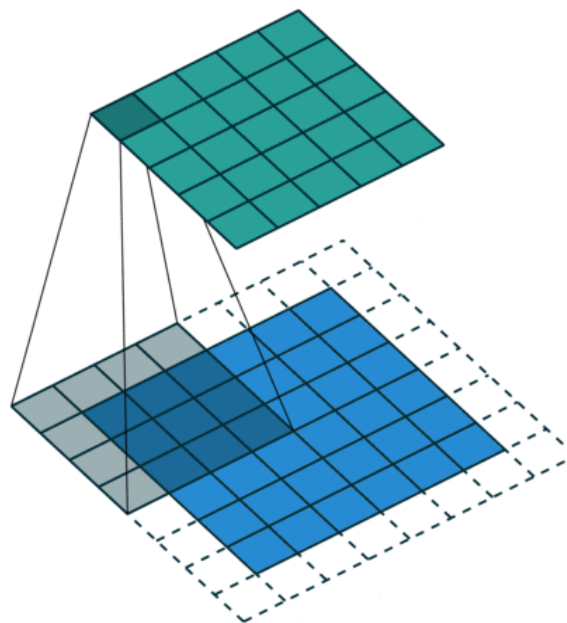
一维卷积示意图

(输出数值个数 $M=4$,
卷积核大小 $K=3$)

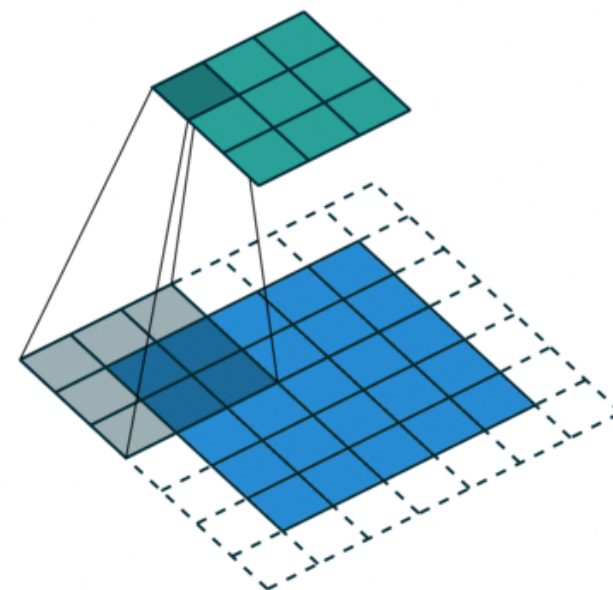
5.1.1 卷积层—动画演示



步长为1
无边界延拓



步长为1
有边界延拓



步长为2
有边界延拓

卷积核尺度：卷积核的长宽

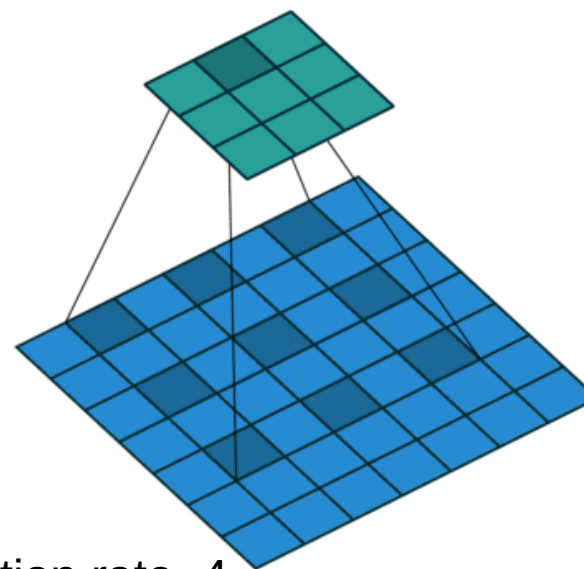
延拓：边界处填充，默认为0

步长：窗口一次滑动的长度

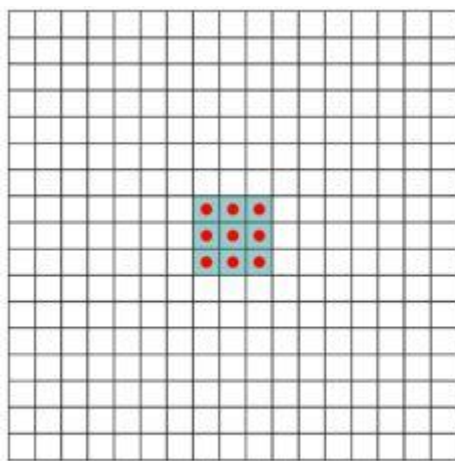
5.1.1 卷积层—变体

➤ 空洞卷积Dilated convolution

- 往卷积核中注入空洞，目标是增加感受野
- 相比正常的卷积而言，空洞卷积多出一个dilation rate参数

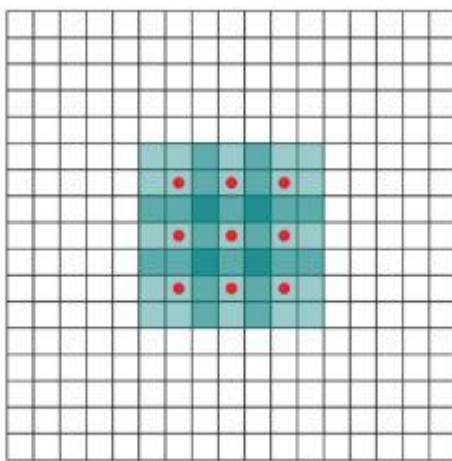


dilation rate=1



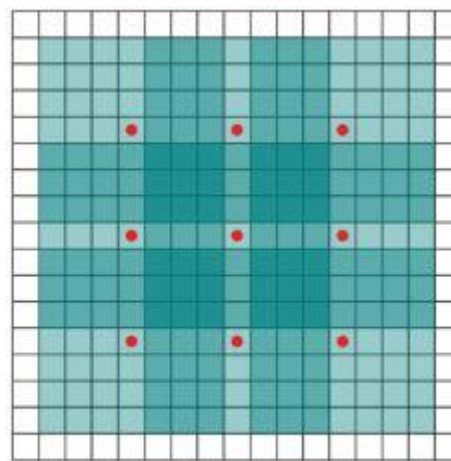
(a)

dilation rate=2



(b)

dilation rate=4

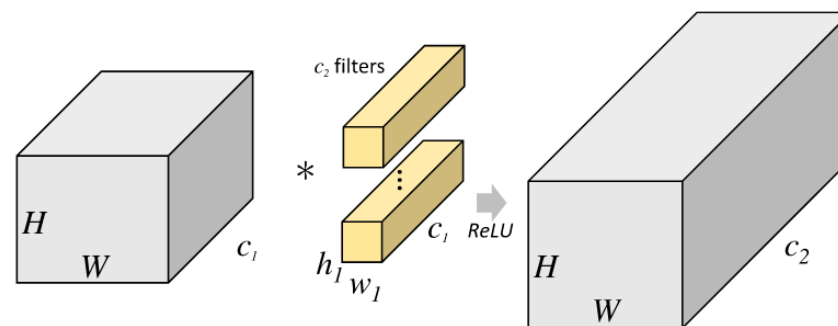


(c)

5.1.1 卷积层—变体

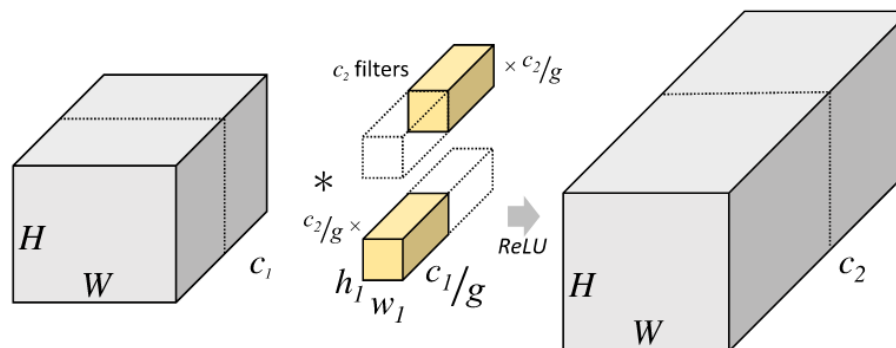
➤ 分组卷积Group convolution

- ✓ 最早在AlexNet中出现
- ✓ 在深度上进行划分，即某几个通道编为一组，相应的，卷积核深度等比例缩小而大小不变；
- ✓ 利用每组的卷积核同它们对应组内的输入数据卷积得到了输出数据以后，再连接组合；
- ✓ 分组后并行计算



A normal convolutional layer. Yellow blocks represent learned parameters, gray blocks represent feature maps/input images (working memory).

<http://blog.csdn.net/Chaolei3>



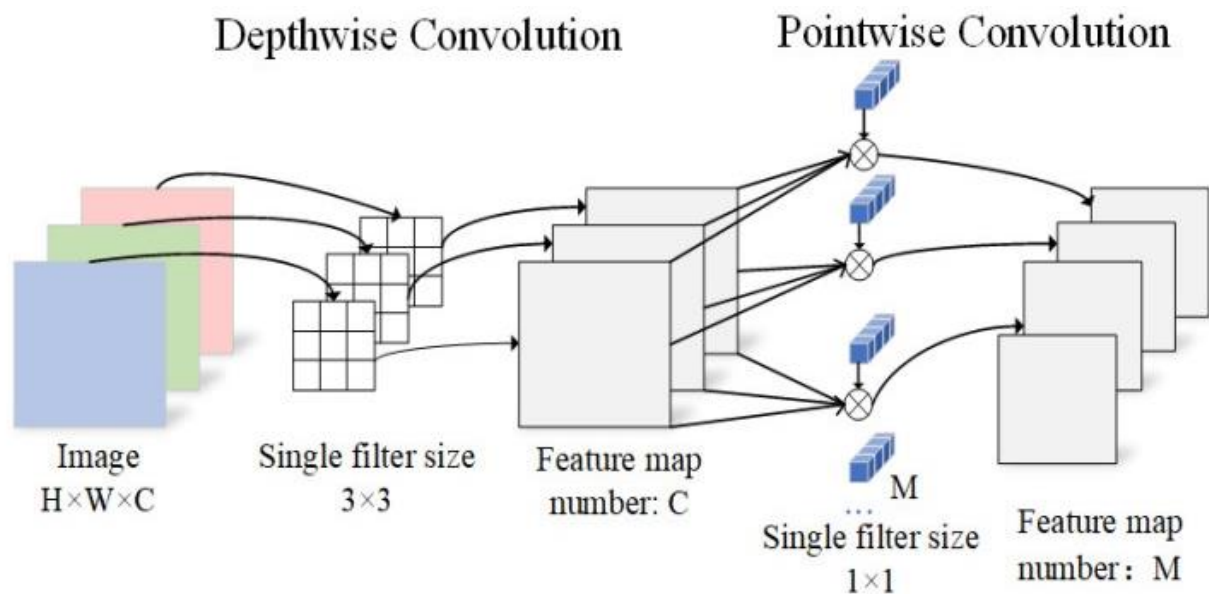
A convolutional layer with 2 filter groups. Note that each of the filters in the grouped convolutional layer is now exactly half the depth, i.e. half the parameters and half the compute as the original filter.

<http://blog.csdn.net/Chaolei3>

5.1.1 卷积层—变体

➤ 深度可分离卷积 Depthwise separable convolution

- ✓ 每一个通道用一个filter卷积之后得到对应一个通道的输出，然后再进行信息的融合。
- ✓ 深度可分离卷积比普通卷积减少了所需要的参数。重要的是深度可分离卷积将以往普通卷积操作同时考虑通道和区域改变成，卷积先只考虑区域，然后再考虑通道。实现了通道和区域的分离。



思考：1x1卷积的作用？

5.1.1 卷积层—变体

➤ 可变形卷积 Deformable convolution

✓ 引入卷积核偏置量的学习

$$y(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n). \quad (1)$$

$$y(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n + \Delta \mathbf{p}_n). \quad (2)$$

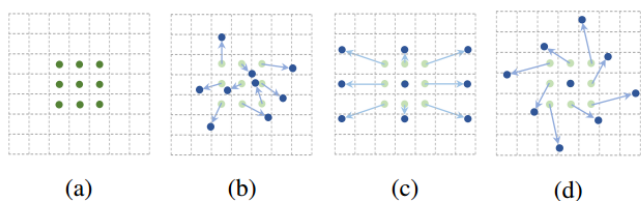
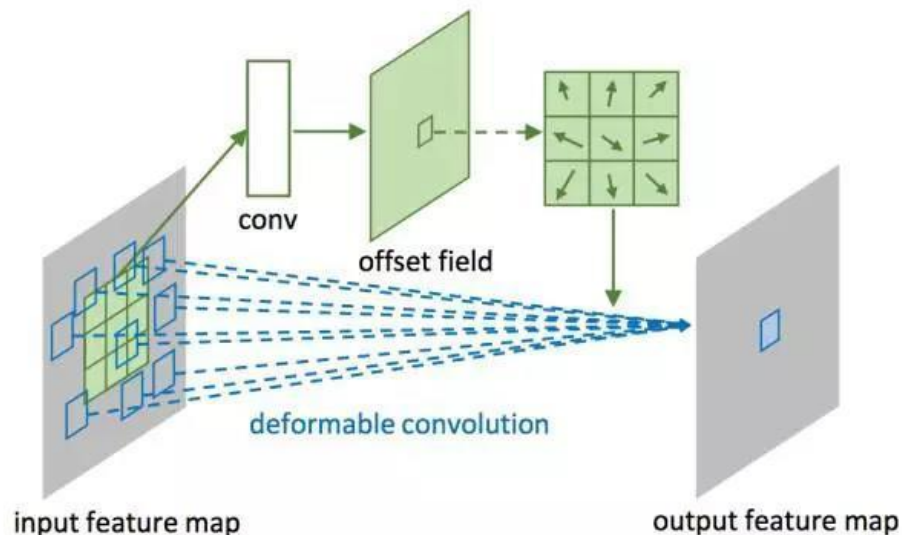


Figure 1: Illustration of sampling grids in 3×3 regular and deformable convolutions. (a) regular sampling grid (green points) of standard convolution. (b) deformed sampling grid with augmented offsets (blue arrows) in deformable convolution. (c)(d) are special cases of (b), showing that the deformable convolution generalizes scale, aspect ratio and rotation transformations.

<http://blog.csdn.net/xbinworld>



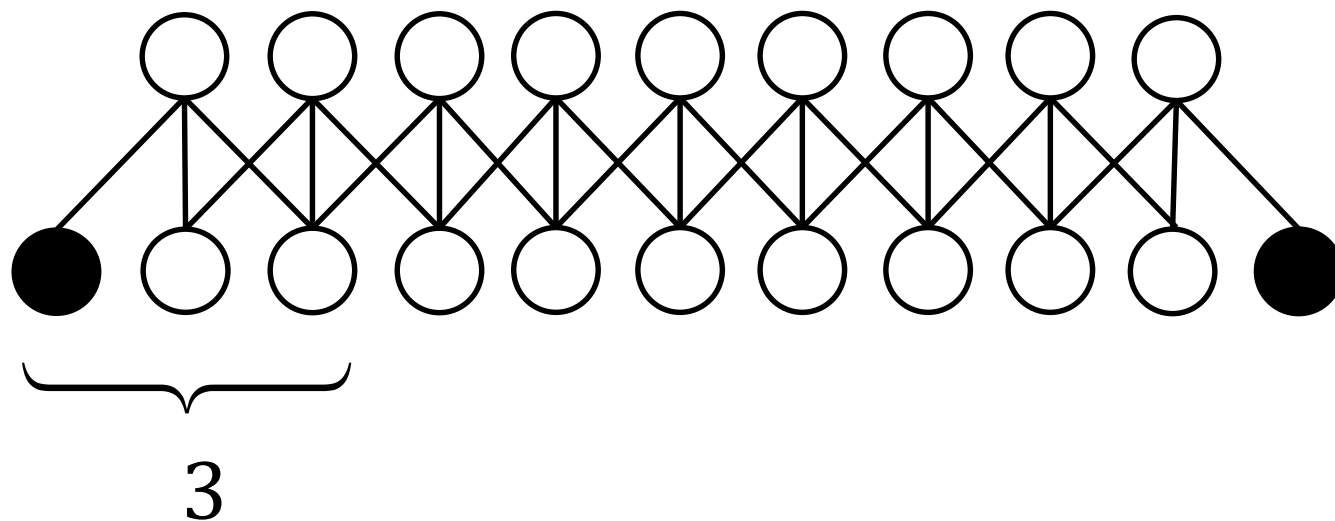
5.1.1 卷积层

➤ 例1: (边界延拓)卷积核尺寸为 3×3 , 步长 $S=1$, 延拓值padding为_____时满足输入与输出特征图长宽相同;

5.1.1 卷积层

➤ 例1: (边界延拓)卷积核尺寸为 3×3 , 步长 $S=1$, 延拓值padding为_____时满足输入与输出特征图长宽相同;

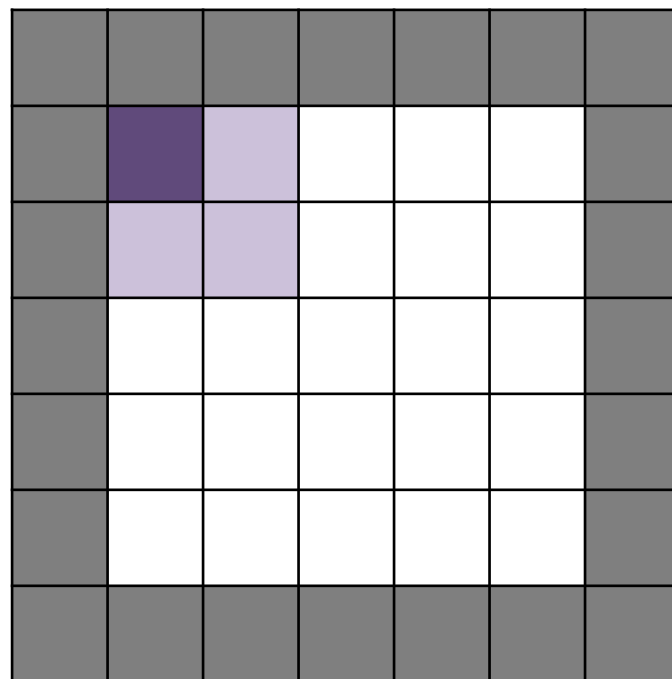
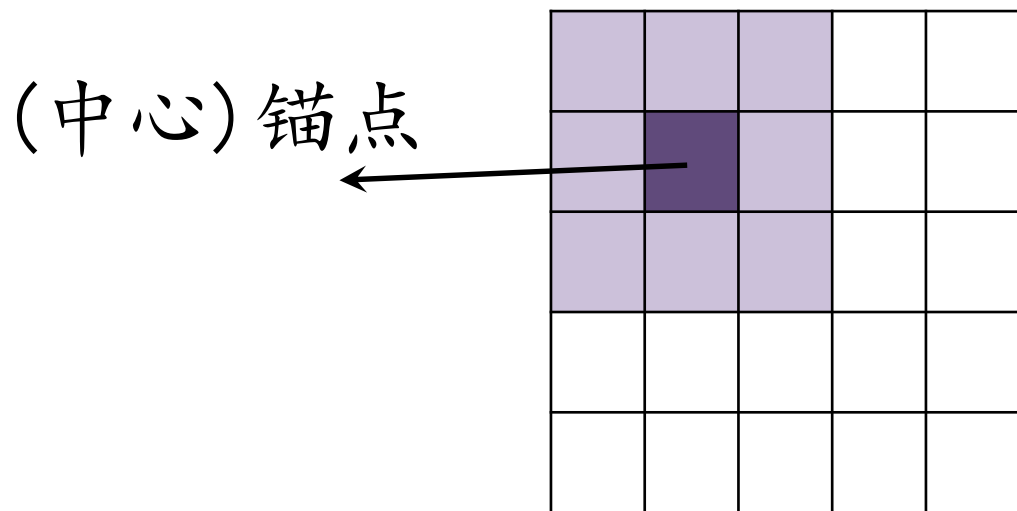
➤ 解答: 边界延拓为1时满足输入与输出特征图长宽相同



5.1.1 卷积层

➤ 例1: (边界延拓)卷积核尺寸为 3×3 , 步长 $S=1$, 延拓值padding为_____时满足输入与输出特征图长宽相同;

➤ 解答: 边界延拓为1时满足输入与输出特征图长宽相同

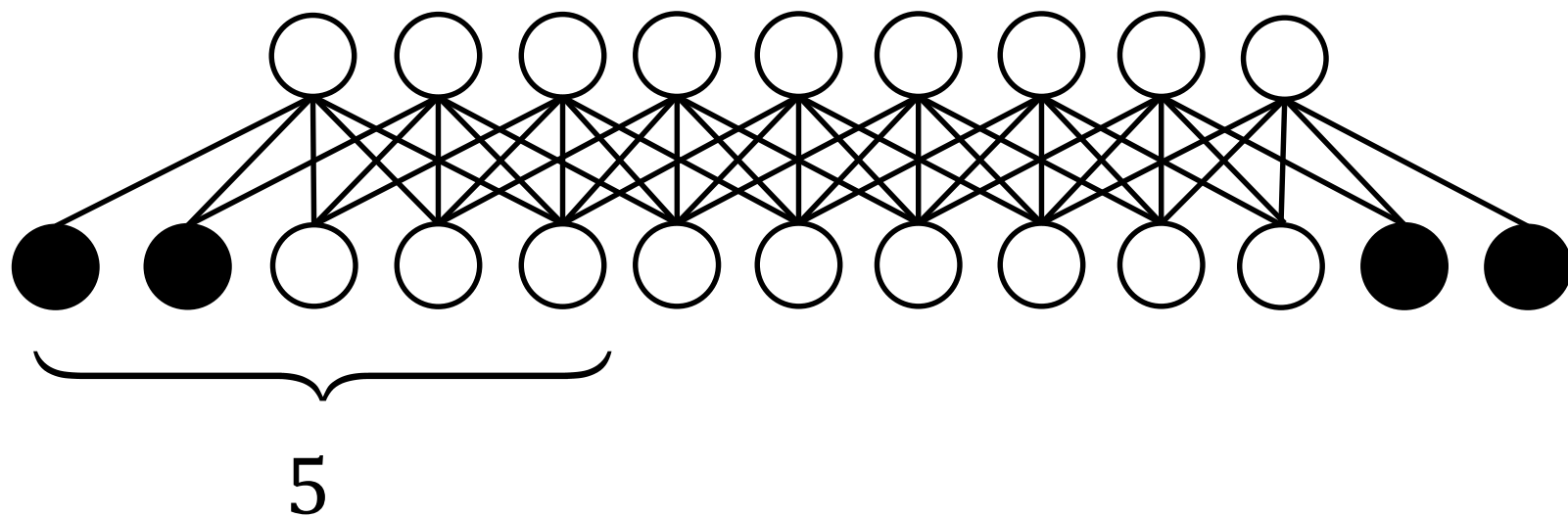


5.1.1 卷积层

➤ 例2: (边界延拓)卷积核尺寸为 5×5 , 步长 $S=1$, 延拓值padding为_____时满足输入与输出特征图长宽相同;

5.1.1 卷积层

- 例2：(边界延拓)卷积核尺寸为 5×5 ，步长 $S=1$ ，延拓值padding为_____时满足输入与输出特征图长宽相同；
- 解答：边界延拓为2时满足输入与输出特征图长宽相同



5.1.1 卷积层

➤ 例3: (边界延拓)卷积核尺寸为 $k \times k$, 步长 $S=1$, 延拓值padding为_____时满足输入与输出特征图长宽相同;

5.1.1 卷积层

➤ 例3：(边界延拓)卷积核尺寸为 $k \times k$ ，步长 $S=1$ ，延拓值padding为_____时满足输入与输出特征图长宽相同；

➤ 解答：边界延拓为 $(k - 1)/2$ 时满足输入与输出特征图长宽相同

➤ 思考1：卷积核尺寸为 $k \times k$ 时，锚点位置距离卷积核边界的距离？

➤ 思考2：为什么通常卷积核尺寸为 k 为奇数？

5.1.1 卷积层

- 例4: (参数量估计)假设卷积神经网络输入单幅图像, 某卷积层输入特征图大小为 $M \times W \times H$, 输出特征图大小为 $N \times W \times H$: 利用大小为 3×3 的卷积核, 该卷积层所需的参数数目是多少?

5.1.1 卷积层

➤例4：(参数量估计)假设卷积神经网络输入单幅图像，某卷积层输入特征图大小为 $M \times W \times H$ ，输出特征图大小为 $N \times W \times H$ ：利用大小为 3×3 的卷积核，该卷积层所需的参数数目是多少？

➤解答：

在没有偏置的情况下参数量 $9MN$

在有偏置的情况下参数量 $9MN + N$

参数量(不考虑偏置)： $K_W \cdot K_H \cdot C_{in} \cdot C_{out}$

参数量(考虑偏置)： $(K_W \cdot K_H \cdot C_{in} + 1) \cdot C_{out}$

5.1.1 卷积层

➤ 例5: (卷积层输出)假设卷积神经网络输入单幅图像, 某卷积层输入特征图大小为 $3 \times 32 \times 32$, 卷积层滤波器尺寸为 $64 \times 5 \times 5$, 步长stride为1, 边界延拓padding2, 输出特征图尺寸是?

5.1.1 卷积层

➤例5：(卷积层输出)假设卷积神经网络输入单幅图像，某卷积层输入特征图大小为 $3 \times 32 \times 32$ ，卷积层滤波器尺寸为 $64 \times 5 \times 5$ ，步长stride为1，边界延拓padding2，输出特征图尺寸是？

➤解答： $(32 + 2 \times 2 - 5) / 1 + 1 = 32$

因此输出特征图尺寸为 $64 \times 32 \times 32$

输出数据： $W_{out} = (W_{in} - K_W + 2P) / S + 1, H_{out} = (H_{in} - K_H + 2P) / S + 1$

5.1.1 卷积层

- 例6: (感受野)试比较两种网络设计的感受野与参数量？
- (a) 两个卷积核大小为 3×3 的卷积层前后连接
- (b) 一个卷积核大小为 5×5 的卷积层

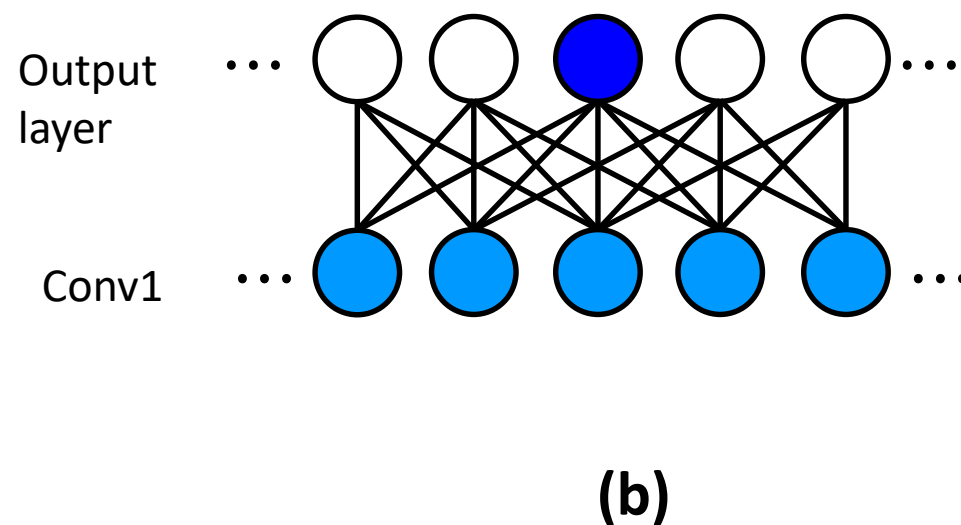
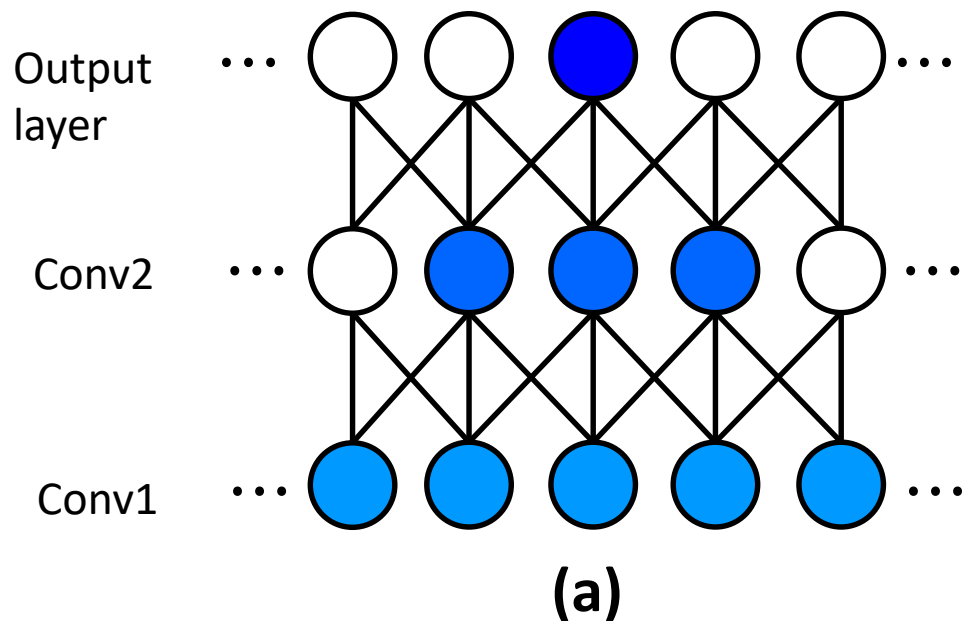
5.1.1 卷积层

➤ 例6: (感受野)试比较两种网络设计的感受野与参数量 ?

➤ (a) 两个卷积核大小为 3×3 的卷积层前后连接

➤ (b) 一个卷积核大小为 5×5 的卷积层

➤ 解答: (a)(b)的网络输出层局部感受野大小均为 5×5



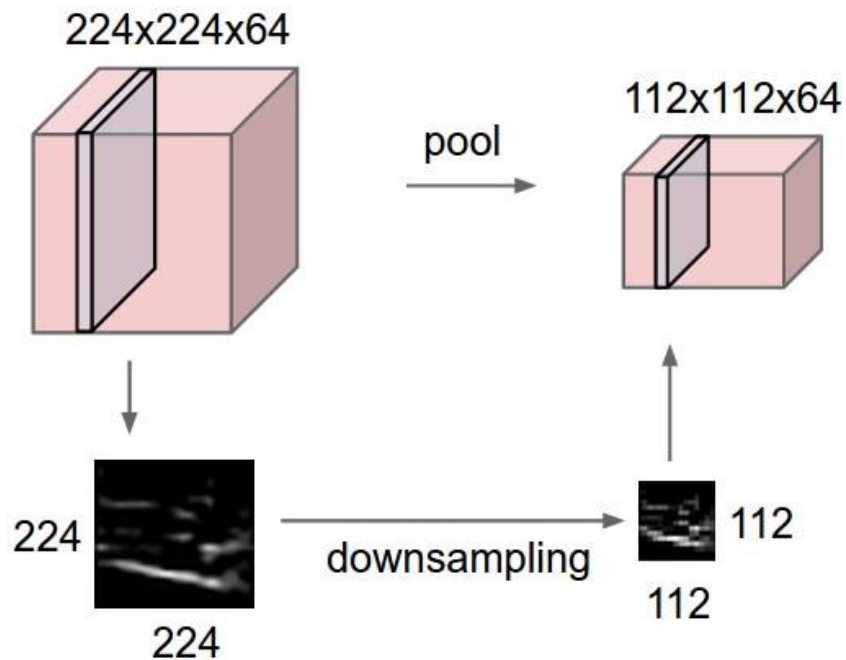
5.1.1 卷积层

- 例6: (感受野)试比较两种网络设计的感受野与参数量 ?
- (a) 两个卷积核大小为 3×3 的卷积层前后连接
- (b) 一个卷积核大小为 5×5 的卷积层
- 解答: (a)(b)的网络输出层局部感受野大小均为 5×5
- 假设该网络每层的通道数均为 C , 则:
 - (a)参数量: 单层卷积层的参数量为 $3 \times 3 \times C \times C = 9C^2$, 因此该网络总参数为 $18C^2$
 - (b)参数量: 单层卷积层的参数量为 $5 \times 5 \times C \times C = 25C^2$, 因此该网络总参数为 $25C^2$

5.1.2 池化层

➤ 池化层/下采样层 Pooling layer

- ✓ 保持主要信息，剔除次要信息
- ✓ 增加感受野，精炼特征图尺寸
- ✓ 仅对特征图单通道进行下采样，不考虑通道间关系



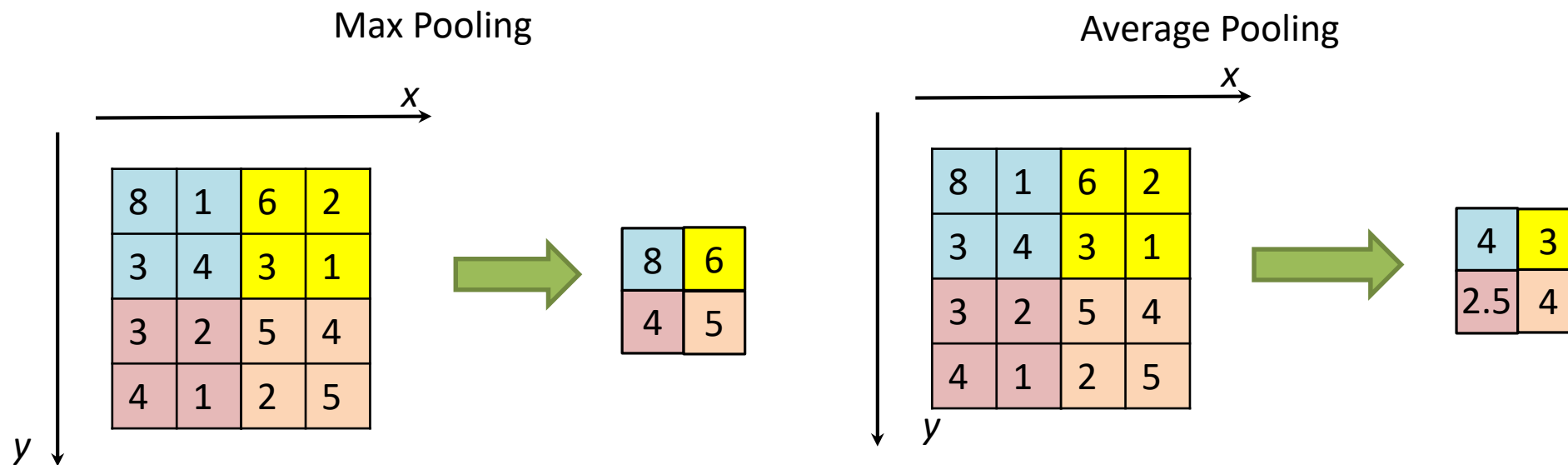
5.1.2 池化层

最大值池化Max Pooling

$$f_j^n(x, y) = \max_{\bar{x} \in N(x), \bar{y} \in N(y)} f_j^{n-1}(\bar{x}, \bar{y})$$

平均值池化Average Pooling

$$f_j^n(x, y) = \left(\sum_{\bar{x} \in N(x), \bar{y} \in N(y)} f_j^{n-1}(\bar{x}, \bar{y}) \right) / K$$



5.1.2 池化层 (pytorch)

MAXPOOL2D

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1,  
                           return_indices=False, ceil_mode=False)
```

[SOURCE]

Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \max_{m=0, \dots, kH-1} \max_{n=0, \dots, kW-1} \text{input}(N_i, C_j, \text{stride}[0] \times h + m, \text{stride}[1] \times w + n)$$

If `padding` is non-zero, then the input is implicitly zero-padded on both sides for `padding` number of points. `dilation` controls the spacing between the kernel points. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.

5.1.2 池化层 (pytorch)

AVGPOOL2D

```
CLASS torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False,  
    count_include_pad=True, divisor_override=None)
```

[\[SOURCE\]](#)

Applies a 2D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \frac{1}{kH * kW} \sum_{m=0}^{kH-1} \sum_{n=0}^{kW-1} input(N_i, C_j, stride[0] \times h + m, stride[1] \times w + n)$$

If `padding` is non-zero, then the input is implicitly zero-padded on both sides for `padding` number of points.

5.1.2 池化层

➤例6：已知某最大池化层的核大小为 2×2 ，步长为2，输入特征图尺寸为 $M \times W \times H$ ，试问该池化层的参数数量是？

5.1.2 池化层

➤例6：已知某最大池化层的核大小为 2×2 ，步长为2，输入特征图尺寸为 $M \times W \times H$ ，试问该池化层的参数量是？

➤解答：0

➤池化层不引入额外参数

5.1.2 池化层

➤例7：已知某最大池化层的核大小为 2×2 ，步长为2，

输入张量为：

0.37	0.28	0.10	0.19
0.18	0.79	0.31	0.37
0.24	0.04	0.37	0.37
0.53	0.53	0.37	0.37

5.1.2 池化层

➤例7：已知某最大池化层的核大小为 2×2 ，步长为2，

输入张量为：

0.37	0.28	0.10	0.19
0.18	0.79	0.31	0.37
0.24	0.04	0.37	0.37
0.53	0.53	0.37	0.37

➤解答：

0.37	0.28	0.10	0.19
0.18	0.79	0.31	0.37
0.24	0.04	0.37	0.37
0.53	0.53	0.37	0.37

0.79	0.37
0.53	0.37

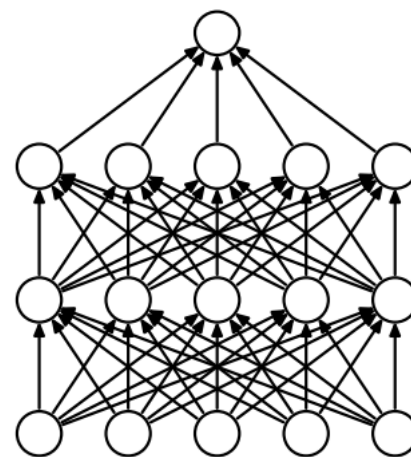
5.1.3 DropOut (Hinton,2012)

□ **训练阶段：**根据Bernoulli分布随机生成随机数 u ，并基于如下准则判断该节点参数是否更新

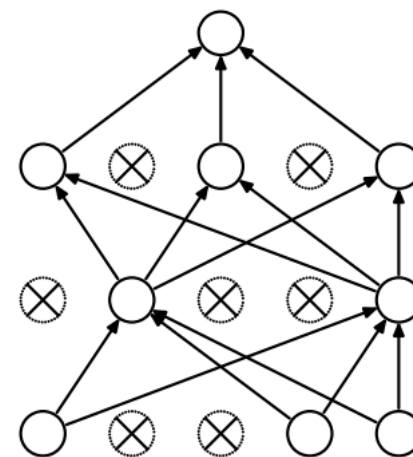
$$y_{train} = \begin{cases} \frac{x}{1-p} & \text{if } u > p \\ 0 & \text{otherwise} \end{cases}$$

- 通过隐式的模型集成来提升性能；
- 通过限制模型容量提高模型泛化能力。

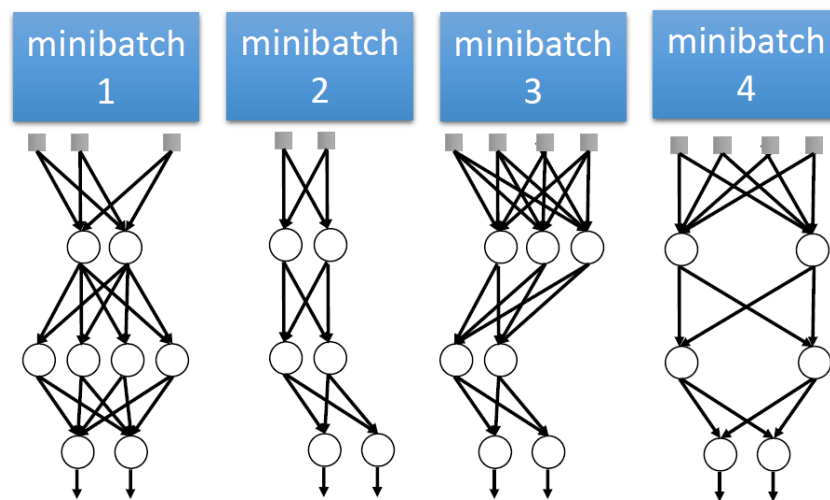
Hinton, G. E., et al. (2012). "Improving neural networks by preventing co-adaptation of feature detectors".



(a) Standard Neural Net



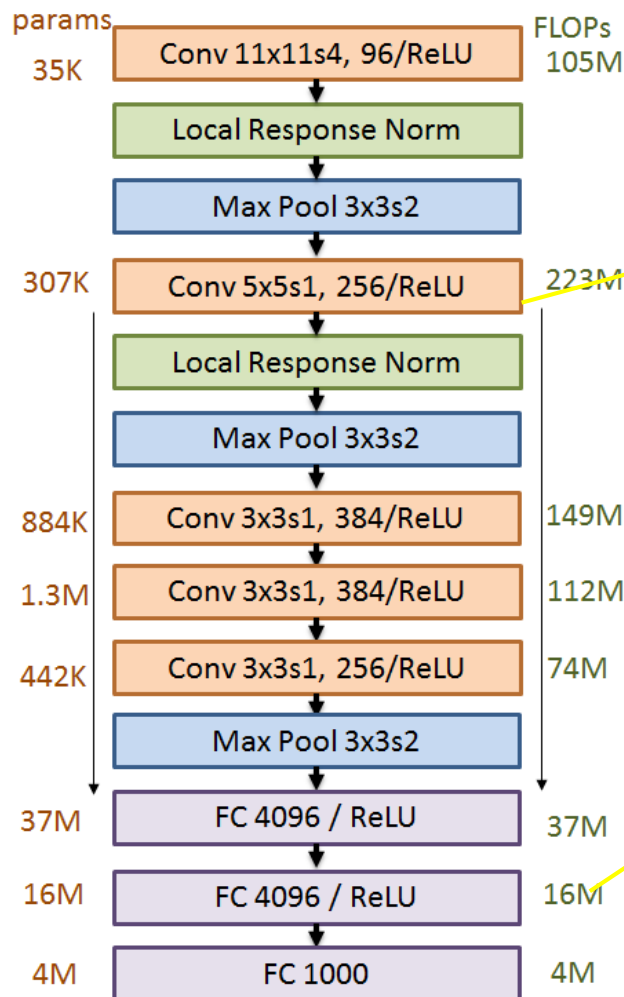
(b) After applying dropout.



变相在同层并行节点中引入激励机制

卷积神经网络

➤ 示例: AlexNet



```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super(AlexNet, self).__init__()
        self.feature = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.feature(x)
        x = x.view(x.size(0), 256 * 6 * 6) # reshape
        x = self.classifier(x)
        return x
```

5.1.4 Batch Normalization (Ioffe, 2015)

- ❑ 逐步尺度归一化，避免梯度消失和梯度溢出；
- ❑ 5x~20x加速收敛，提升泛化能力。
- ❑ 减少对初始化依赖。

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

// mini-batch mean

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

// mini-batch variance

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

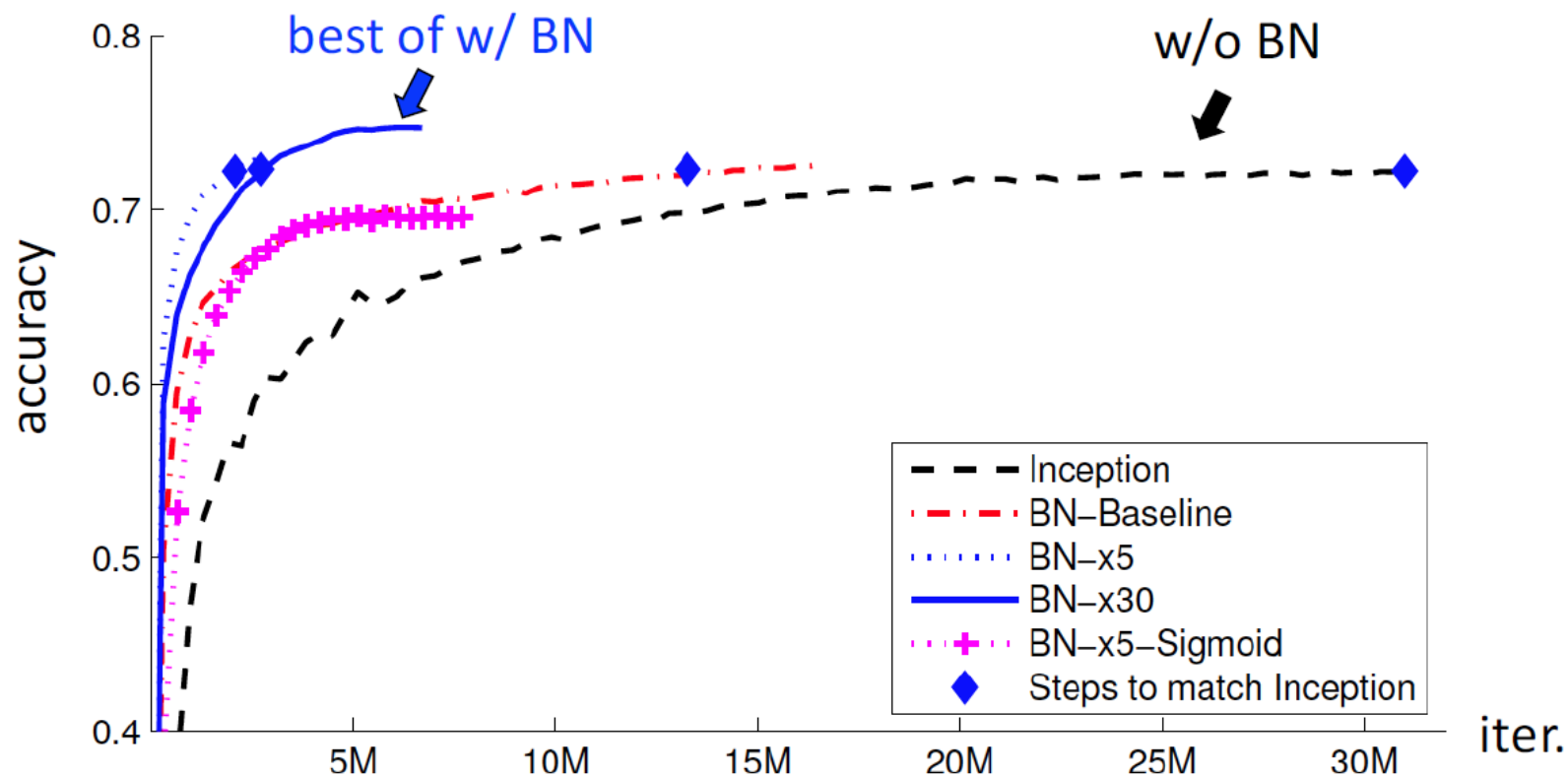
// normalize

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$$

// scale and shift

5.1.4 Batch Normalization (Ioffe, 2015)

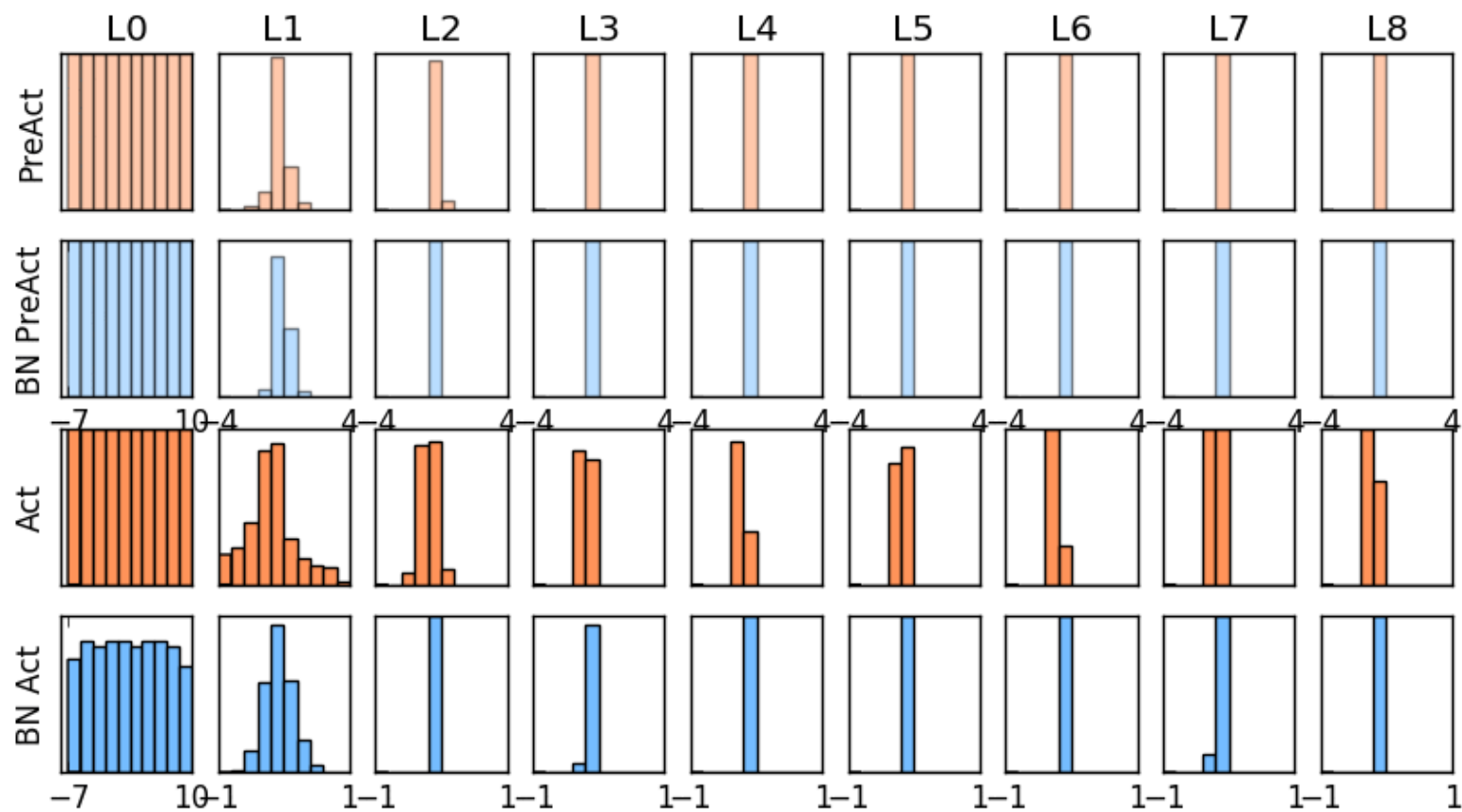
- 逐步尺度归一化，避免梯度消失和梯度溢出；
- 5x~20x加速收敛，提升泛化能力。
- 减少对初始化依赖。



Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML 2015

5.1.4 Batch Normalization (Ioffe, 2015)

- 逐步尺度归一化，避免梯度消失和梯度溢出；
- 5x~20x加速收敛，提升泛化能力。
- 减少对初始化依赖。



https://github.com/MorvanZhou/PyTorch-Tutorial/blob/master/tutorial-contents/504_batch_normalization.py

5.1.4 Batch Normalization (Ioffe, 2015)

BATCHNORM2D

```
CLASS torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True,  
                             track_running_stats=True)
```

[SOURCE]

Applies Batch Normalization over a 4D input (a mini-batch of 2D inputs with additional channel dimension) as described in the paper [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#).

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

The mean and standard-deviation are calculated per-dimension over the mini-batches and γ and β are learnable parameter vectors of size C (where C is the input size). By default, the elements of γ are set to 1 and the elements of β are set to 0. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.

Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default `momentum` of 0.1.

If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as well.

训练时，每个批次计算所用的均值和方差采用当前批次的统计量
测试时，每个批次计算所用的均值和方差采用训练完成时得到的统计量

卷积神经网络

➤ 示例：ResNet

layer name	output size	18-layer	34-layer
conv1	112×112		
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$
	1×1		avg
FLOPs		1.8×10^9	3.6×10^9

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_planes, planes, stride=1):
        super(BasicBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
                                stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != self.expansion*planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, self.expansion*planes,
                           kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(self.expansion*planes)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

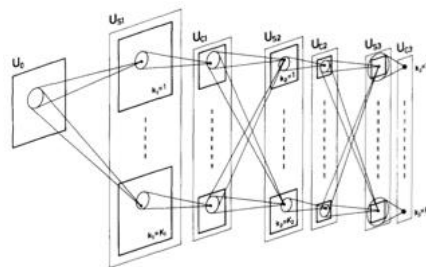
第5讲 卷积神经网络

5.1 基本单元

5.2 典型卷积神经网络架构

5.3 卷积神经网络应用

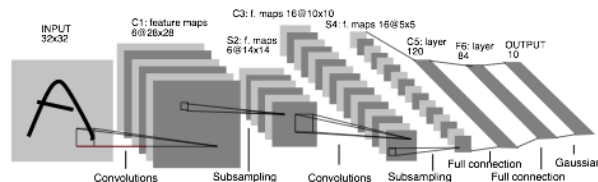
5.2 典型卷积神经网络架构



Fukushima 1980

Neocognitron

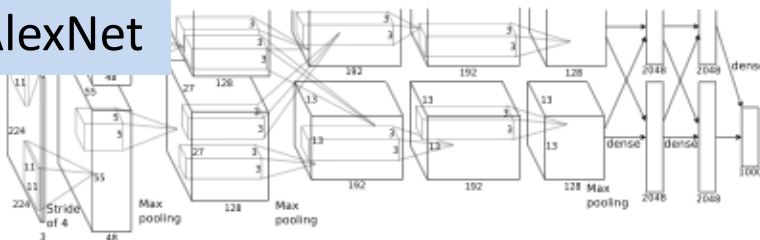
基于Hubel & Wiesel对视觉皮层的功能划分，分别设计简单神经元，复杂神经元，超复杂神经元，学习多层特征。



LeCun et al. 1989-1998

Hand-written digit reading

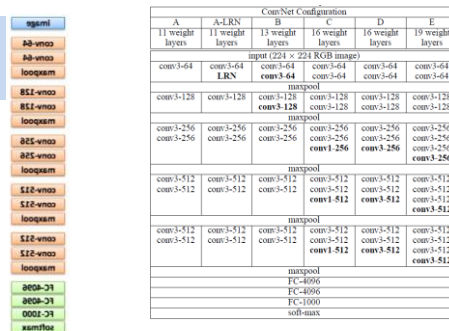
深度卷积神经网络



Krizhevsky, Sutskever, Hinton 2012

ImageNet classification breakthrough “SuperVision” CNN

GPU训练, imageNet百万量级数据



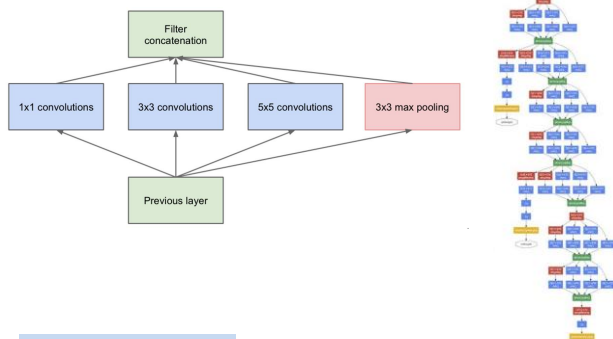
Simonyan, Zisserman 2014

Very deep convolutional networks for large-scale image recognition

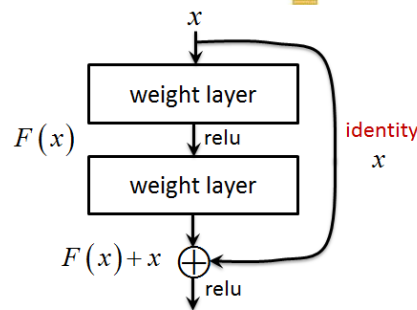
3x3卷积核，替代5x5, 7x7卷积核

5.2 典型卷积神经网络架构

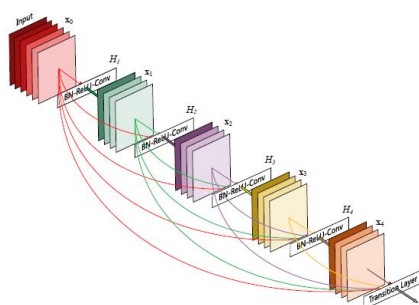
GoogLeNet



ResNet



DenseNet



Szegedy, Liu, Jia, Sermanet et al. 2015

Going Deeper with Convolutions

Szegedy, Ioffe, Vanhoucke, Alemi. 2016

Inception-v4, inception-resnet and the impact of residual connections on learning

Inception结构，1x1卷积用于特征降维

He et al. , 2016

Deep Residual Learning for Image Recognition (CVPR 2016 best paper)

残差结构，解决信息传递受阻

Huang, Liu, Weinberger, Maaten 2017

Densely connected convolutional networks (CVPR 2017 best paper)

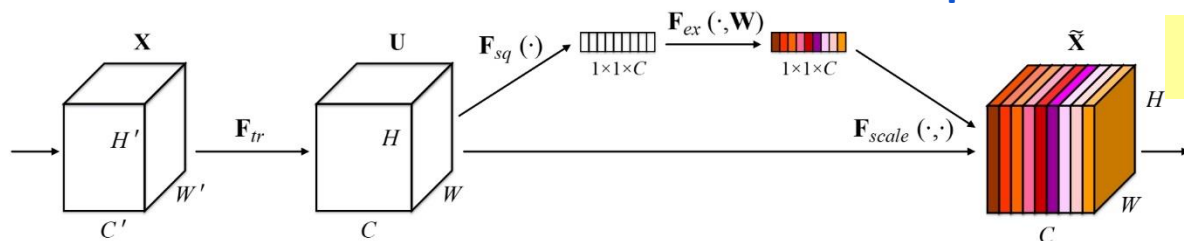
前层网络输出信息共享

5.2 典型卷积神经网络架构

SENet

Jie Hu, Li Shen, Gang Sun., CVPR 2018

Squeeze-and-Excitation Networks

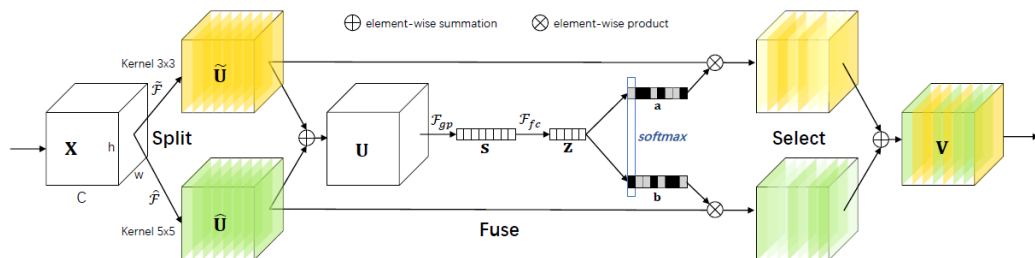


引入通道注意力机制

SKNet

Xiang Li, Wenhai Wang, Xiaolin Hu, Jian Yang, CVPR 2019.

Selective Kernel Networks

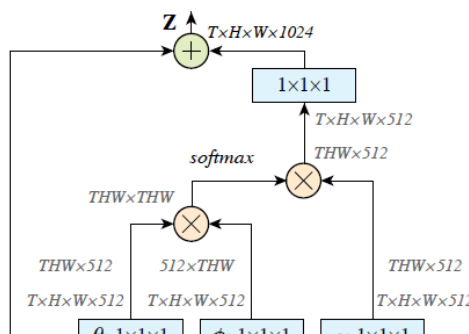


每层的感受野自适应调整，通过类通道选择机制调整

NonLocal/GCNet



自注意力机制来建模像素



Xiaolong Wang, Ross Girshick, Abhinav Gupta, Kaiming He, CVPR 2018.

Non-local Neural Networks

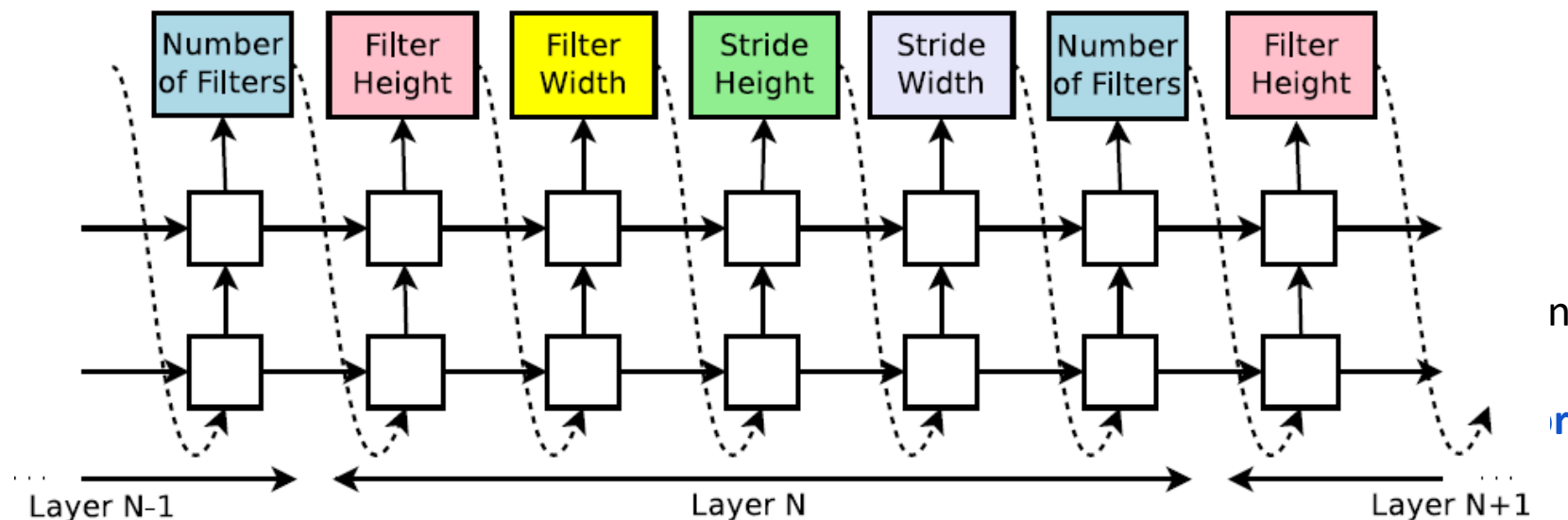
Yue Cao, Jiarui Xu, Stephen Lin, Fangyun Wei, Han Hu, 2019

GCNet: Non-local Networks Meet Squeeze-Excitation Networks and Beyond

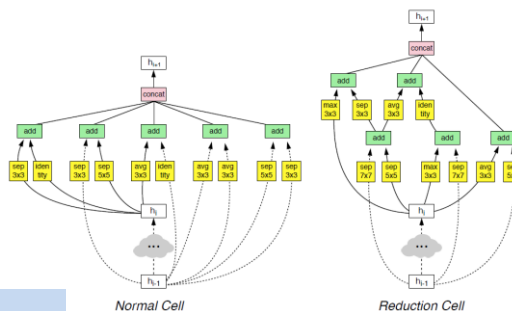
5.2 典型卷积神经网络架构

NAS: Neural Architecture Search

利用强化学习寻找最优网络



NASNet



AmoebaNet

Barret Zoph, Vijay Vasudevan, Jonathon Shlens, Quoc V. Le, CVPR 2018. **Learning Transferable Architectures for Scalable Image Recognition**

Normal Cell + Reduction Cell

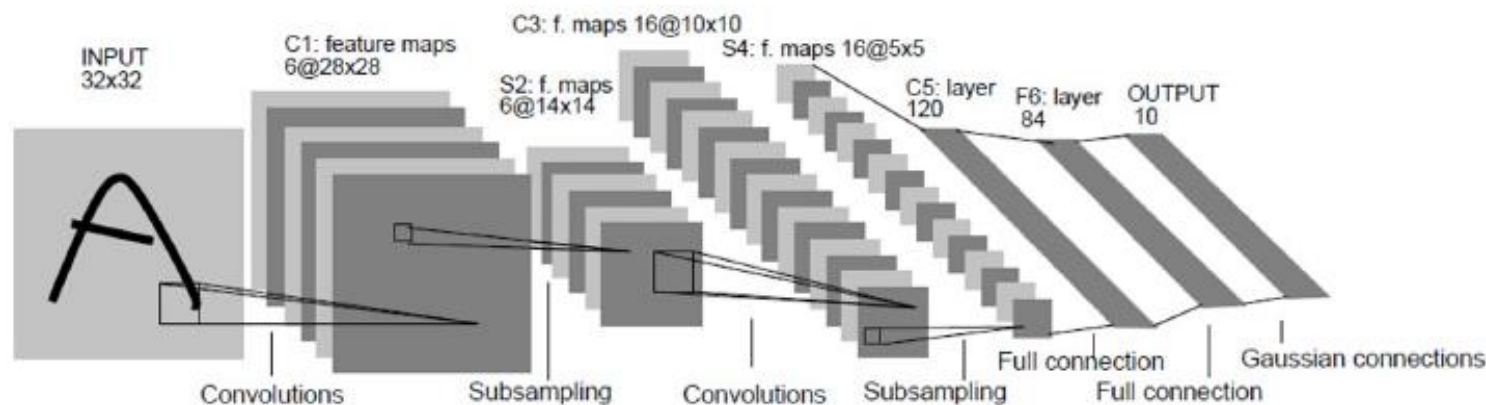
Esteban Real, Alok Aggarwal, Yanping Huang, Quoc V. Le
Regularized Evolution for Image Classifier Architecture Search

进化策略: Aging Evolution

5.2 典型卷积神经网络架构

➤ LeNet: 卷积神经网络的开山之作

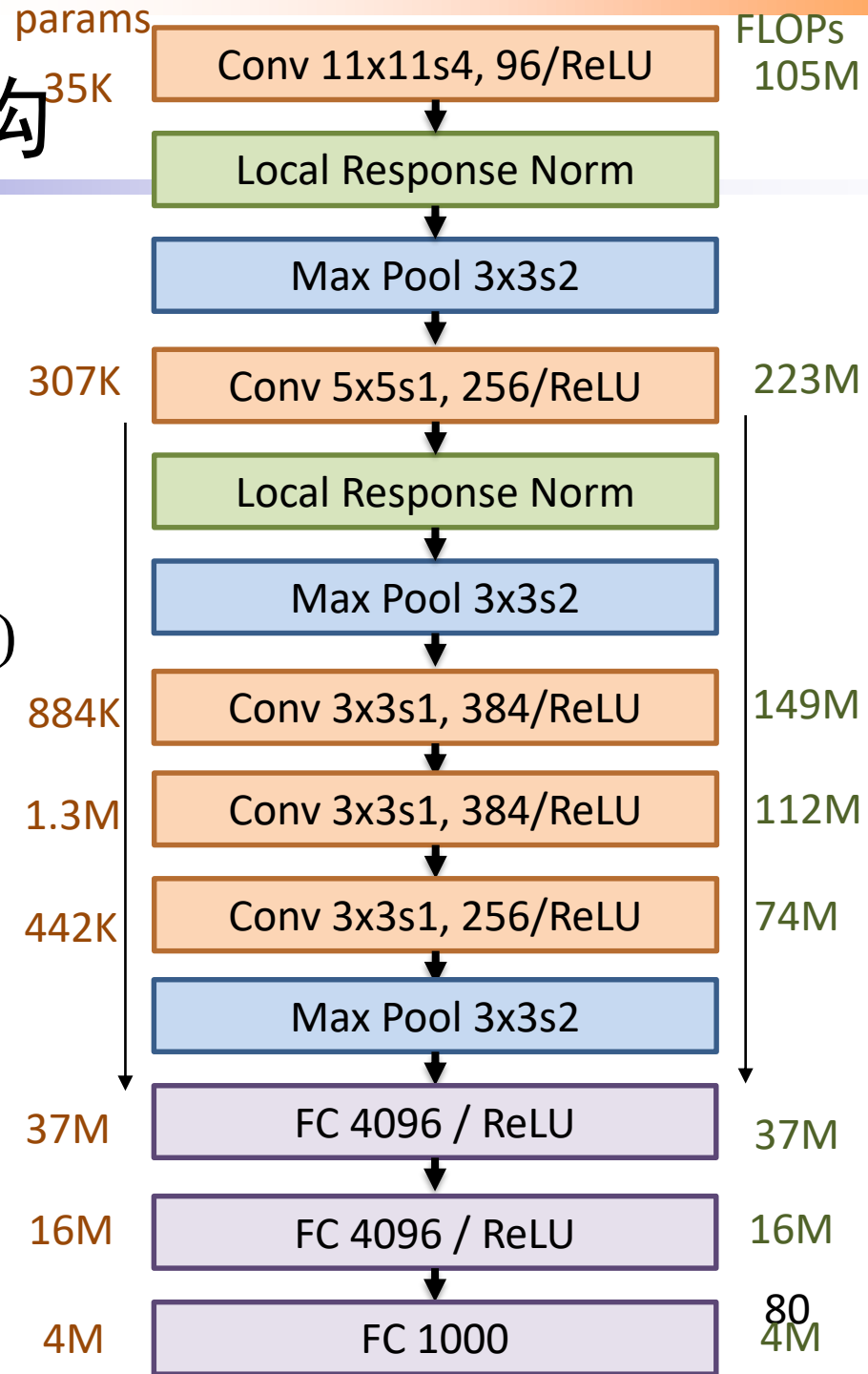
- ◆ C1 卷积层: 核大小 5×5 , 输出特征图大小为 $6 \times 28 \times 28$;
- ◆ S2 下采样层: 输出特征图大小为 $6 \times 14 \times 14$;
- ◆ C3 卷积层: 局部组合, 输出特征图大小为 $16 \times 10 \times 10$;
- ◆ S4 下采样层: 提取得到16个 5×5 的特征图;
- ◆ C5 卷积层: 卷积核大小为 1×1 , 提取120个特征图;
- ◆ F6 是全连接层;
- ◆ 输出层由径向基函数 (Radial Basis Function, RBF) 单元组成, 每个RBF单元计算输入向量和参数向量之间的欧式距离。输入离参数向量越远, RBF输出的越大。



5.2 典型卷积神经网络架构

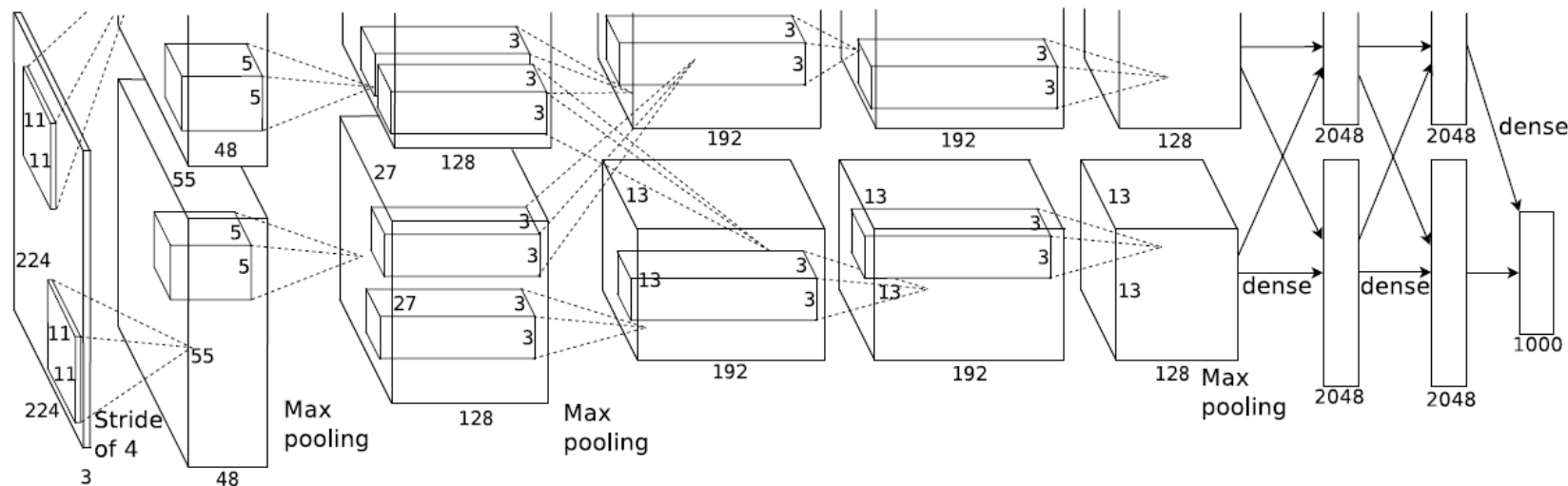
➤ AlexNet

- ◆ 前五层的卷积层，剩余三层的全连接层；
- ◆ ReLU非线性激活函数；
- ◆ 数据增强 (Data augmentation)
- ◆ Dropout
- ◆ 训练：双GPU并行，基于ImageNet百万量级数据



5.2 典型卷积神经网络架构

➤ AlexNet (case study)



- ◆ 输入RGB三通道图像归一化大小为227x227x3;
- ◆ 卷积层Conv1: 通道数96, 核大小11x11, 步长为4;
 - 输出特征图大小为: 55x55x96

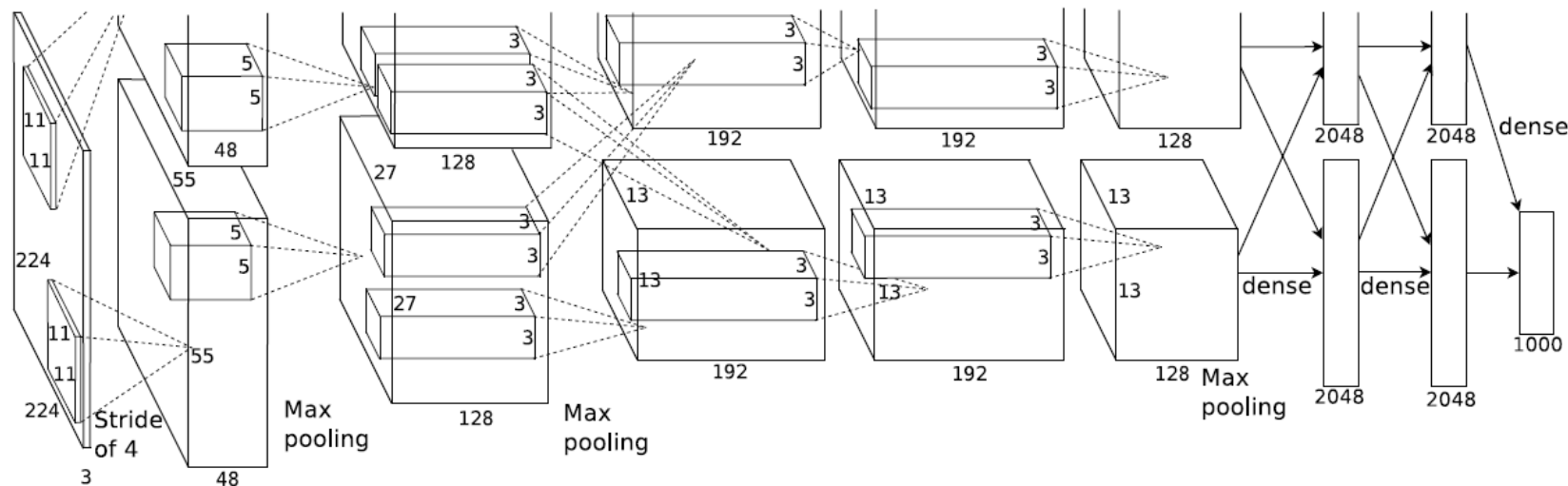
$$(227-11)/4+1 = 55$$

$$W_{out} = (W_{in} - K_W + 2P)/S + 1$$

$$H_{out} = (H_{in} - K_H + 2P)/S + 1$$

5.2 典型卷积神经网络架构

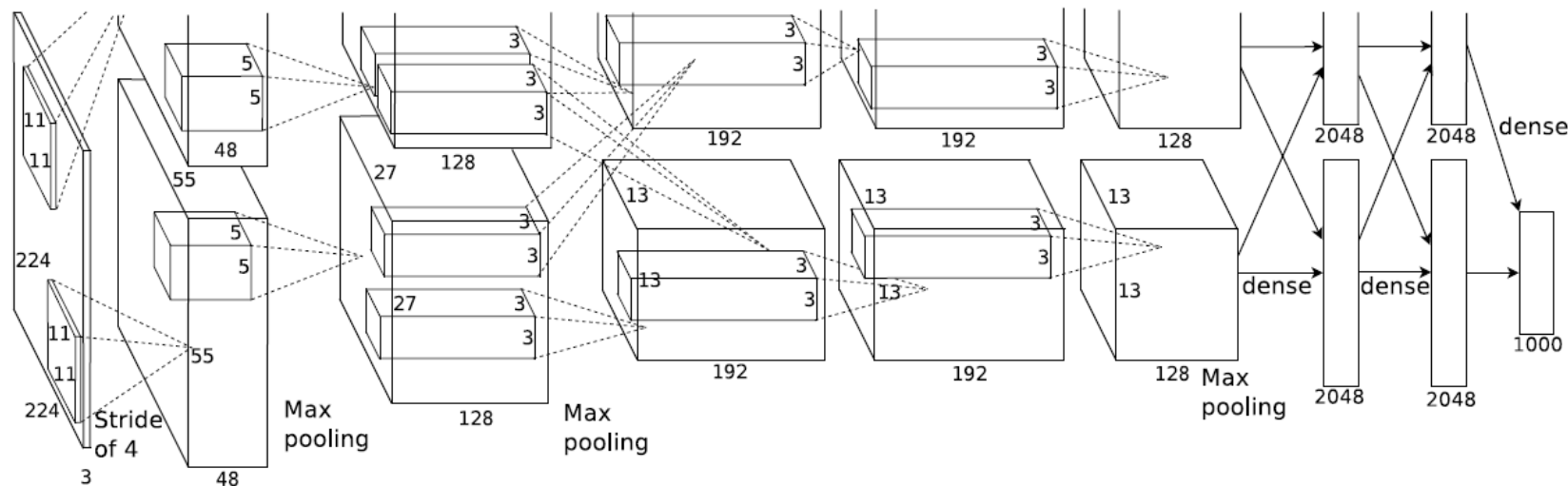
➤ AlexNet (case study)



- ◆ 输入RGB三通道图像归一化大小为227x227x3;
- ◆ 卷积层Conv1: 通道数96, 核大小11x11, 步长为4;
 - 输出特征图大小为: 55x55x96
 - 参数量: $(11*11*3)*96 = 35K$

5.2 典型卷积神经网络架构

➤ AlexNet (case study)



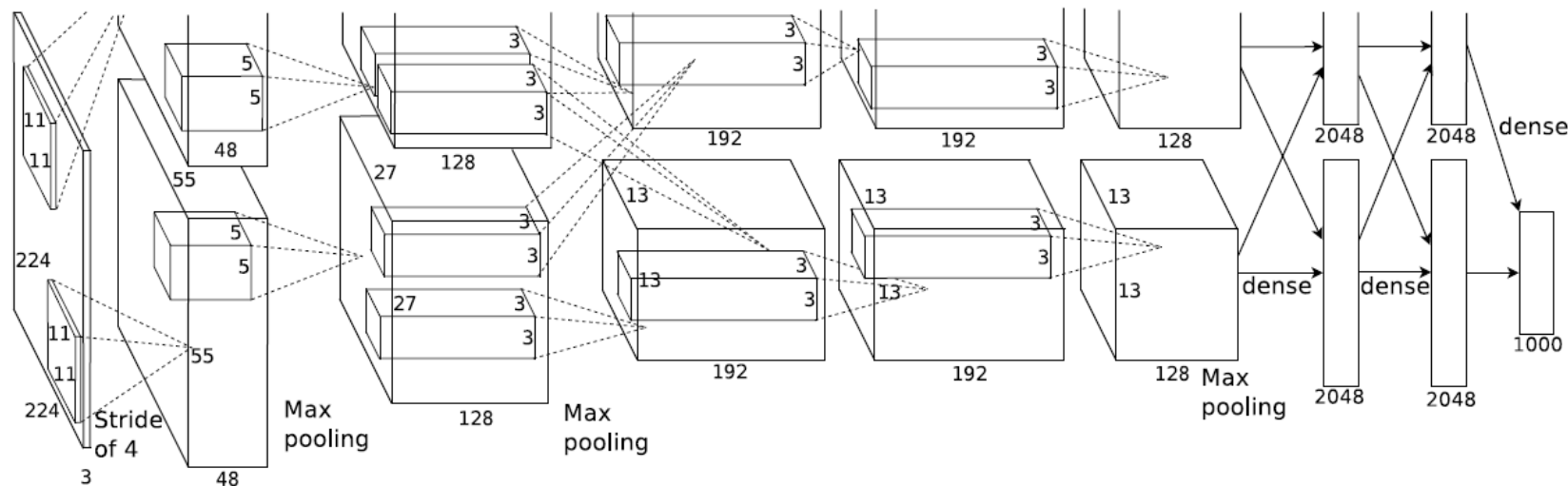
- ◆ Conv1输出特征图大小：55x55x96
- ◆ 池化层Pool1：核大小3x3, 步长为2;
 - 输出特征图大小为：27x27x96

$$(55-3)/2+1 = 27$$

$$W_{out} = (W_{in} - K_W + 2P)/S + 1$$
$$H_{out} = (H_{in} - K_H + 2P)/S + 1$$

5.2 典型卷积神经网络架构

➤ AlexNet (case study)



- ◆ Pool1输出特征图大小：27x27x96
- ◆ 卷积层Conv2：核大小3x3, 步长为2;
 - 输出特征图大小为：27x27x96

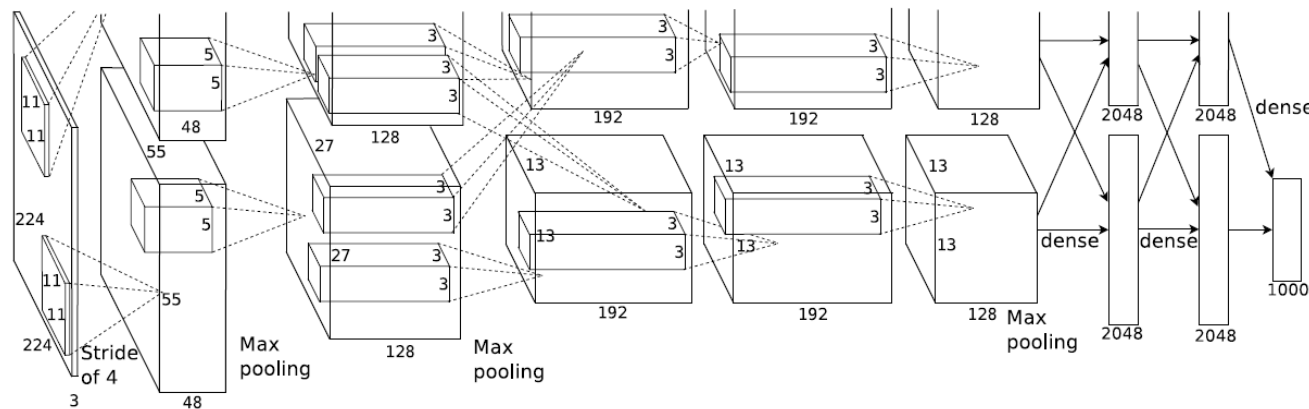
$$(55-3)/2+1 = 27$$

$$W_{out} = (W_{in} - K_W + 2P)/S + 1$$
$$H_{out} = (H_{in} - K_H + 2P)/S + 1$$

5.2 典型卷积神经网络架构

► AlexNet

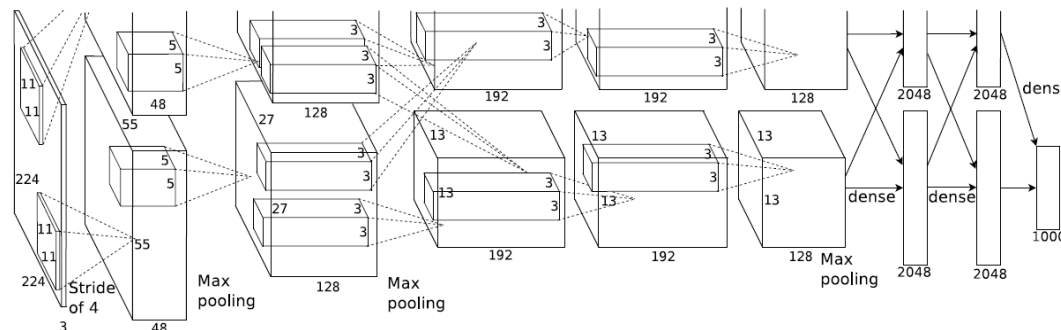
- ◆ 前五层的卷积层，剩余三层的全连接层；
- ◆ 输入图像归一化大小为 $227 \times 227 \times 3$ ；
- ◆ 第一个卷积层:96个,核大小 11×11 ,步长为4；
- ◆ 第二个卷积层:256(128×2)个,核大小 $5 \times 5 \times 48$ ；
- ◆ 第三个卷积层:384(192×2)个,核大小 $3 \times 3 \times 256$ ；
- ◆ 第四个卷积层:384(192×2)个,核大小为 $3 \times 3 \times 192$ ；
- ◆ 第五个卷积层:256(128×2)个核大小为 $3 \times 3 \times 192$ 的核。第三、第四和第五个卷积层之间没有局部归一化层和Pooling层。全连接层都各有4096个神经元。



5.2 典型卷积神经网络架构

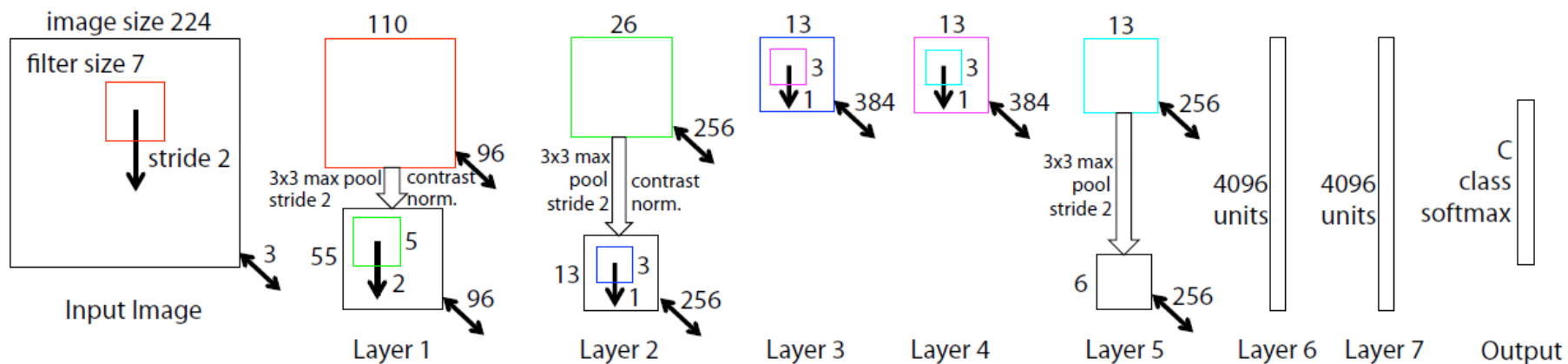
➤ AlexNet: Simplified architecture

- ◆ [227x227x3] INPUT
- ◆ [55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
- ◆ [27x27x96] MAX POOL1: 3x3 filters at stride 2
- ◆ [27x27x96] NORM1: Normalization layer
- ◆ [27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
- ◆ [13x13x256] MAX POOL2: 3x3 filters at stride 2
- ◆ [13x13x256] NORM2: Normalization layer
- ◆ [13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
- ◆ [13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
- ◆ [13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
- ◆ [6x6x256] MAX POOL3: 3x3 filters at stride 2
- ◆ [4096] FC6: 4096 neurons
- ◆ [4096] FC7: 4096 neurons
- ◆ [1000] FC8: 1000 neurons (class scores)



5.2 典型卷积神经网络架构

ZFNNet



◆ 相比 AlexNet:

- CONV1: change from (11x11 stride 4) to (7x7 stride 2)
- CONV3,4,5: instead of 384, 384, 256 filters use 512, 1024, 512

5.2 典型卷积神经网络架构

➤ VGGNet

- ◆ 卷积核大小固定为 3×3 ，利用多个叠加的最小 3×3 的卷积核的卷积层来模仿 $5 \times 5, 7 \times 7$ 的卷积层；

- ◆ 问：两个 3×3 卷积核叠加，感受野同 5×5 相同么？参数比较呢？

- ◆ 问：三个 3×3 卷积核叠加，感受野同 7×7 相同么？参数比较呢？

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

5.2 典型卷积神经网络架构

➤ VGGNet

- ◆ 典型版本：VGG16 和 VGG19；
- ◆ 两个网络的相同点是最后都有三层的全连接，且层与层之间均有Max Pooling；
- ◆ 卷积核大小固定为3x3，利用多个叠加的最小3x3的卷积核的卷积层来模仿5x5,7x7的卷积层；
- ◆ 通过降低卷积核的大小来减少网络参数，提升网络的深度以提升深度网络的性能。

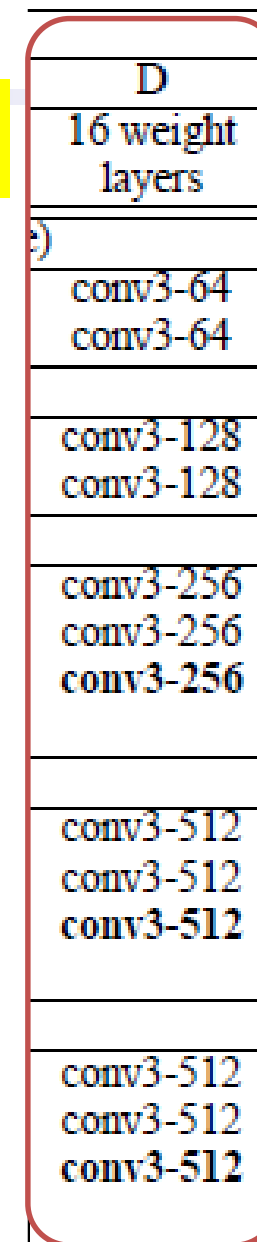
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

5.2 典型卷积神经网络架构

➤ VGGNet

TOTAL memory: 24M * 4 bytes ~= 93MB / image
TOTAL params: 138M parameters

- INPUT: [224x224x3] memory: 224*224*3=150K params: 0
- CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
- CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
- POOL2: [112x112x64] memory: 112*112*64=800K params: 0
- CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
- CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
- POOL2: [56x56x128] memory: 56*56*128=400K params: 0
- CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
- CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
- CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
- POOL2: [28x28x256] memory: 28*28*256=200K params: 0
- CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
- CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
- CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
- POOL2: [14x14x512] memory: 14*14*512=100K params: 0
- CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
- CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
- CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
- POOL2: [7x7x512] memory: 7*7*512=25K params: 0
- FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
- FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
- FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

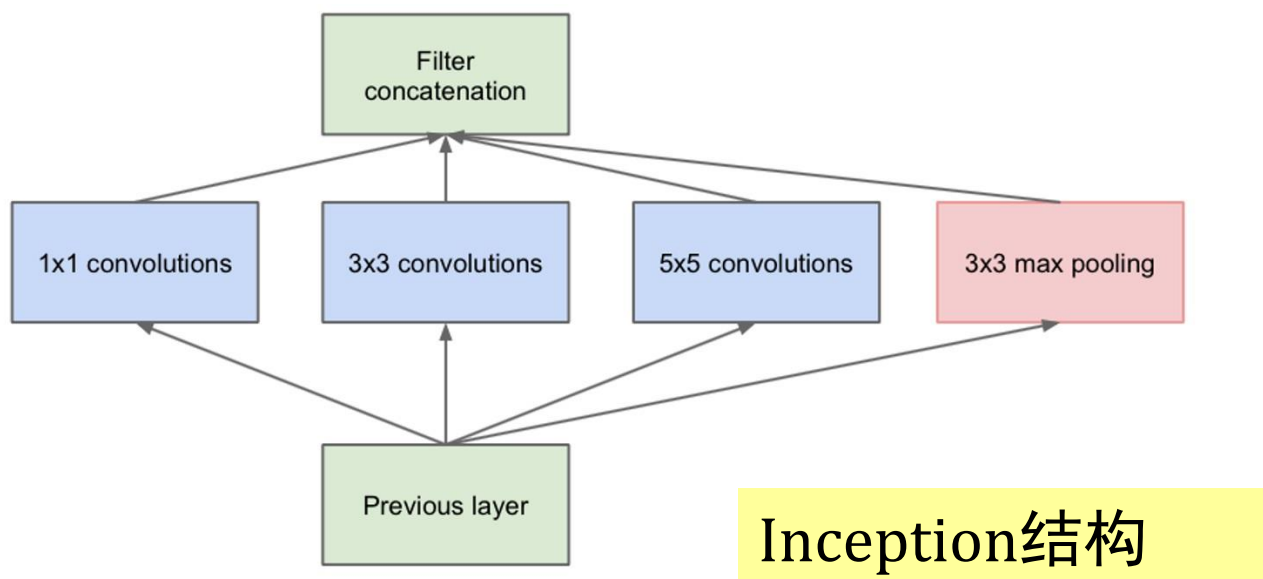


5.2 典型卷积神经网络架构

➤ GoogLeNet

◆ Inception结构:

- 利用1x1, 3x3, 5x5 卷积以及3x3 池化后的特征图进行深度上的合并，拓宽网络的广度，增加网络的尺度适应性。
- 多尺度感受野 (multiple receptive fields)

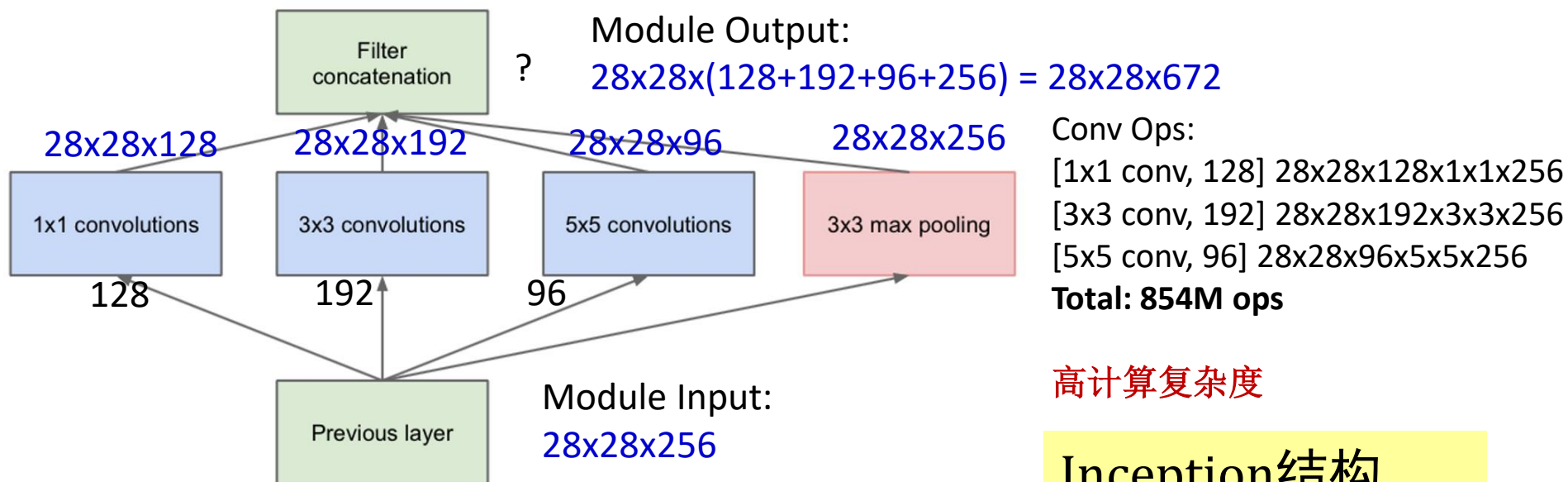


5.2 典型卷积神经网络架构

➤ GoogLeNet

◆ Inception结构:

- 利用1x1, 3x3, 5x5 卷积以及3x3 池化后的特征图进行深度上的合并, 拓宽网络的广度, 增加网络的尺度适应性;
- 多尺度感受野 (multiple receptive fields)

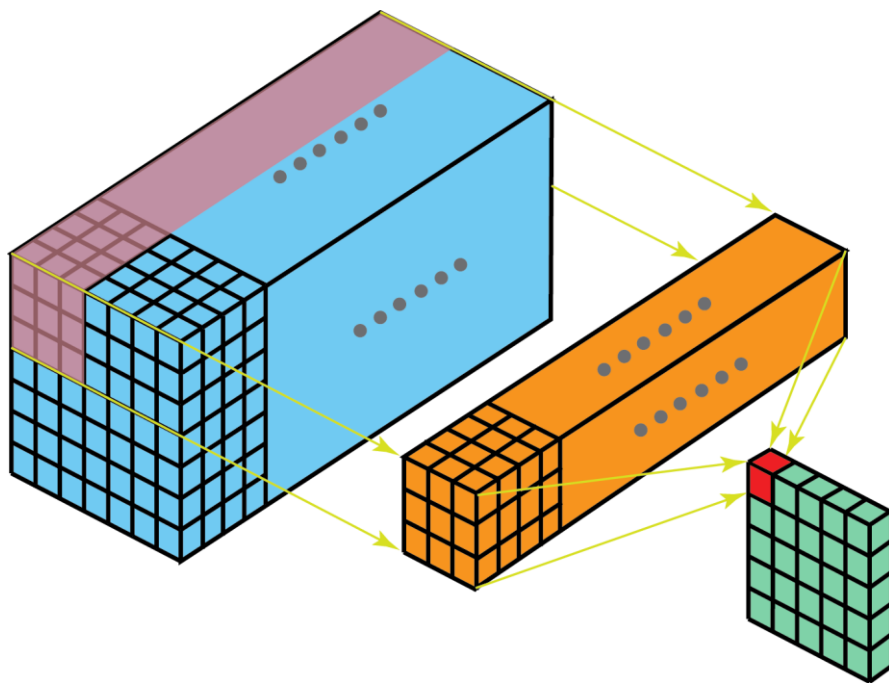


Inception结构

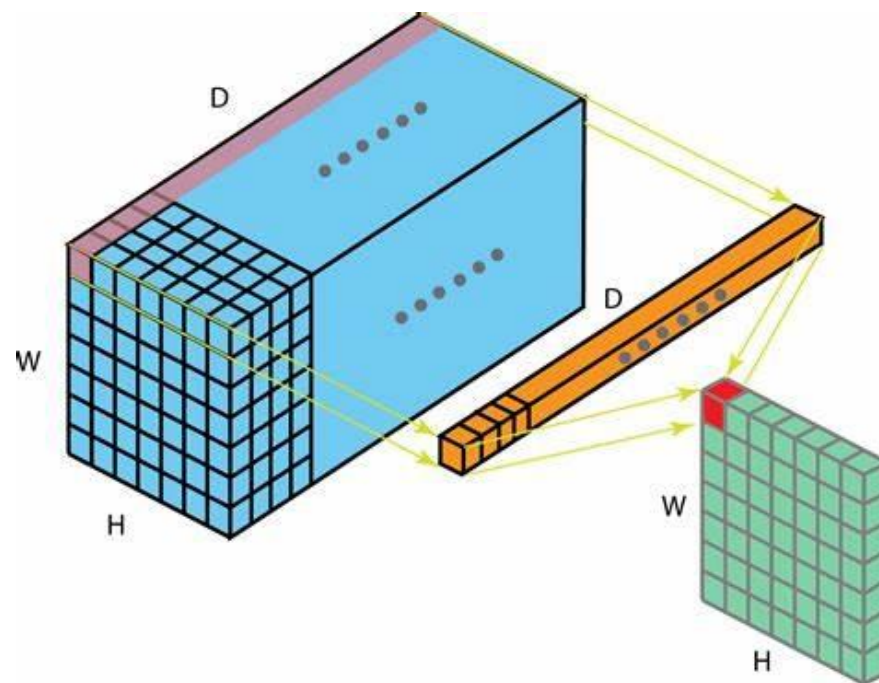
5.2 典型卷积神经网络架构

➤ GoogLeNet

- ◆ 带特征降维的Inception结构：
 - 1x1 卷积：特征降维与升维



3x3 卷积



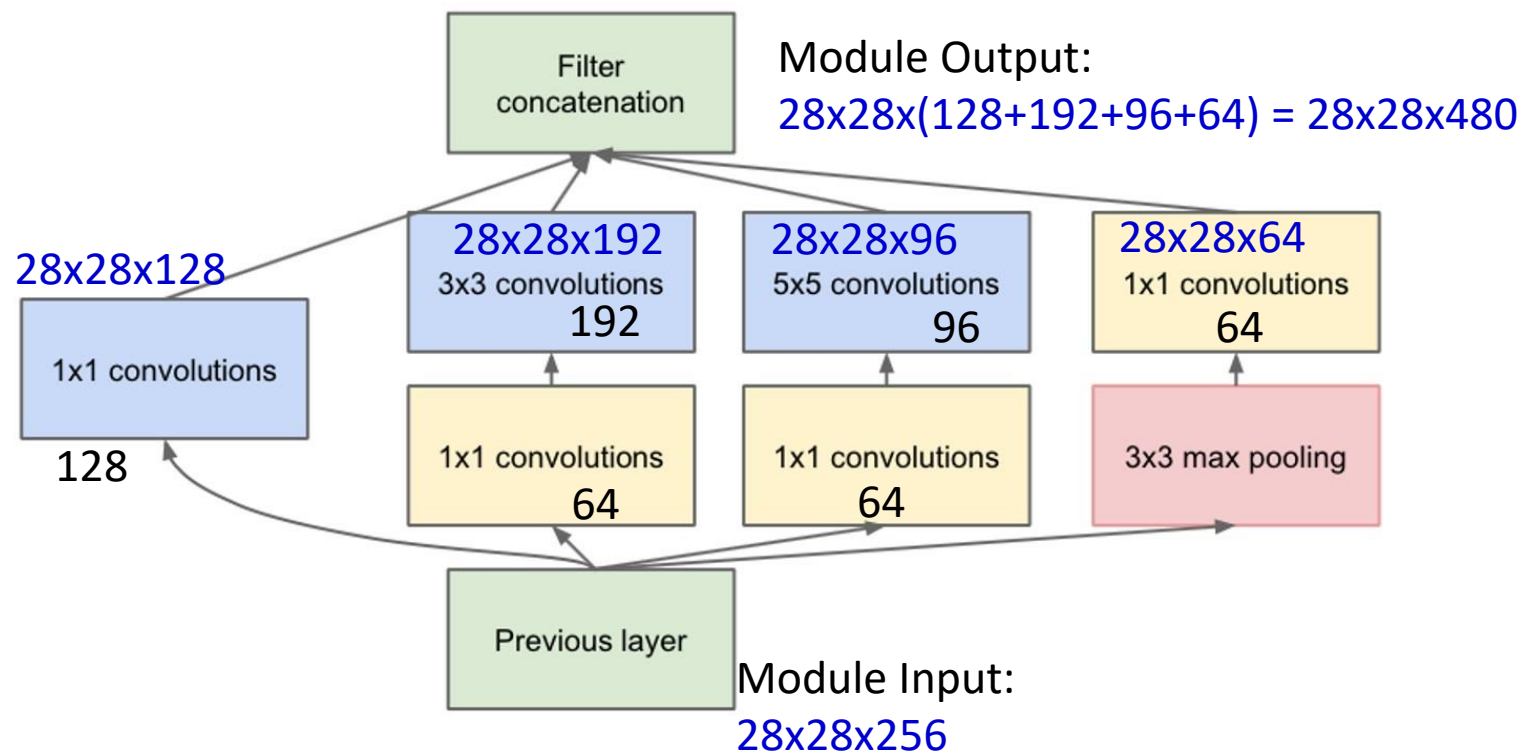
1x1 卷积

5.2 典型卷积神经网络架构

➤ GoogLeNet

◆ 带特征降维的Inception结构：

- 在3x3,5x5卷积层之前引入1x1 卷积；
- 在3x3 max pooling层之后引入1x1 卷积；

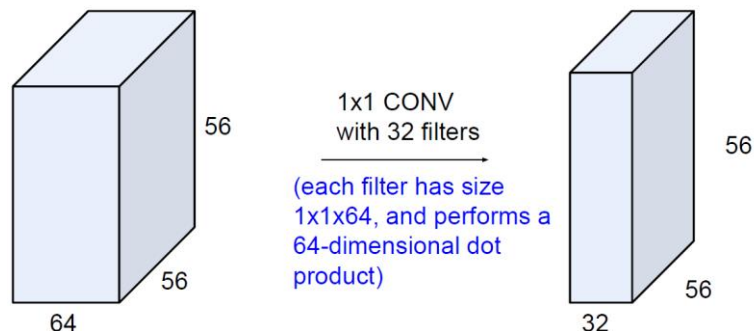
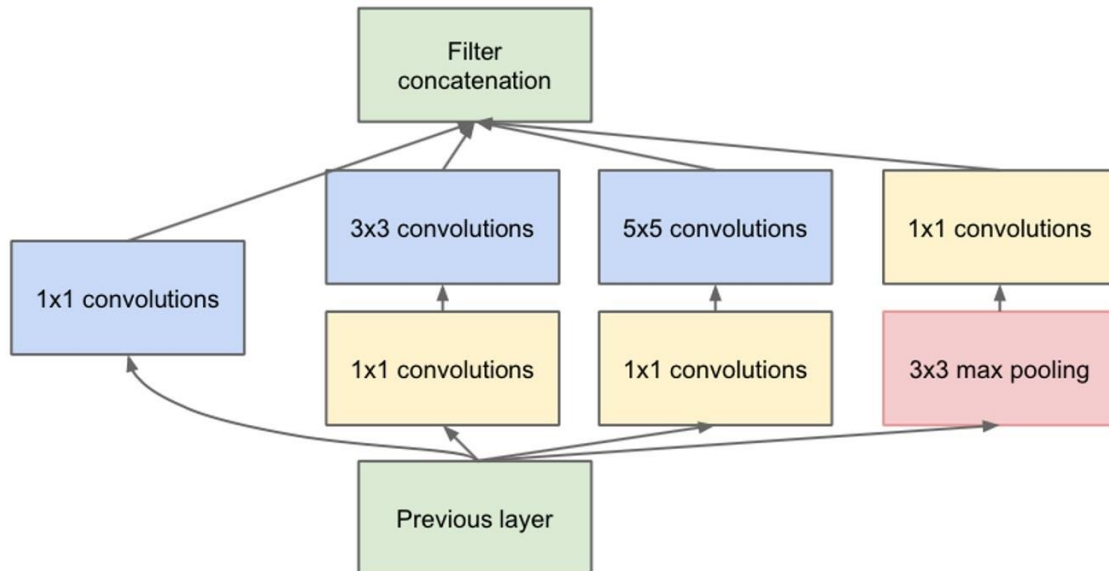


5.2 典型卷积神经网络架构

➤ GoogLeNet

◆ 带特征降维的Inception结构:

- 1x1 卷积: 特征降维与升维



Conv Ops:

[1x1 conv, 64] 28x28x64x1x1x256
[1x1 conv, 64] 28x28x64x1x1x256
[1x1 conv, 128] 28x28x128x1x1x256
[3x3 conv, 192] 28x28x192x3x3x64
[5x5 conv, 96] 28x28x96x5x5x64
[1x1 conv, 64] 28x28x64x1x1x256

Total: 358M ops

5.2 典型卷积神经网络架构

➤ Inception-Net

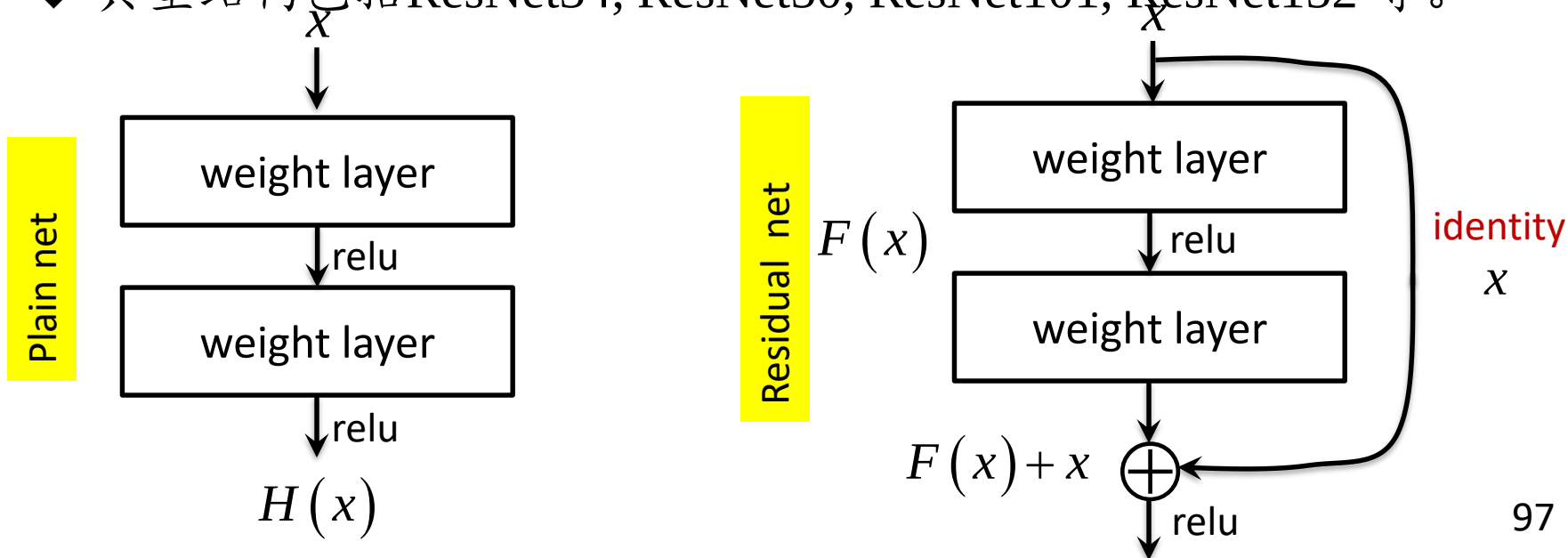
- ◆ Inception-V2 增加了BN 层，减少每层数据的内部方差漂移，利用两个3x3卷积滤波器代替5x5 的滤波器以减少参数；
- ◆ Inception-V3 通过分解增强了网络的非线性；
- ◆ Inception-V4 结合了残差网络的设计理念。



5.2 典型卷积神经网络架构

➤ ResNet(深度残差网络)

- ◆ 针对问题：信息传递受阻；
- ◆ 解决方案：普通浅层网络中引入“快捷连接”(shortcut connection), 转变成残差网络实现；
- ◆ 残差网络将输入作为参照，学习输入与输出之间的残差函数，输出表示为残差函数与输入之和；
- ◆ 残差函数更容易优化，提升网络的深度；
- ◆ 典型结构包括ResNet34, ResNet50, ResNet101, ResNet152 等。



5.2 典型卷积神经网络架构

➤ ResNet(深度残差网络)

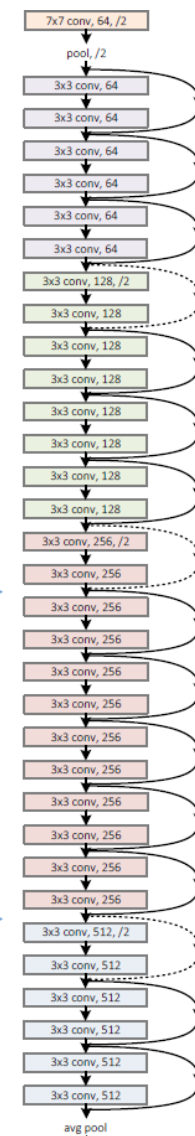
$$x_L = x_l + \sum_{i=l}^{L-1} F(x_i)$$



$$\frac{\partial E}{\partial x_l} = \frac{\partial E}{\partial x_L} \frac{\partial x_L}{\partial x_l} = \frac{\partial E}{\partial x_L} \left(1 + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} F(x_i) \right)$$

$$\frac{\partial E}{\partial x_l}$$

$$\frac{\partial E}{\partial x_L}$$



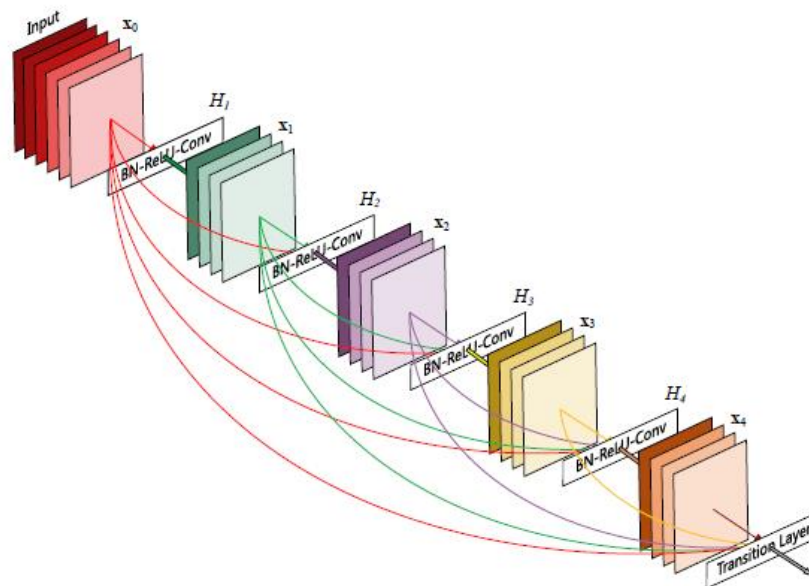
5.2 典型卷积神经网络架构

➤ DenseNet

◆ 稠密连接：每层之前层的输出为输入，对于有 L 层的传统网络，

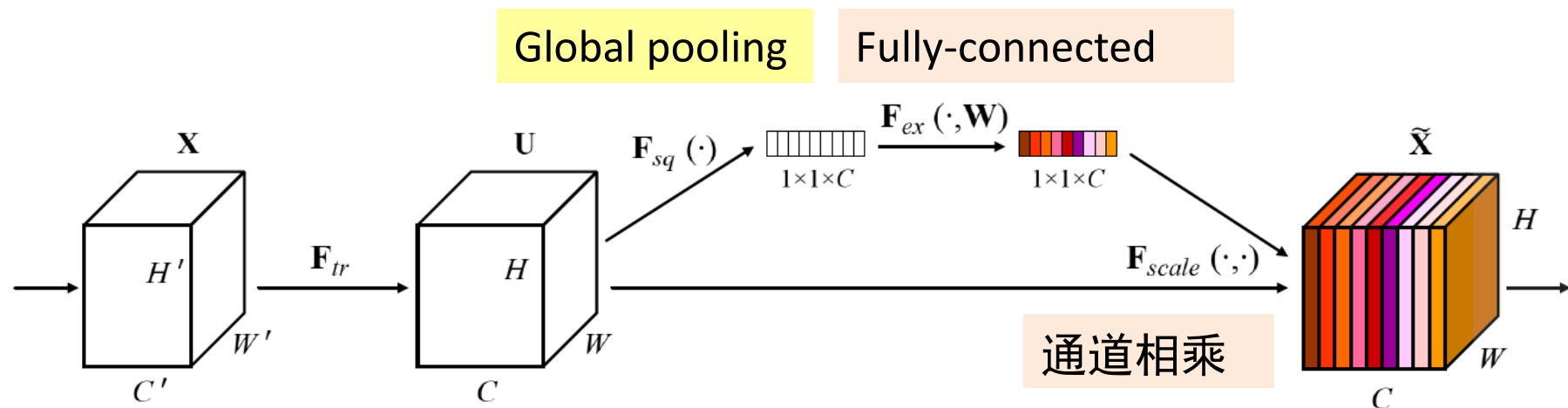
一共有 $\frac{L(L+1)}{2}$ 个连接

- ✓ 尽量缩短前层和层之间的连接, 最大化信息的流动
- ✓ 有效解决梯度消失问题；强化特征传播；支持特征重用；大幅度减少参数数量



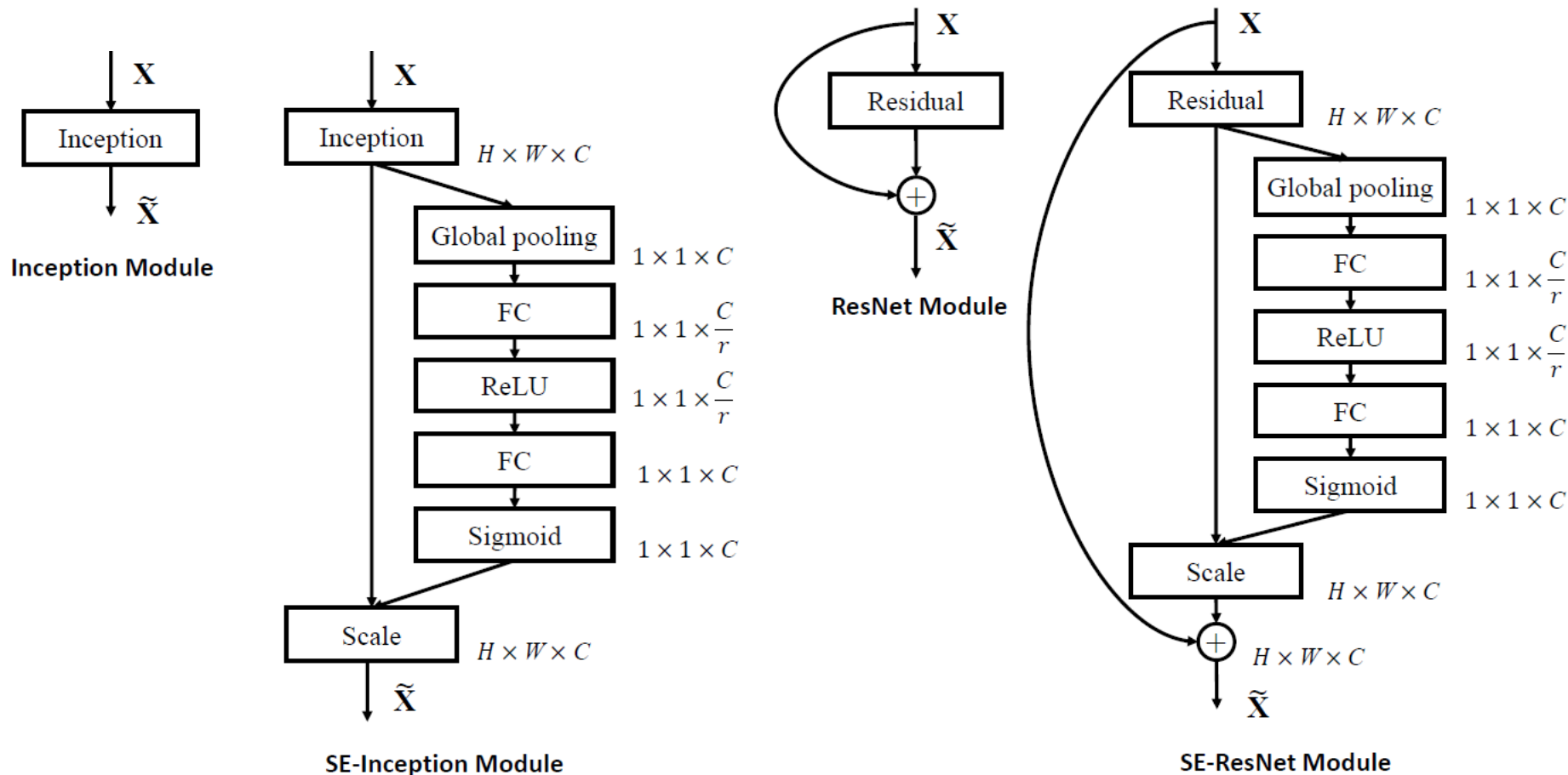
5.2 典型卷积神经网络架构

➤ SENet: 通道注意力机制



5.2 典型卷积神经网络架构

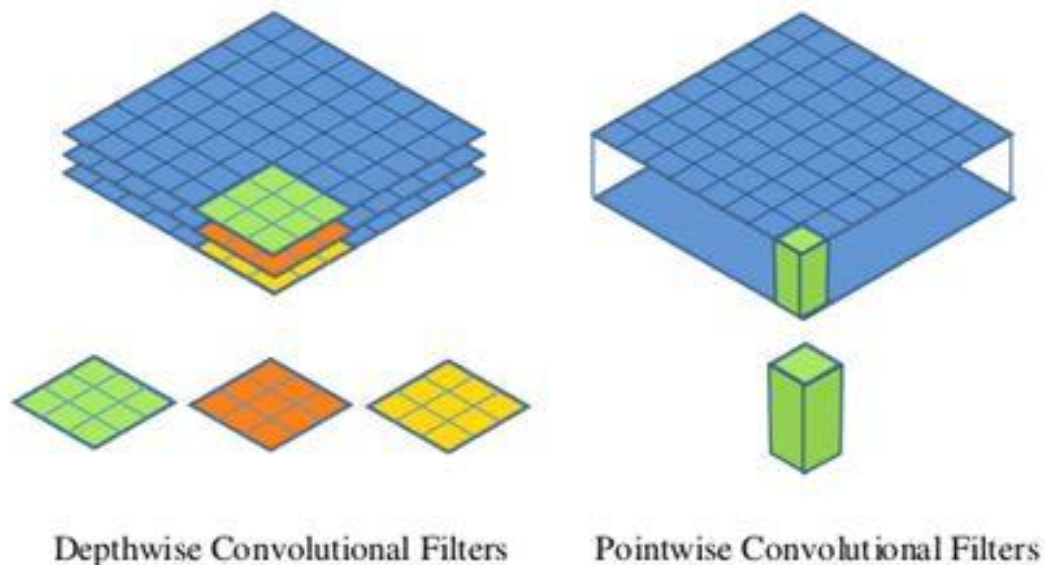
➤ SENet: 通道注意力机制



5.2 典型卷积神经网络架构

➤ MobileNet

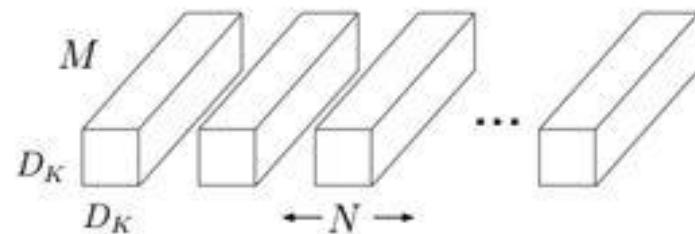
- ✓ 基本单元：深度可分离卷积；
- ✓ 首先是采用depthwise convolution对不同输入通道分别进行卷积，然后采用pointwise convolution将上面的输出再进行结合；



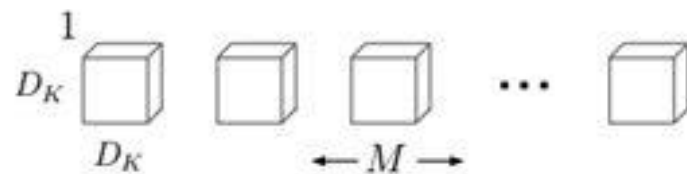
5.2 典型卷积神经网络架构

➤ MobileNet

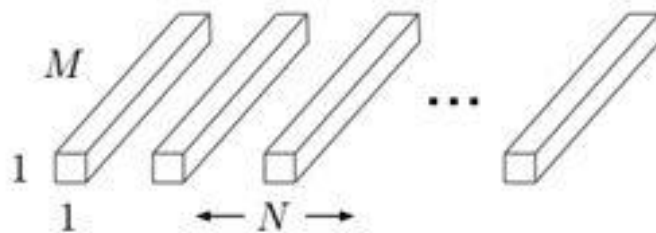
- ✓ 基本单元：深度可分离卷积；
- ✓ 首先是采用depthwise convolution对不同输入通道分别进行卷积，然后采用pointwise convolution将上面的输出再进行结合；
- ✓ 整体效果和标准卷积差不多，可大大减少计算量和模型参数量。



(a) Standard Convolution Filters



(b) Depthwise Convolutional Filters

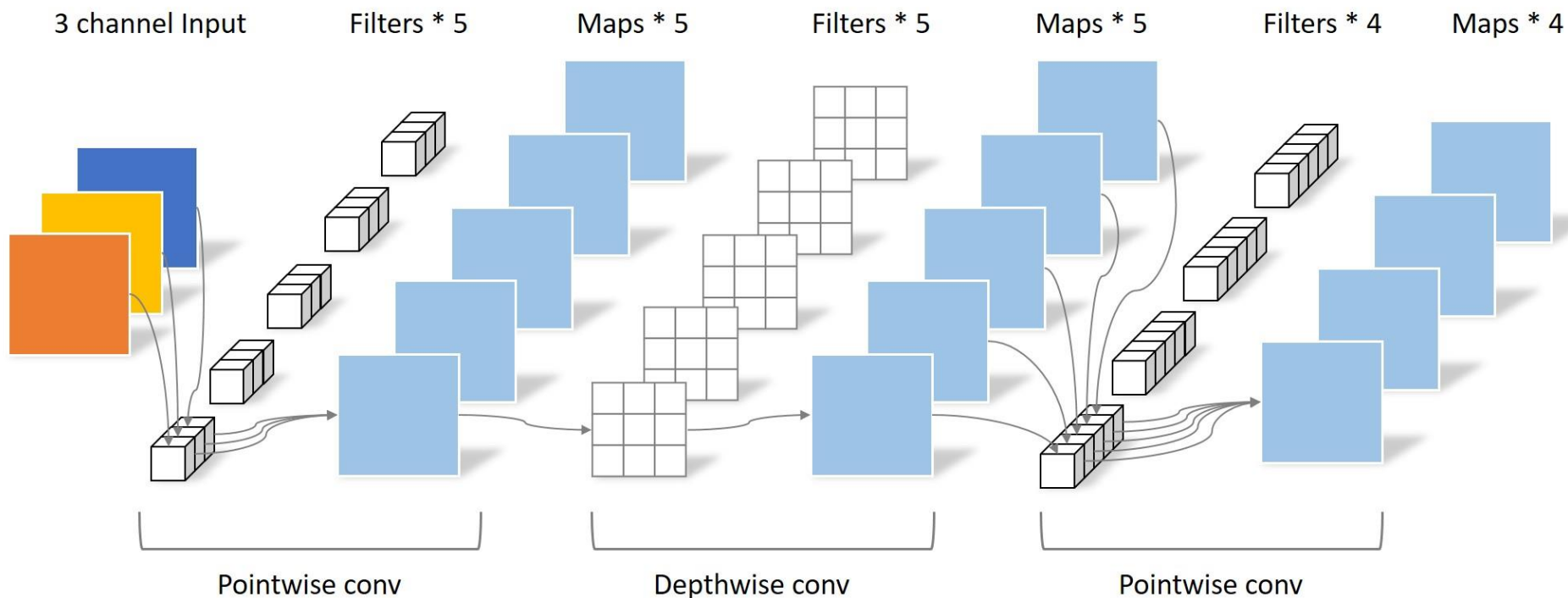


(c) 1×1 Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution

5.2 典型卷积神经网络架构

➤ MobileNet

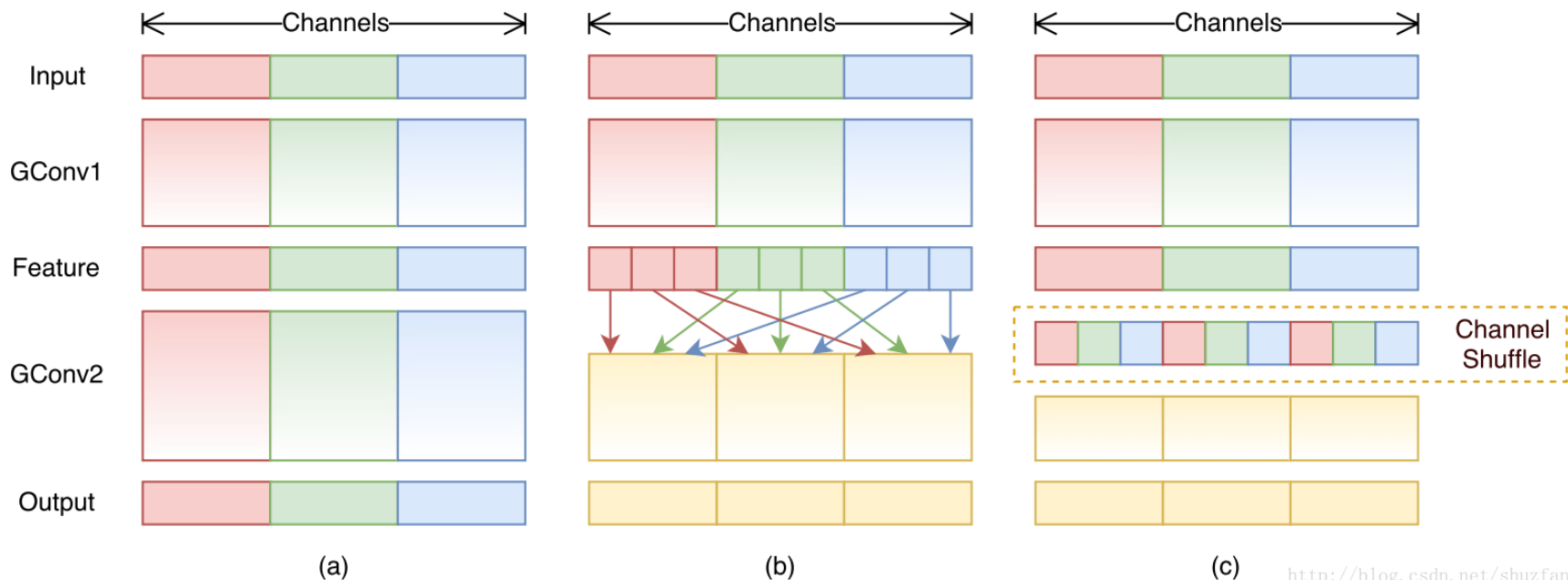
✓ 基本单元：深度可分离卷积；



5.2 典型卷积神经网络架构

➤ ShuffleNet

- ✓ 利用分组卷积，但分组卷积会造成输出通道只和某些输入通道有关；
- ✓ 利用channel shuffle解决全局信息流通不畅，网络表达能力不足的问题



第5讲 卷积神经网络

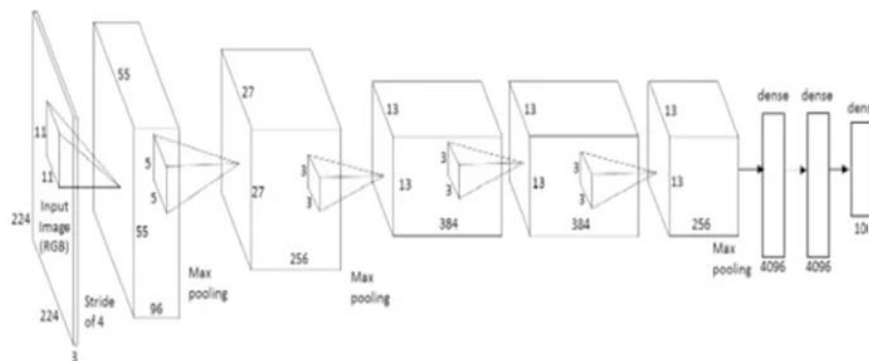
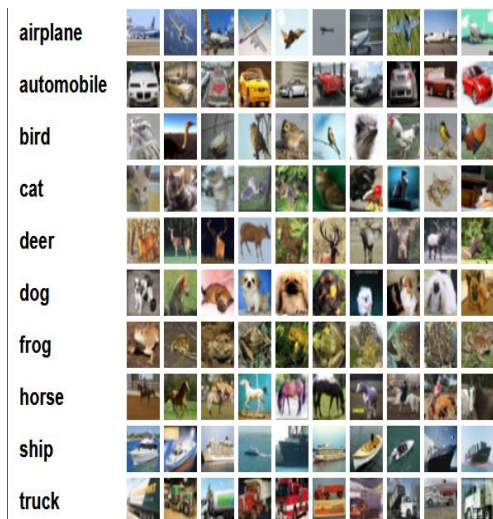
5.1 基本单元

5.2 典型卷积神经网络架构

5.3 卷积神经网络应用

5.3 卷积神经网络：应用

➤ 图像分类 image classification



classes

=

('plane',
'car',
'bird',
'cat',
'deer',
'dog',
'frog',
'horse',
'ship',
'truck')

Step1: 准备数据集，包括训练集和测试集

Step2: 设计卷积神经网络

Step3: 确定损失函数，选择优化策略

Step4: 在训练集上训练学习神经网络参数

Step5: 在测试集上测试神经网络性能

5.3 卷积神经网络：应用

➤ 图像分类 image classification

Step1: 准备数据集，包括训练集和测试集

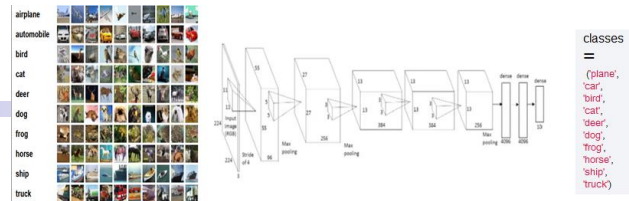
```
import torch
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

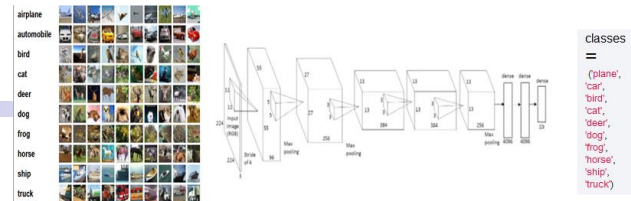
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                          shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```



5.3 卷积神经网络：应用



➤ 图像分类 image classification

Step2: 设计卷积神经网络CNN，参见本节内容

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
```

```
    def __init__(self):
```

```
        super(Net, self).__init__()
```

```
        self.conv1 = nn.Conv2d(3, 6, 5)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv2 = nn.Conv2d(6, 16, 5)
```

```
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
```

```
        self.fc2 = nn.Linear(120, 84)
```

```
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
```

```
        x = self.pool(F.relu(self.conv1(x)))
```

```
        x = self.pool(F.relu(self.conv2(x)))
```

```
        x = x.view(-1, 16 * 5 * 5)
```

```
        x = F.relu(self.fc1(x))
```

```
        x = F.relu(self.fc2(x))
```

```
        x = self.fc3(x)
```

```
        return x
```

```
net = Net()
```

卷积层 (5.1.1)

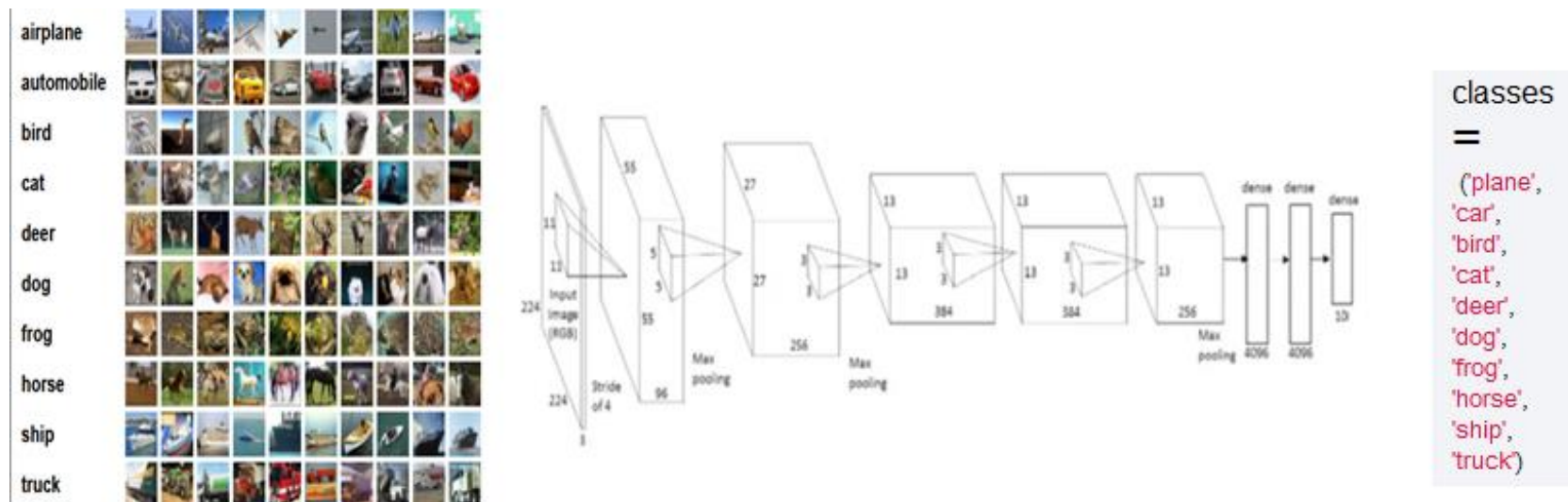
池化层 (5.1.2)

全连接层 (4.2)

激活函数 (4.2.1)

5.3 卷积神经网络：应用

➤ 图像分类 image classification



Step3: 确定损失函数，选择优化策略

```
import torch.optim as optim
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

损失函数 (4.2.3)

优化方法，参见(4.3.3)

5.3 卷积神经网络：应用

➤ 图像分类 image classification

Step4: 在训练集上训练学习神经网络参数

```
for epoch in range(2): # loop over the dataset multiple times
```

```
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
```

```
        # get the inputs
```

```
        inputs, labels = data
```

```
        # zero the parameter gradients
```

```
        optimizer.zero_grad()
```

```
        # forward + backward + optimize
```

```
        outputs = net(inputs)
```

```
        loss = criterion(outputs, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        # print statistics
```

```
        running_loss += loss.item()
```

```
    if i % 2000 == 1999: # print every 2000 mini-batches
```

```
        print('%d, %5d' % loss: %.3f' %
```

```
            (epoch + 1, i + 1, running_loss / 2000))
```

```
        running_loss = 0.0
```

```
print('Finished Training')
```

Mini-batch training (4.3.3)

```
[1, 2000] loss: 2.191
[1, 4000] loss: 1.863
[1, 6000] loss: 1.664
[1, 8000] loss: 1.581
[1, 10000] loss: 1.520
[1, 12000] loss: 1.493
[2, 2000] loss: 1.433
[2, 4000] loss: 1.406
[2, 6000] loss: 1.373
[2, 8000] loss: 1.340
[2, 10000] loss: 1.308
[2, 12000] loss: 1.314
Finished Training
```

5.3 卷积神经网络：应用

➤ 图像分类 image classification

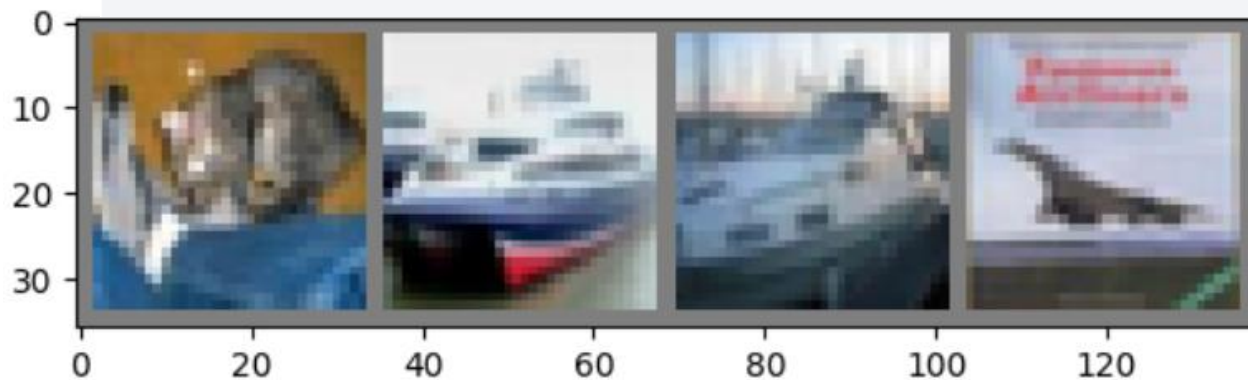
Step5: 在测试集上测试神经网络性能，统计准确率

```
dataiter = iter(testloader)
images, labels = dataiter.next()

# print images
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))

_, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
                                for j in range(4)))
```



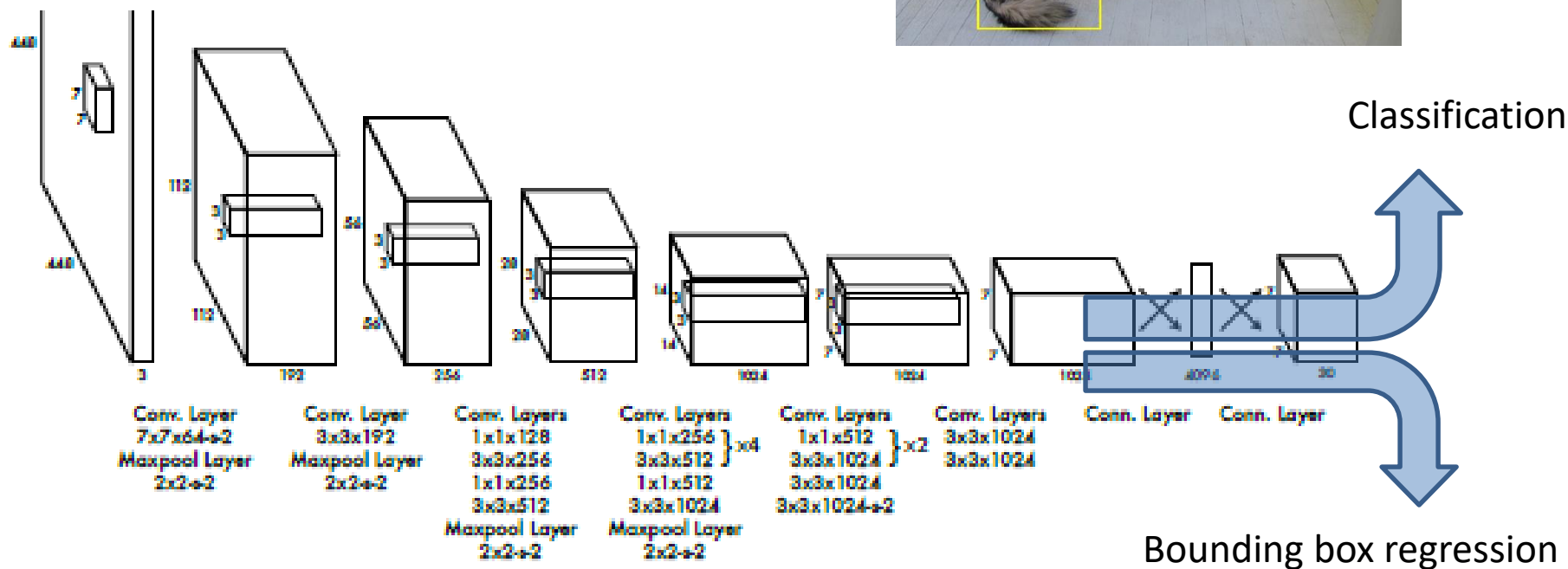
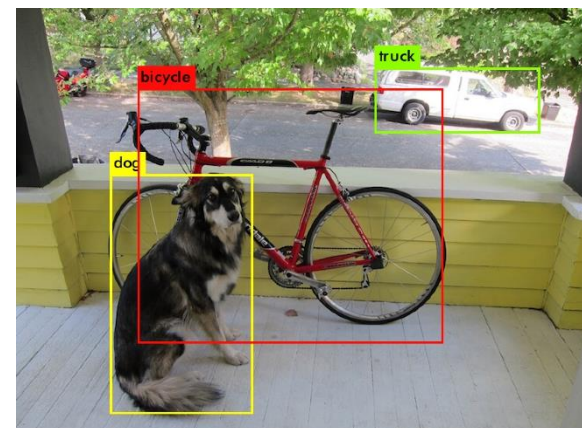
Predicted: cat ship ship ship

https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html#sphx-glr-beginner-blitz-cifar10-tutorial-py

5.3 卷积神经网络：应用

- 目标检测 object detection

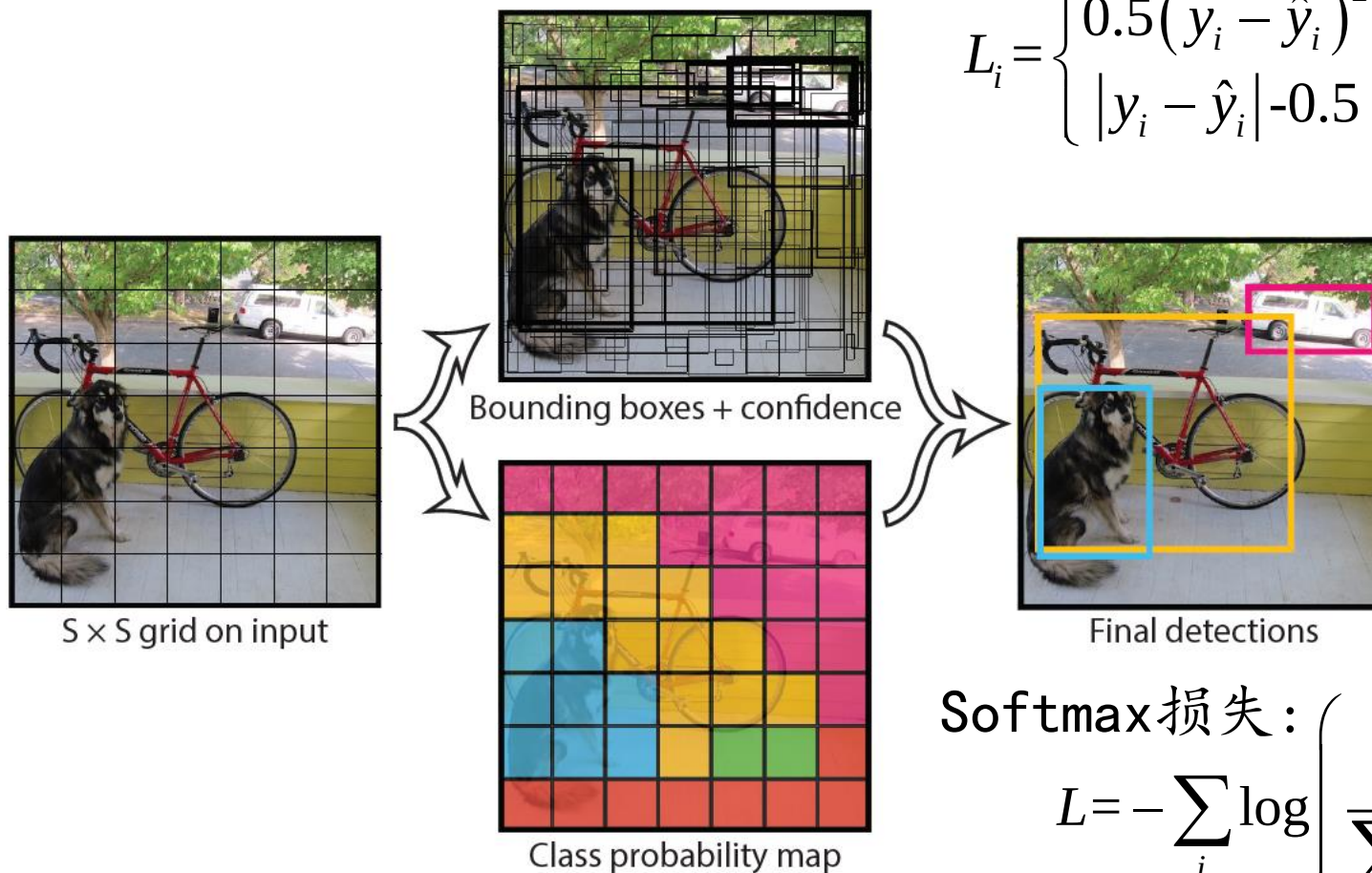
Multi-task learning
多任务学习



Redmon et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.

5.3 卷积神经网络：应用

- 目标检测 object detection



Smooth-L1 损失

$$L_i = \begin{cases} 0.5(y_i - \hat{y}_i)^2 & |y_i - \hat{y}_i| < 1 \\ |y_i - \hat{y}_i| - 0.5 & \text{otherwise} \end{cases}$$

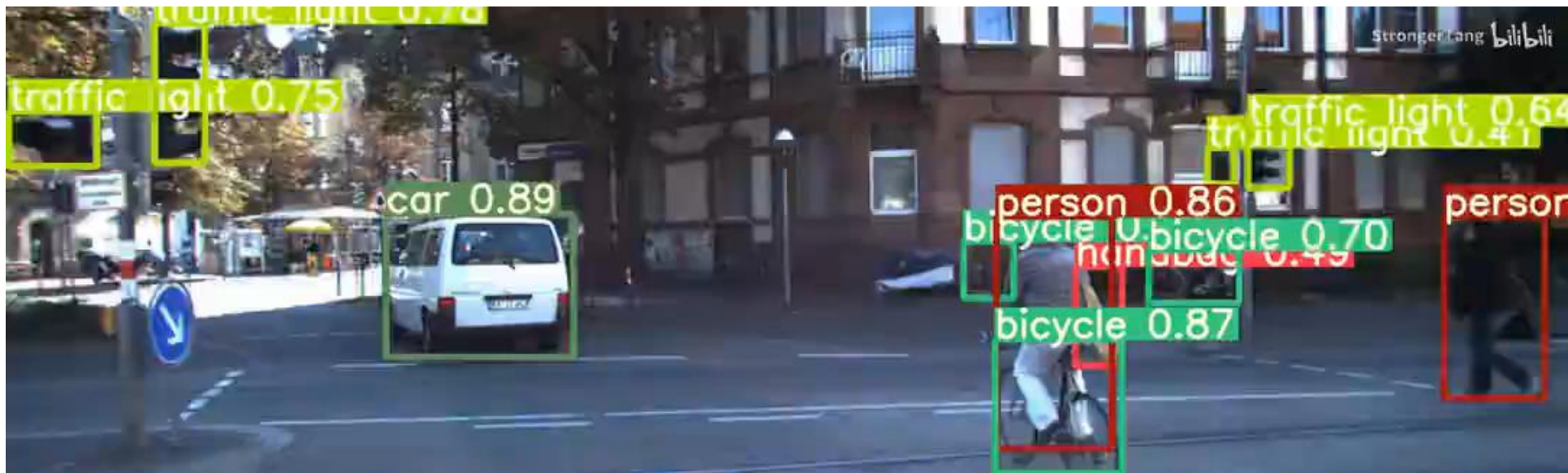
Softmax 损失:

$$L = - \sum_i \log \left(\frac{\exp(z_{i,l})}{\sum_k \exp(z_{i,k})} \right)$$

Redmon et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.

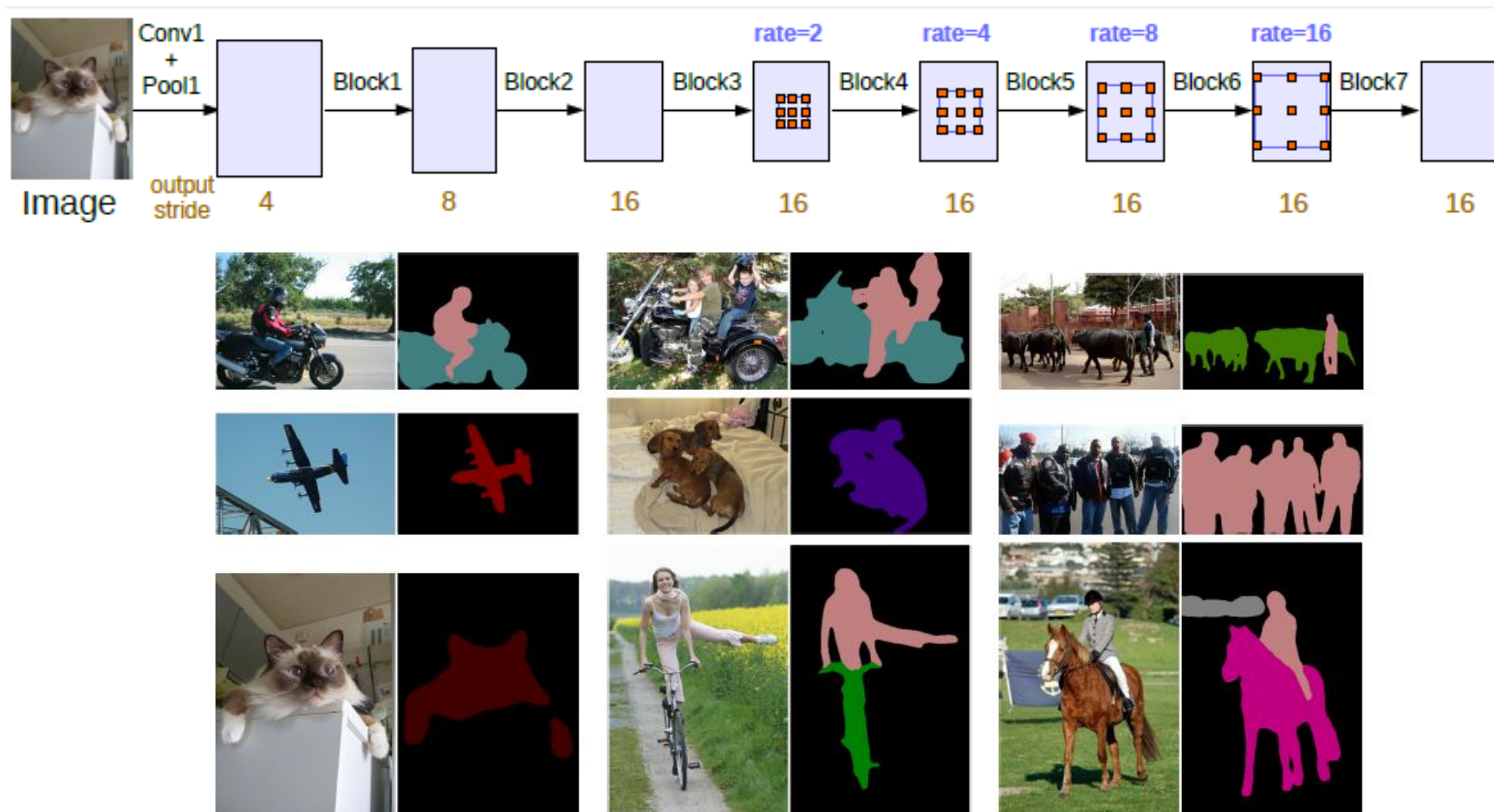
5.3 卷积神经网络：应用

- 目标检测 object detection



5.3 卷积神经网络：应用

- 语义分割 semantic segmentation

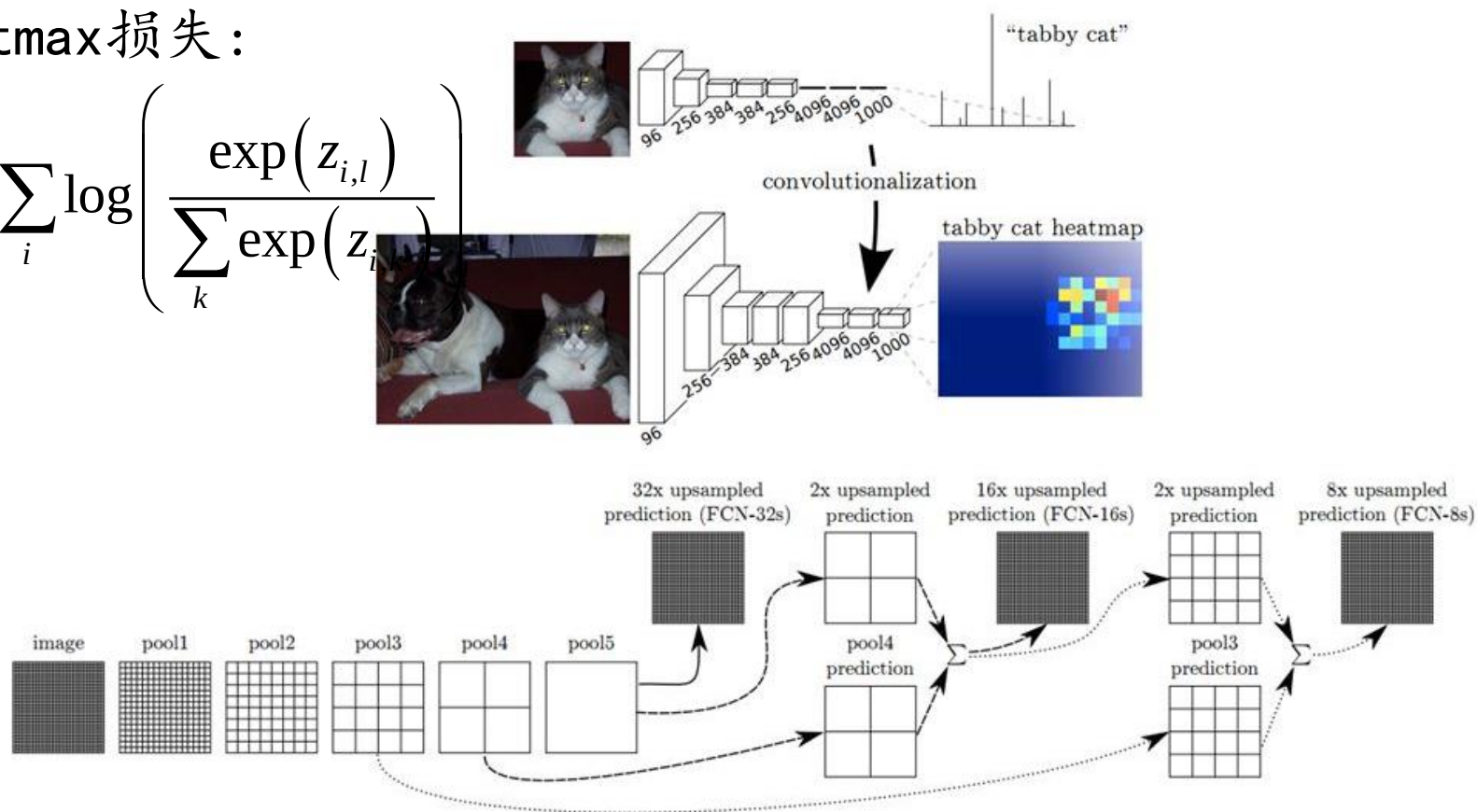


5.3 卷积神经网络：应用

- 语义分割 semantic segmentation

Softmax损失：

$$L = - \sum_i \log \left(\frac{\exp(z_{i,l})}{\sum_k \exp(z_{i,k})} \right)$$



5.3 卷积神经网络：应用

- 语义分割 semantic segmentation

ICNet for Real-Time Semantic Segmentation on High-Resolution Images

Hengshuang Zhao¹ Xiaojuan Qi¹ Xiaoyong Shen¹ Jianping Shi² Jiaya Jia¹

¹The Chinese University of Hong Kong ²SenseTime Group Limited

*Each frame in the video is processed independently at the rate of 30 fps on a 1024*2048 resolution image.*

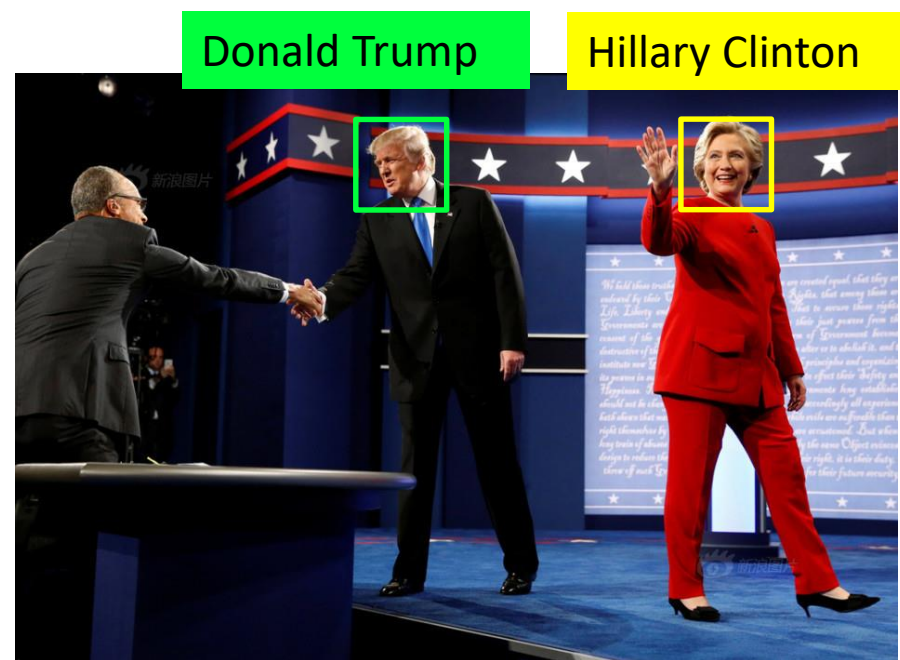
5.3 卷积神经网络：应用

- 语义分割 semantic segmentation



5.3 卷积神经网络：应用

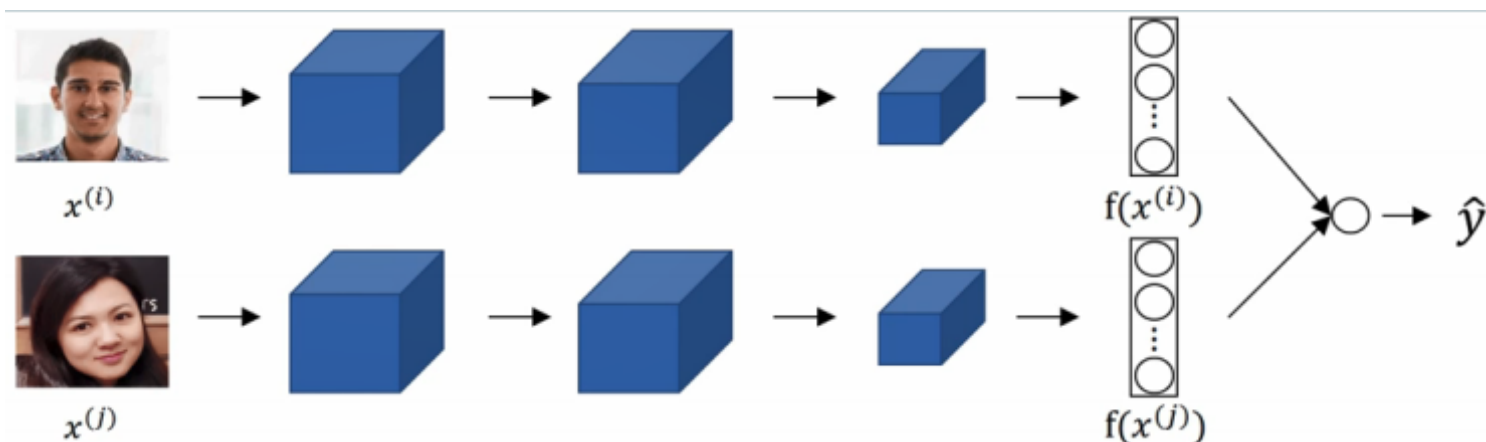
- 人脸认证/识别 face verification/recognition



5.3 卷积神经网络：应用

- 人脸认证/识别 face verification/recognition

Metric learning
度量学习



代表第 i 对是否属于同一类别；例如：
同一人人脸 $y_i = 1$ ；不同人人脸 $y_i = 0$

Contrastive Loss
$$L_i = \frac{1}{2} y_i d_i^2 + (1 - y_i) \max(\text{margin} - d_i, 0)^2$$

$$d_i = \|f(\mathbf{x}^{(i)}) - f(\mathbf{x}^{(j)})\|$$
 代表第 i 对特征的距离

Y Sun et al. Deep Learning Face Representation from Predicting 10,000 Classes. CVPR, 2014.

5.3 卷积神经网络：应用

➤ 应用：图像及视频分析

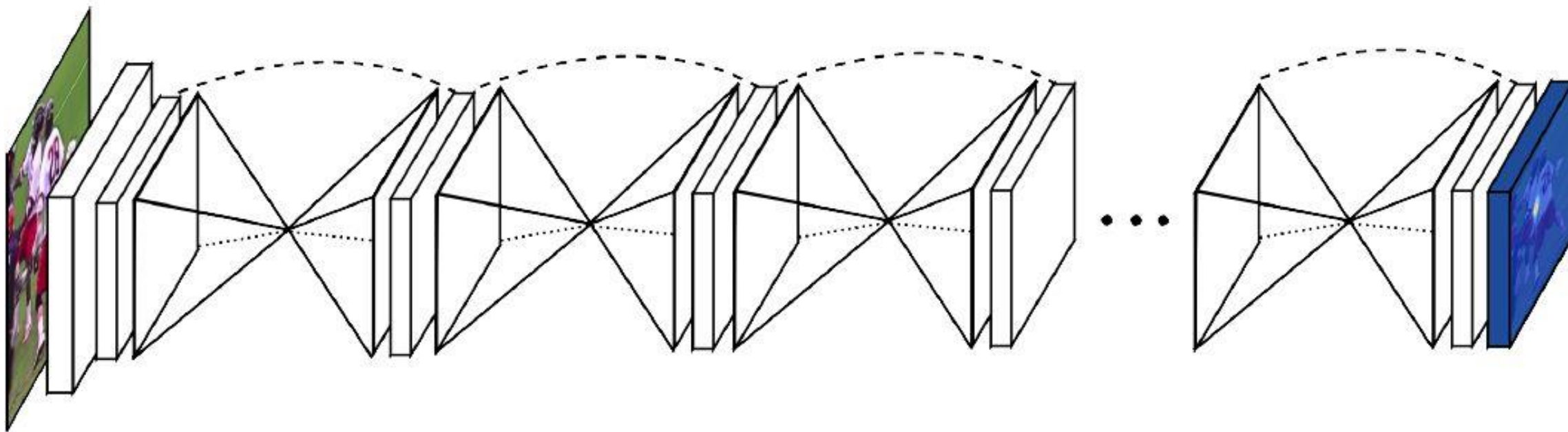


Fig. 1. Our network for pose estimation consists of multiple stacked hourglass modules which allow for repeated bottom-up, top-down inference.

5.3 卷积神经网络：应用

➤ 应用：视频监控与交互行为分析



5.3 卷积神经网络：应用

➤ 应用：视频监控与交互行为分析



5.3 卷积神经网络：应用

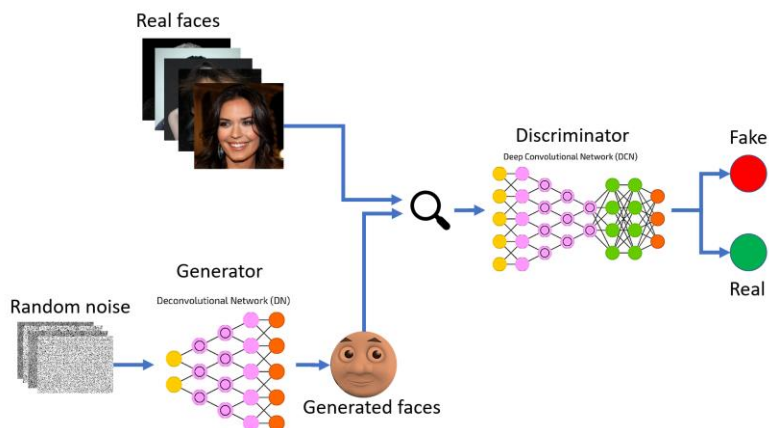
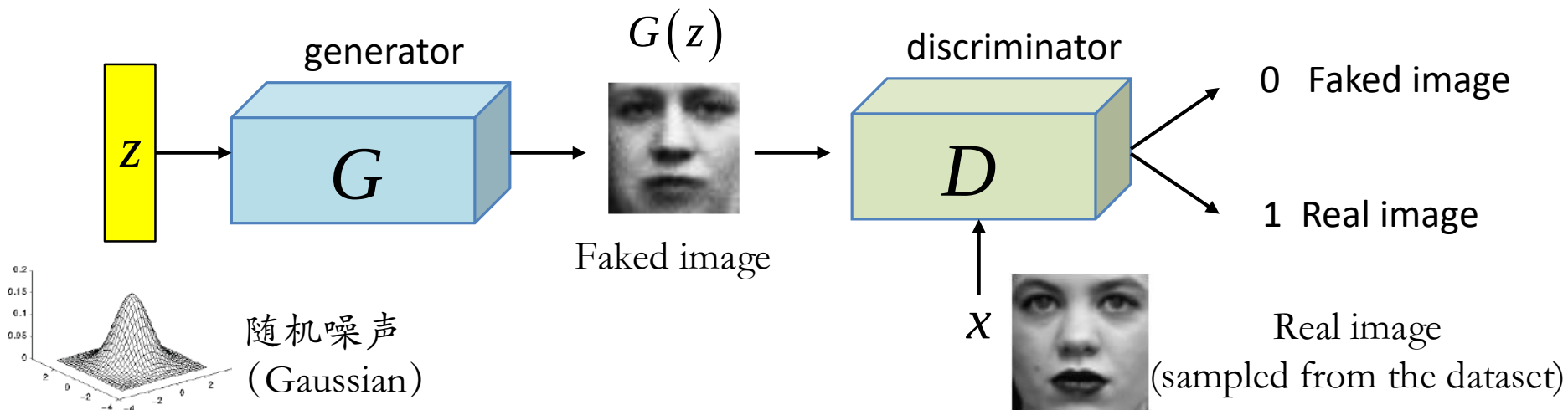
➤ 应用：视频监控与交互行为分析



5.3 卷积神经网络：应用

➤ 生成式对抗网络 GAN (Goodfellow, 2014)

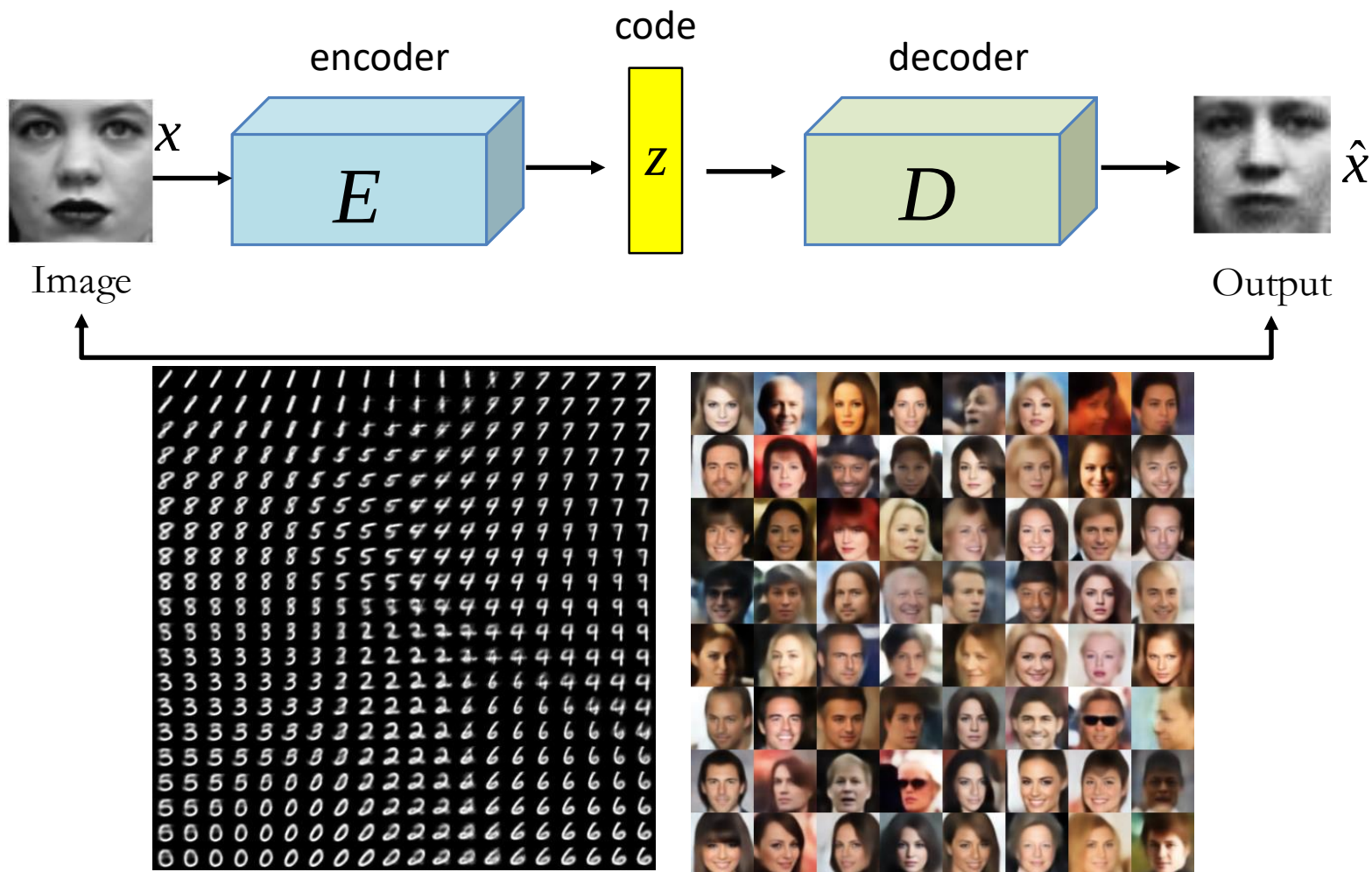
- ◆ 相互对抗的深度神经网络:生成器和判别器组成



5.3 卷积神经网络：应用

➤ 自编码器 Auto encoder (Hinton, Salakhutdinov 2006)

- ◆ 可应用于特征提取以及去噪



5.3 卷积神经网络：应用

► 应用：风格转换 Style Transfer

Content Image



This image is licensed under [CC-BY 3.0](#)

+

Style Image



Starry Night by Van Gogh is in the public domain

=

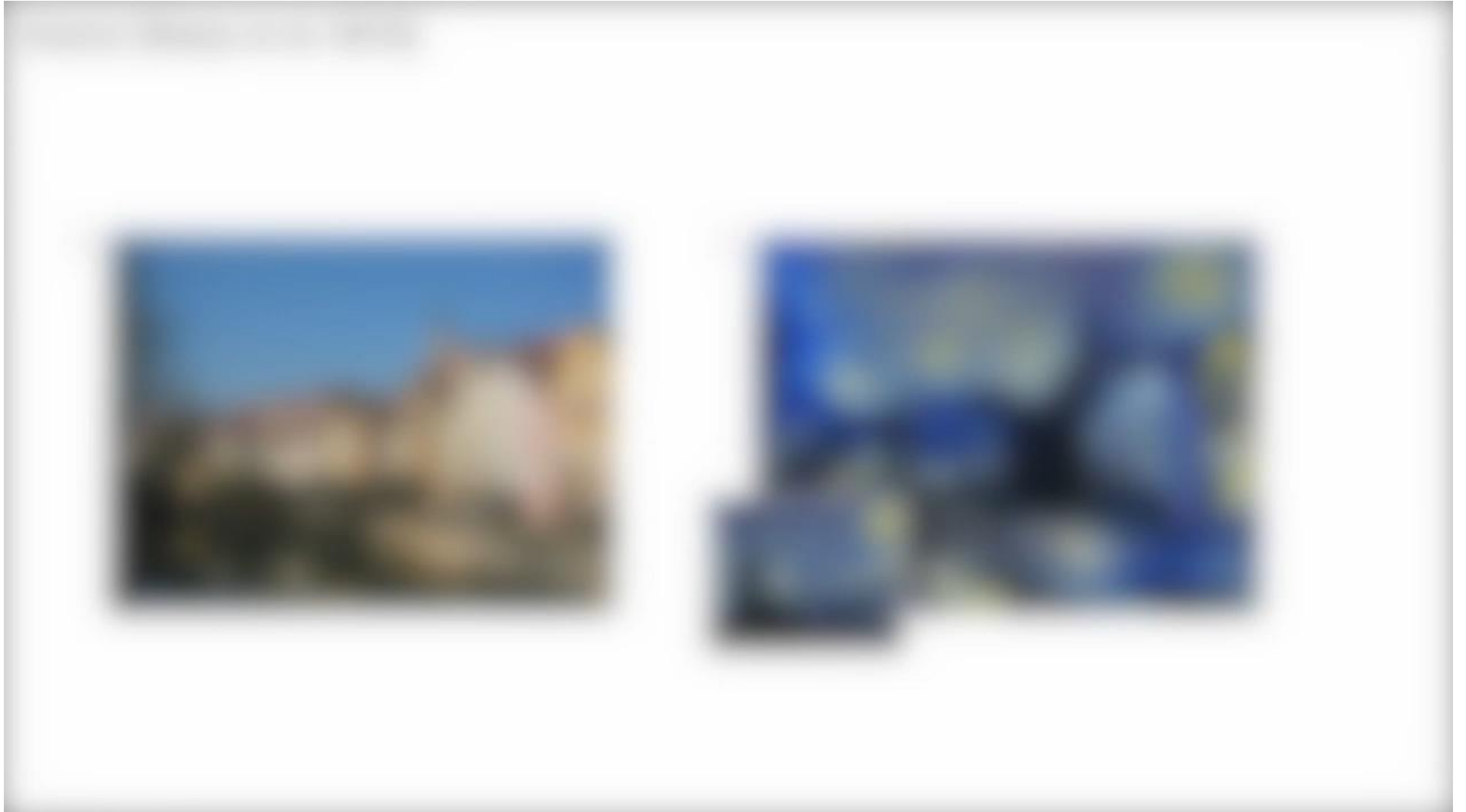
Style Transfer!



This image copyright Justin Johnson, 2015. Reproduced with permission.

5.3 卷积神经网络：应用

➤ 应用：风格转换 Style Transfer



5.3 卷积神经网络：应用

➤ 应用：风格转换 Style Transfer

@whats_ai



5.3 卷积神经网络：应用

➤ 应用：Inpainting & Object Removal

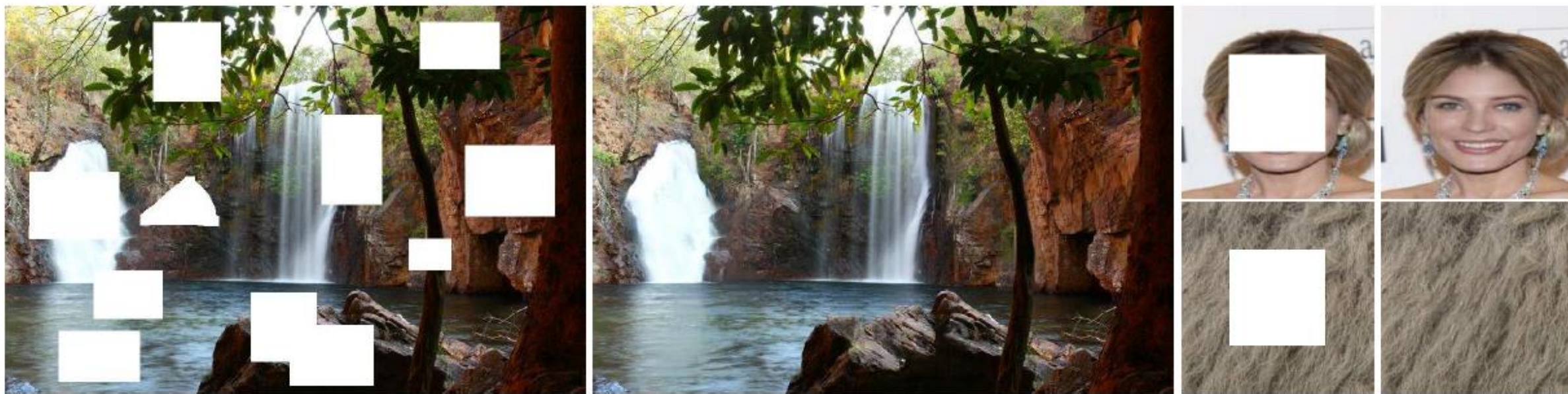
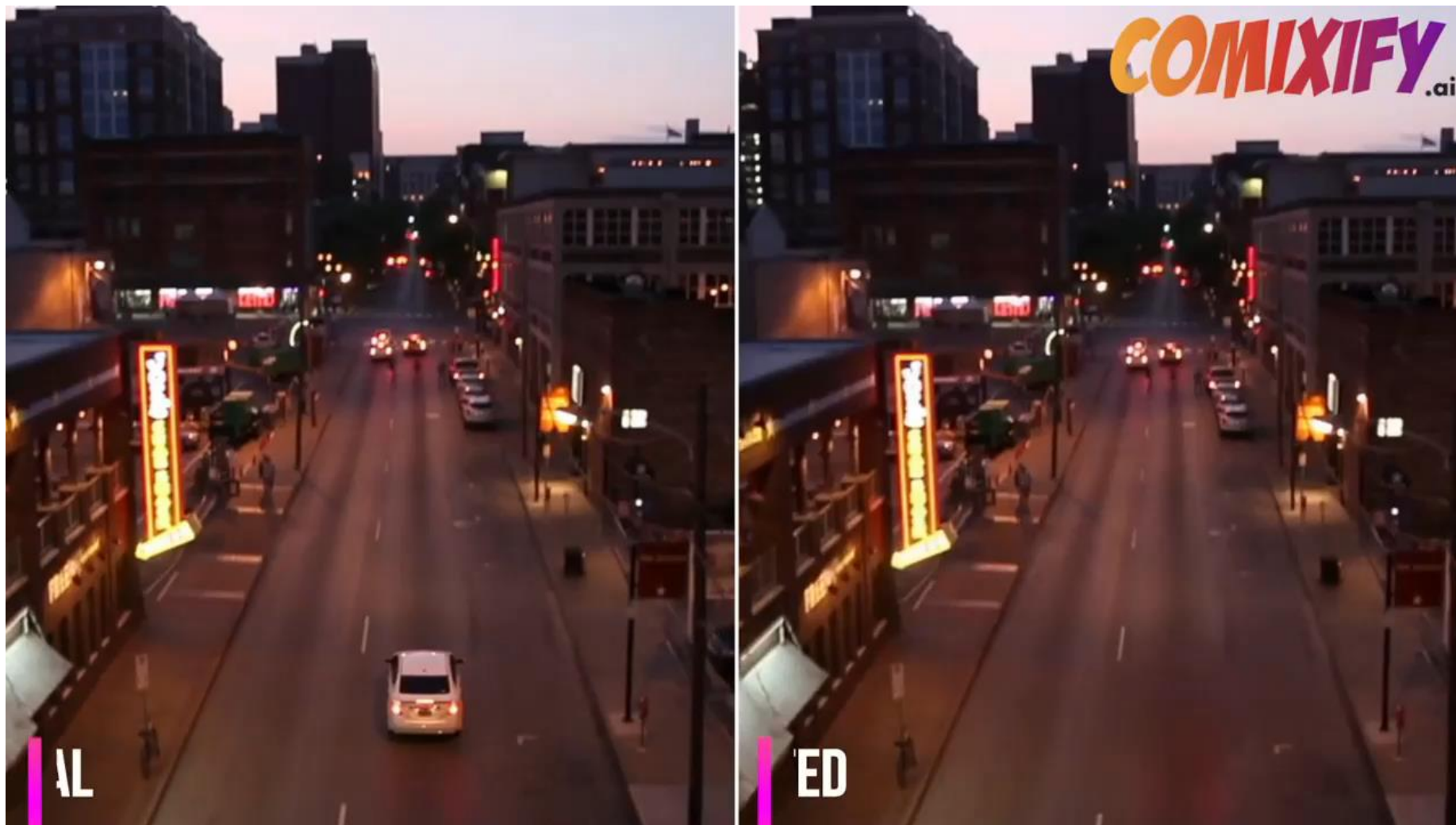


Figure 1: Example inpainting results of our method on images of natural scene, face and texture. Missing regions are shown in white. In each pair, the left is input image and right is the direct output of our trained generative neural networks without any post-processing.

5.3 卷积神经网络：应用

➤ 应用：图像/视频修复 Video Object Removal



5.3 卷积神经网络：应用

► 应用：图像/视频合成

Liquid Warping GAN with Attention: A Unified Framework for Human Image Synthesis

Wen Liu, Zhixin Piao, Zhi Tu, Wenhan Luo, Lin Ma, and Shenghua Gao

Abstract—We tackle human image synthesis, including human motion imitation, appearance transfer, and novel view synthesis, within a unified framework. It means that the model, once being trained, can be used to handle all these tasks. The existing task-specific methods mainly use 2D keypoints (pose) to estimate the human body structure. However, they only express the position information with no abilities to characterize the personalized shape of the person and model the limb rotations. In this paper, we propose to use a 3D body mesh recovery module to disentangle the pose and shape. It can not only model the joint location and rotation but also characterize the personalized body shape. To preserve the source information, such as texture, style, color, and face identity, we propose an Attentional Liquid Warping GAN with Attentional Liquid Warping Block (AtLWB) that propagates the source information in both image and feature spaces to the synthesized reference. Specifically, the source features are extracted by a denoising convolutional auto-encoder for characterizing the source identity well. Furthermore, our proposed method can support a more flexible warping from multiple sources. To further improve the generalization ability of the unseen source images, a one/few-shot adversarial learning is applied. In detail, it firstly trains a model in an extensive training set. Then, it finetunes the model by one/few-shot unseen image(s) in a self-supervised way to generate high-resolution (512×512 and 1024×1024) results. Also, we build a new dataset, namely Impersonator (IPER) dataset, for the evaluation of human motion imitation, appearance transfer, and novel view synthesis. Extensive experiments demonstrate the effectiveness of our methods in terms of preserving face identity, shape consistency, and clothes details. All codes and dataset are available on <https://impersonator.org/work/impersonator-plus-plus.html>.

Index Terms—Human Image Synthesis, Motion Imitation, Appearance Transfer, Novel View Synthesis, Generative Adversarial Network, and One-Few-Shot Learning

1 INTRODUCTION

Human image synthesis aims to make believable and photo-realistic images of humans, including motion imitation [1], [2], [3], appearance transfer [4], [5] and novel view synthesis [6], [7]. It has vast potential applications in character animation, reenactment, virtual clothes try-on, movie or game making, etc. Given a source human image and a human reference image, i) the goal of motion imitation is to generate an image with the texture from source human and pose from reference human, as depicted in the top row of Fig. 1; ii) human novel view synthesis aims to synthesize new images of the human body, captured from different viewpoints, as illustrated in the middle row of Fig. 1; iii) the goal of appearance transfer is to generate a human image preserving the source face identity while wearing the clothes of the reference, as shown in the bottom row of Fig. 1 where each garment (upper-clothes or pants) might come from different people.

Taking human motion imitation as an example, existing methods can be roughly categorized into an image-to-image translation-based [8], [9], [10] pipeline and a warping-based pipeline [11], [12], [13], [14]. The image-to-image translation-based pipeline learns a person-specific mapping function from the human conditions, characterized by a skeleton, dense pose, and parsing result, to the image from a video with paired sequences of conditions and images. Thus, everybody needs to train their model from scratch, and

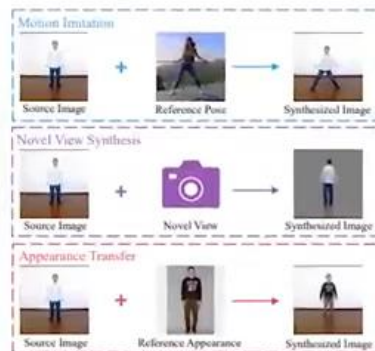


Fig. 1: Illustration of human motion imitation, novel view synthesis and appearance transfer. The 1st row is the source image and the 2nd row is reference condition, such as image or novel viewpoint of camera. The 3rd row is the synthesized results.

• Wen Liu is with School of Information Science and Technology, ShanghaiTech University, and Chinese Academy of Sciences, Shanghai Institute of Microsystem and Information Technology, and University of Chinese Academy of Sciences, China.

5.3 卷积神经网络：应用

► 应用：图像超分、视频上色、视频插帧

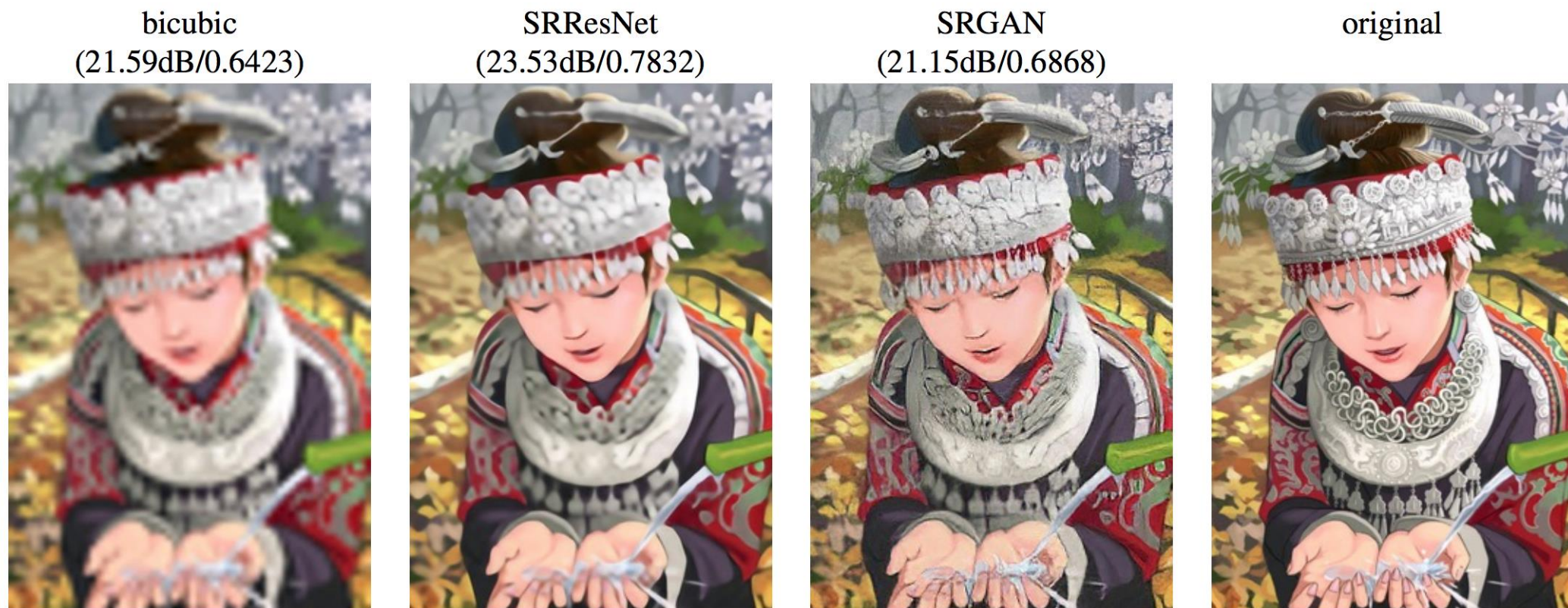
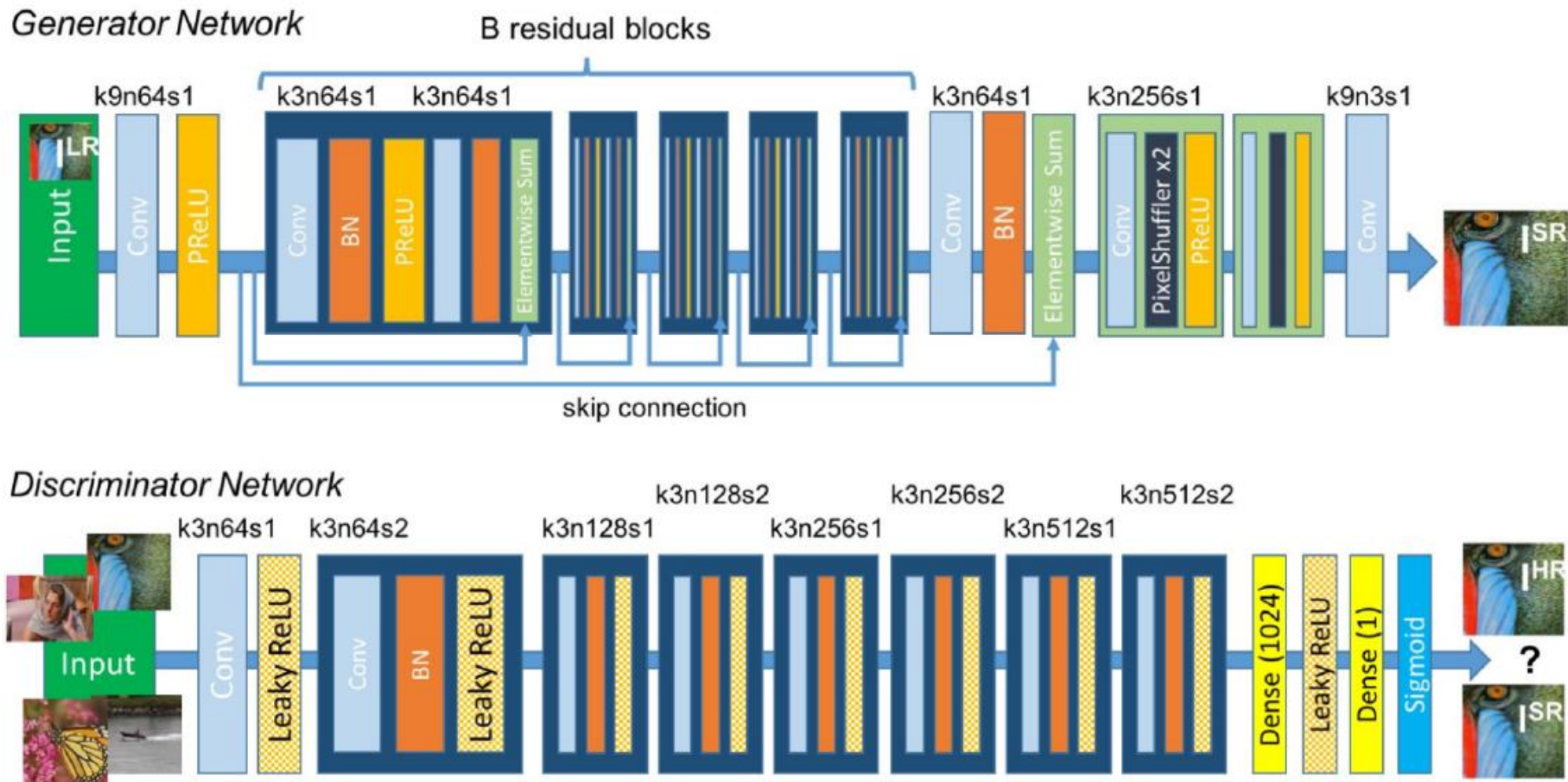


Figure 2: From left to right: bicubic interpolation, deep residual network optimized for MSE, deep residual generative adversarial network optimized for a loss more sensitive to human perception, original HR image. Corresponding PSNR and SSIM are shown in brackets. [4× upscaling]

5.3 卷积神经网络：应用

➤ 应用：图像超分、视频上色、视频插帧



5.3 卷积神经网络：应用

➤ 应用：图像超分 Super Resolution



5.3 卷积神经网络：应用

➤ 应用：视频插帧 Video Interpolation

original video(4k30)



DAIN ai interpolated video(720p@240fps)



normal speed

5.3 卷积神经网络：应用

➤ 应用：图像超分 Super Resolution



5.3 卷积神经网络：应用

➤ 应用：图像超分 Super Resolution



5.3 卷积神经网络：应用

➤ 应用：视频修复



5. 卷积神经网络(小结)

➤ 卷积神经网络CNN

- 设计思想：局部感知和权值共享
- 认知基础：视觉神经元的分层感知
- 基本单元：卷积层、池化层、全连接层、激活函数
- 卷积层和池化层超参数、参数量估计
- 卷积层拓展：空洞卷积、分组卷积、深度可分离卷积
- 卷积层拓展：可变形卷积

➤ 典型卷积神经网络结构

- AlexNet, ZFNet, VGGNet, Inception, ResNet, Dense Net, DPN
- SqueezeNet, ShuffleNet, MobileNet

➤ 卷积神经网络应用

- 图像分类，目标识别，目标检测，语义分割

Multi-task learning
多任务学习

Transfer learning
迁移学习

谢谢大家！

欢迎提出宝贵意见！

假设神经网络的某层，输入特征图大小为 $M \times W \times H$ ，输出特征图大小为 $N \times W \times H$ ：利用大小为 3×3 的深度可分离卷积（注：首先使用 M 个深度为1的 3×3 卷积获取 M 幅特征图，而后利用 N 个深度为 M 的 1×1 卷积获取输出特征图），在没有偏置的条件下卷积层所需的参数数目是

- ☐ A $9M+9N$
- ☒ B $9M+MN$
- ☐ C $9MN$
- ☐ D $9N+MN$

提交

假设神经网络的某层，输入特征图大小为 $M \times W \times H$ ，输出特征图大小为 $N \times W \times H$ ：利用大小为 3×3 的深度可分离卷积（注：首先使用 M 个深度为1的 3×3 卷积获取 M 幅特征图，而后利用 N 个深度为 M 的 1×1 卷积获取输出特征图），在有偏置的条件下卷积层所需的参数数目是

- ☐ A $9M + MN + M$
- ☒ B $10M + MN + N$
- ☐ C $10M + MN + M$
- ☐ D $9M + MN + M$

提交

卷积核尺寸为 3×3 ，步长 $S=1$ ，空洞率为2时，延拓值padding为()时满足输入与输出特征图长宽相同；

A 0

B 1

C 2

D 4

提交

VGGNet中，使用多个 3×3 卷积核的堆叠替换 11×11 卷积核，如果要求感受野一致，需要()个 3×3 卷积层。

☐ A 3

☐ B 4

☒ C 5

☐ D 6

提交